

ASIR

PROMETEO
by thePower

Tema 1: El servidor de Aplicaciones Web

Implantación de Aplicaciones Web – 2º ASIR

Olga M. Moreno Martín

10 / 09 / 2026

PROMETEO

Licencia



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

Olga M. Moreno Martín

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

PROMETEO

SECCIÓN 1

Introducción a la Implantación de Aplicaciones

01

01

Introducción

¿En qué se diferencia una aplicación instalada localmente de una que se utiliza a través del navegador?

¿Cómo se estructuran internamente las aplicaciones web actuales?

¿Qué modelos arquitectónicos y protocolos permiten su funcionamiento eficiente y seguro?

Características de las aplicaciones de escritorio



Instalación local

El usuario debe descargar e instalar el software, lo que implica consumo de espacio en disco y, en algunos casos, permisos de administrador.



Actualización manual

Las nuevas versiones deben instalarse manualmente. Si no se actualiza, pueden mantenerse errores o vulnerabilidades.



Independencia de conexión

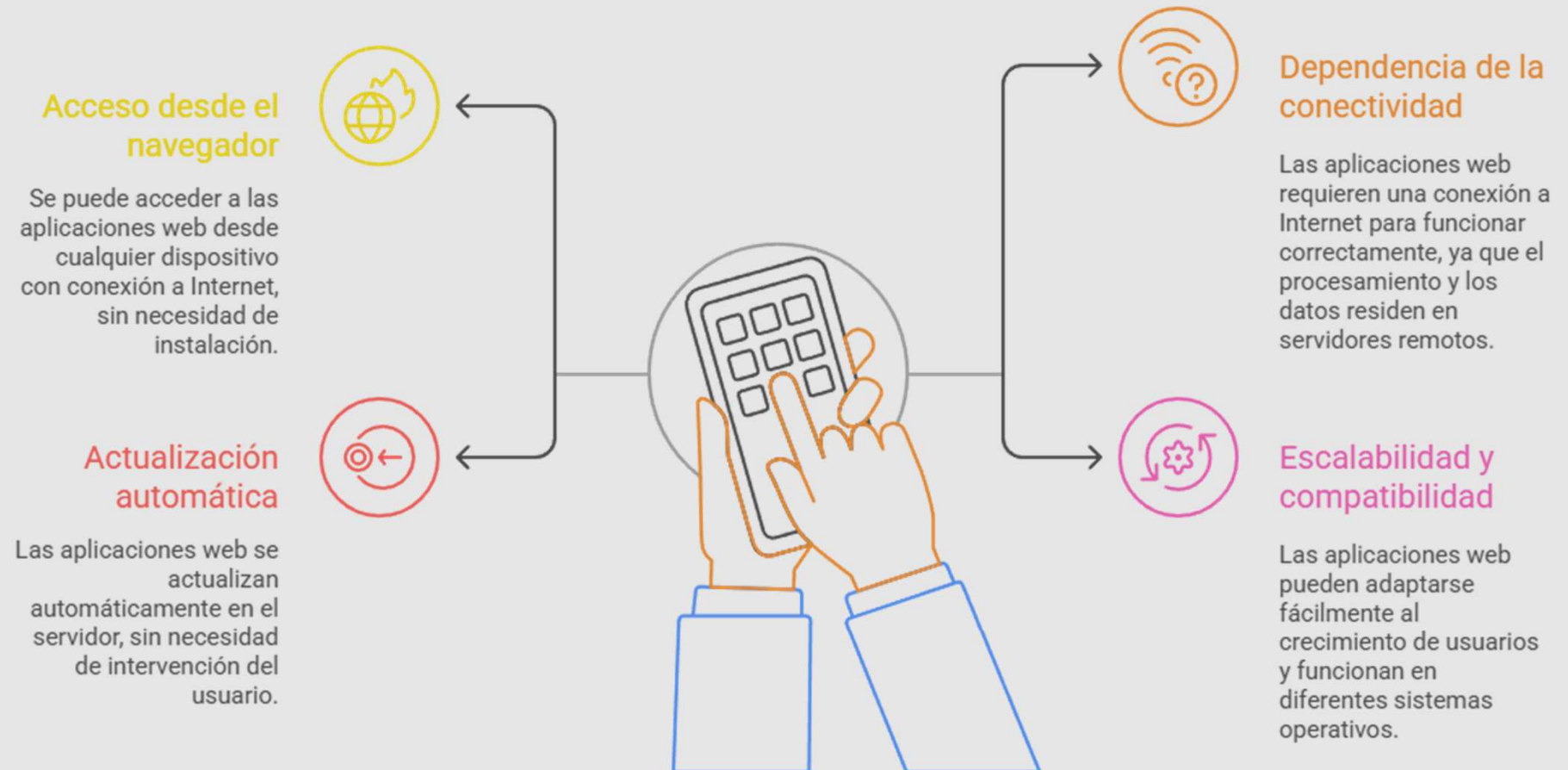
Funcionan sin acceso a Internet, lo que resulta útil en entornos desconectados.



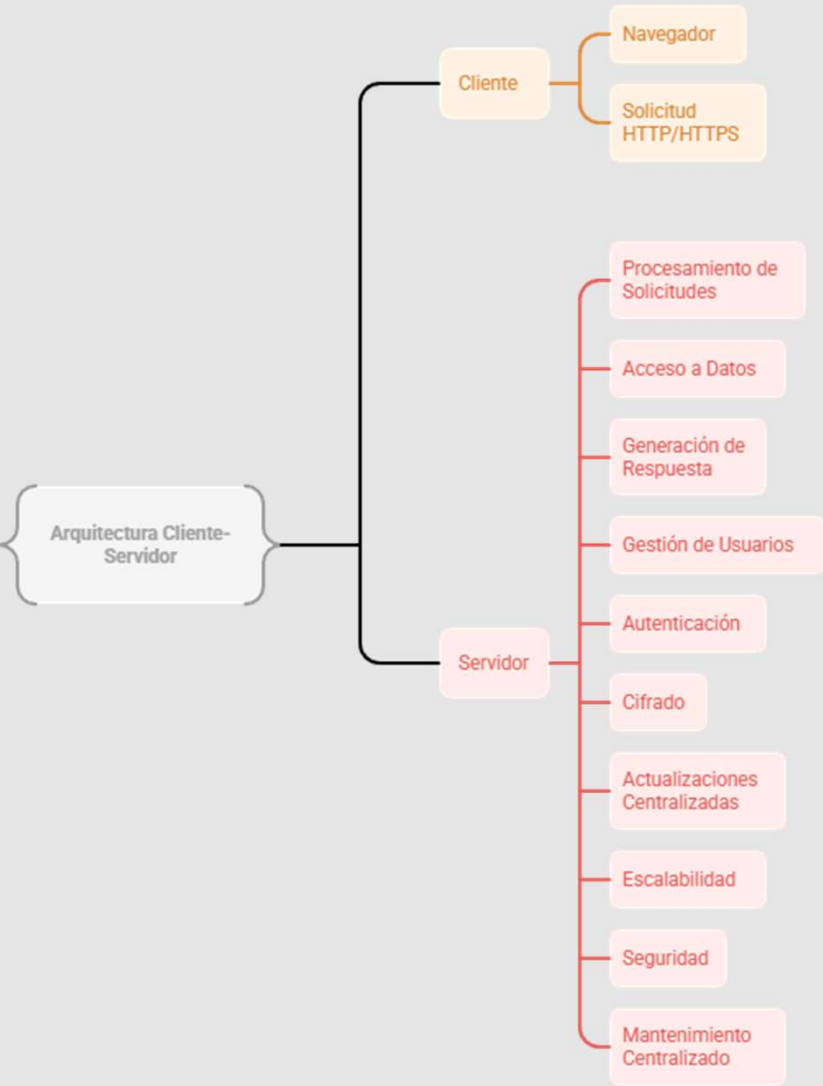
Consumo de recursos

Requieren memoria, CPU y otros recursos del sistema, lo que puede afectar el rendimiento en equipos limitados.

Características de las aplicaciones web



Arquitectura Cliente-Servidor: Componentes y Funcionamiento



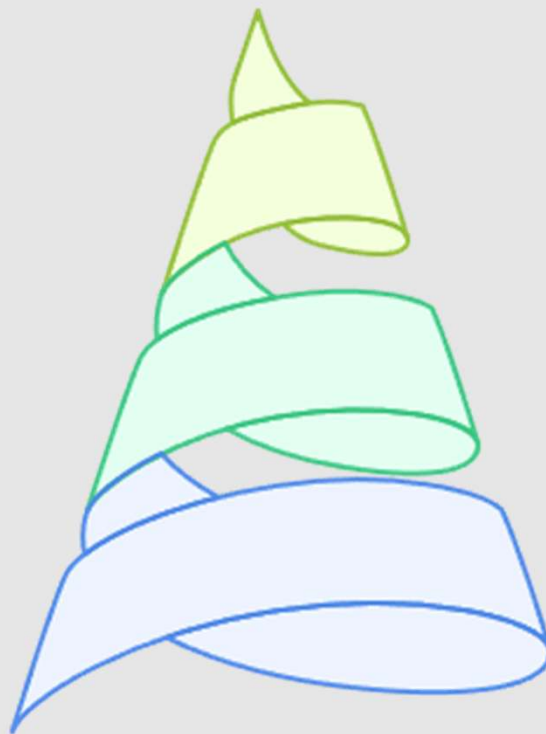
Equilibrando los beneficios y riesgos del modelo cliente-servidor



<https://www.apachefriends.org/es/download.html>



Jerarquía de Arquitectura de Tres Niveles



Capa de Presentación



Interfaz con la que interactúa el usuario

Capa de Lógica de Negocio



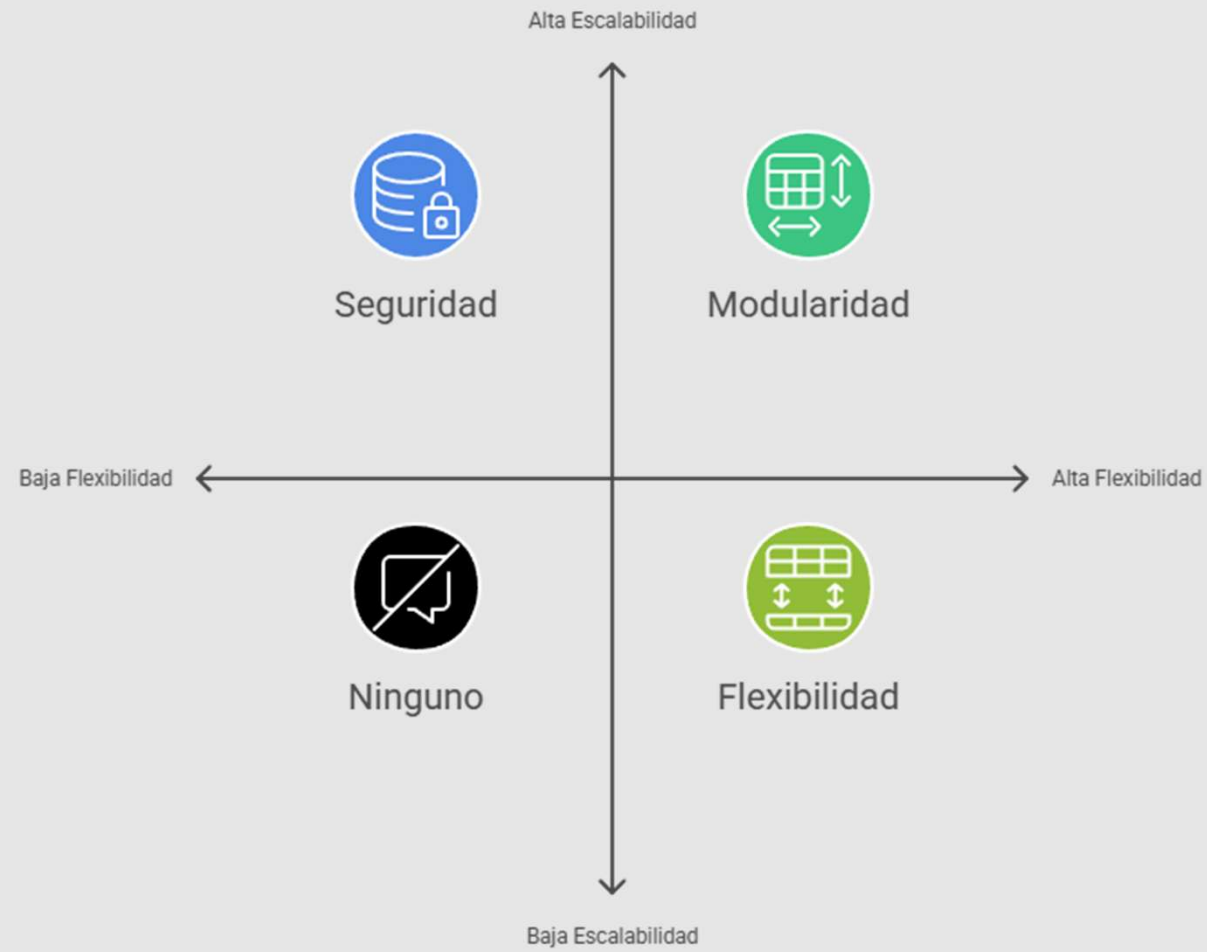
Procesa las solicitudes del usuario y aplica las reglas del sistema

Capa de Datos



Gestiona el almacenamiento y recuperación de datos

Ventajas del Modelo de Capas



PROMETEO

Secuencia de Comunicación HTTP



El cliente envía una solicitud

El navegador inicia una solicitud al servidor.

El servidor analiza y procesa la solicitud.

El servidor procesa la solicitud



El servidor responde con datos

El servidor envía los datos solicitados de vuelta al cliente.

El navegador recibe y muestra los datos.

El cliente recibe la respuesta



El servidor mantiene la independencia

El servidor no guarda información sobre solicitudes anteriores.

El servidor indica el resultado de la solicitud.

El servidor proporciona códigos de estado



Características principales de HTTPS



Cifrado de la información

Evita que los datos puedan ser leídos por terceros durante su transmisión.

Utiliza certificados digitales para asegurar que el cliente se conecta al servidor correcto.

Autenticación del servidor



Protección de integridad

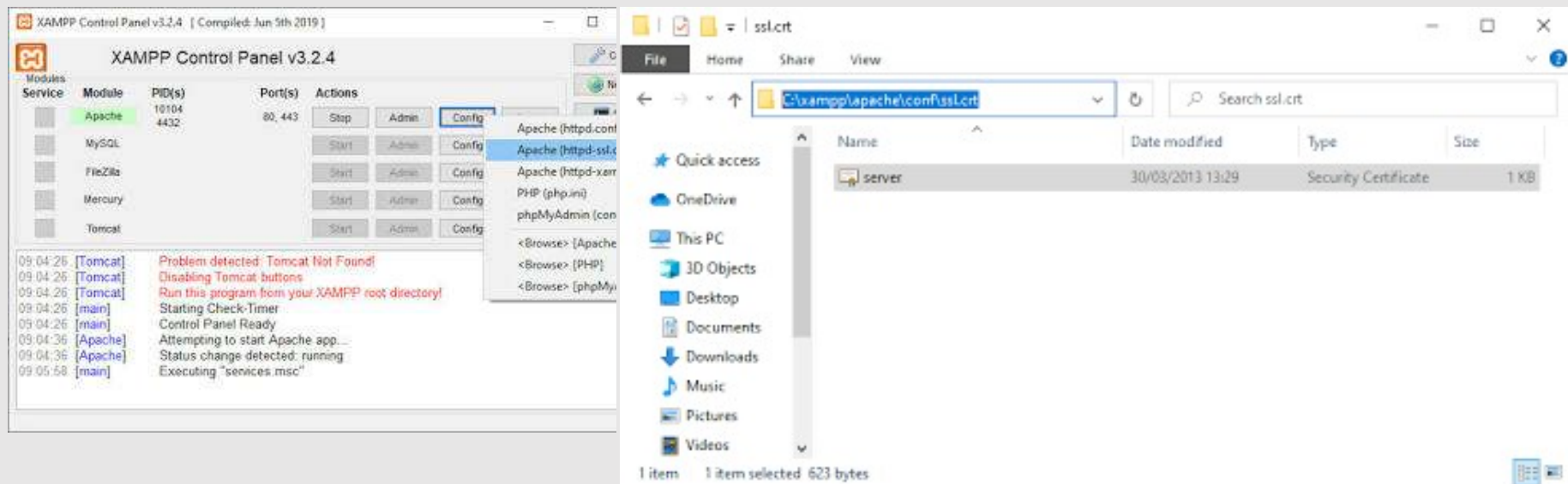
Asegura que los datos no han sido modificados durante el tránsito.

<https://letsencrypt.org/es/>

Certificados SSL gratuitos

PROMETEO

Dónde poner el archivo



SSL – HTTPS

- Para poner un certificado SSL en XAMPP, crea una carpeta para los archivos SSL, genera o descarga los archivos del certificado y la clave privada, colócalos en esa carpeta, edita el archivo de configuración de Apache (`httpd-ssl.conf`) para especificar la ruta de los archivos del certificado y la clave, y finalmente, reinicia el servidor Apache para activar la configuración.

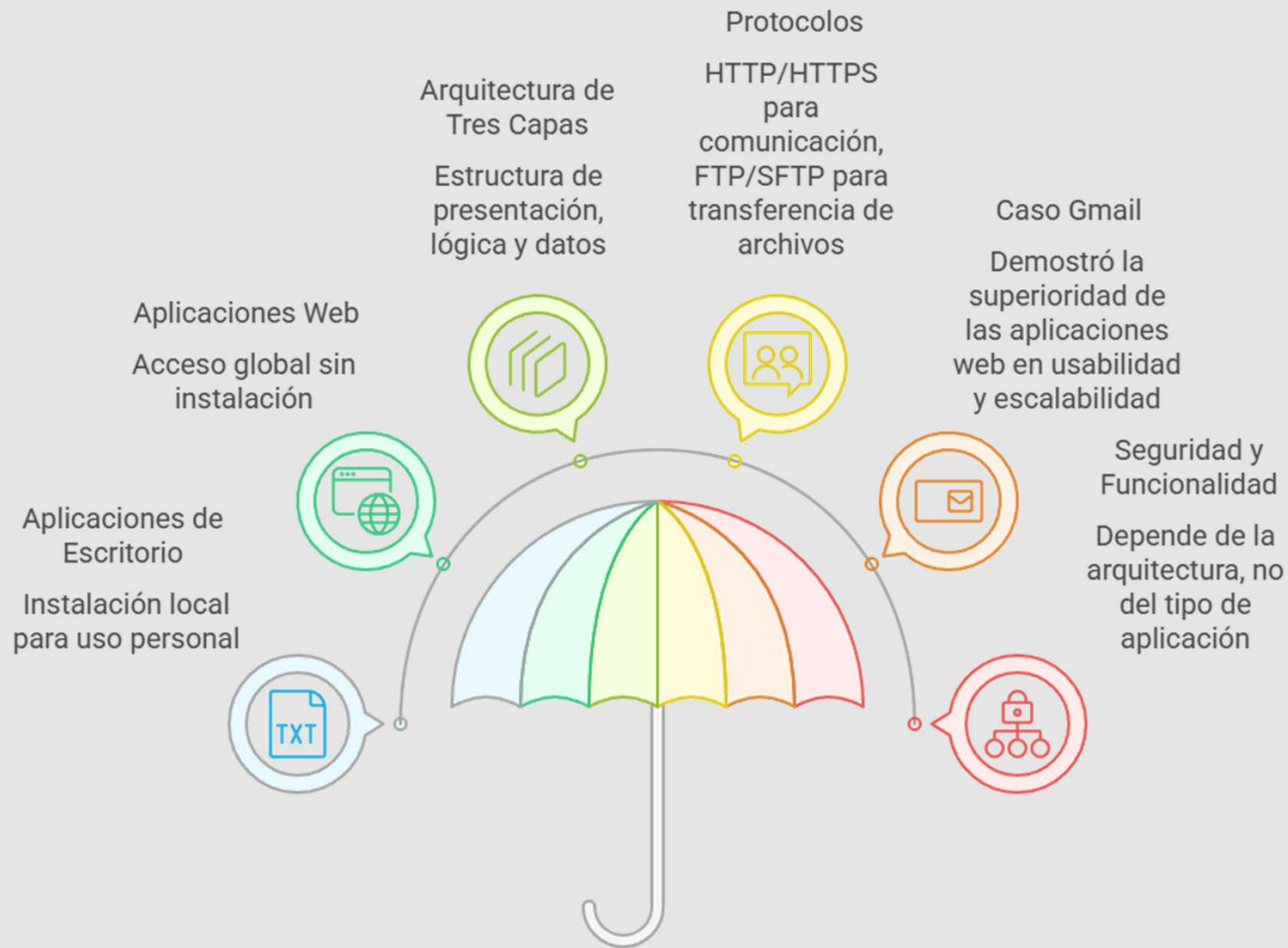
Mitos

Mito: "Las aplicaciones de escritorio son siempre más seguras que las web."

FALSO. La seguridad depende de factores como actualizaciones frecuentes, cifrado de datos y gestión de vulnerabilidades. Una aplicación de escritorio desactualizada puede ser más vulnerable que una aplicación web con parches constantes.

Mito: "Las aplicaciones web no pueden usarse sin Internet."

FALSO. Gracias a tecnologías como Progressive Web Apps (PWA) y Service Workers, muchas aplicaciones web permiten trabajar offline y sincronizar datos cuando la conexión vuelve (ejemplo: Google Docs).



PROMETEO

<https://tutorialmarkdown.com/guia>

<https://marketplace.visualstudio.com/items?itemName=bierner.markdown-mermaid>

<https://marketplace.visualstudio.com/items?itemName=shdl01wyy.markdown-preview-enhanced>

<https://marketplace.visualstudio.com/items?itemName=yzane.markdown-pdf>

<https://marketplace.visualstudio.com/items?itemName=yzhang.markdown-all-in-one>

SECCIÓN 2

Introducción a Docker

02

Contenido

1. Docker

1. ¿Qué es?
2. ¿Cómo funciona?
3. Instalación
4. DockerHub
5. Demo
6. Comandos básicos
7. Obtener información de contenedores
8. El estado de los contenedores
9. Acceder a contenedores existentes
10. Volúmenes
11. Redes
12. Despliegue orquestado



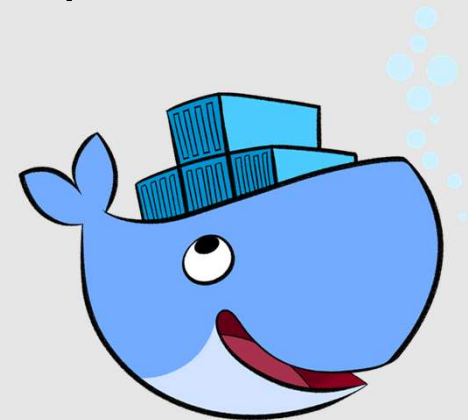
Docker: ¿Qué es?

Docker es una plataforma para ejecutar aplicaciones en unidades ligeras llamadas **contenedores**.

Los contenedores se han arraigado en el software en todas partes, desde las funciones sin servidor en la nube hasta la planificación estratégica en la empresa. Y son unidades lógicas que incluyen todo lo necesario para que el software se ejecute, incluidas bibliotecas, herramientas de sistema, código y tiempo de ejecución.

Docker se está convirtiendo en una competencia central para los operadores y desarrolladores de toda la industria TI.

“Docker es un gestor de contenedores”

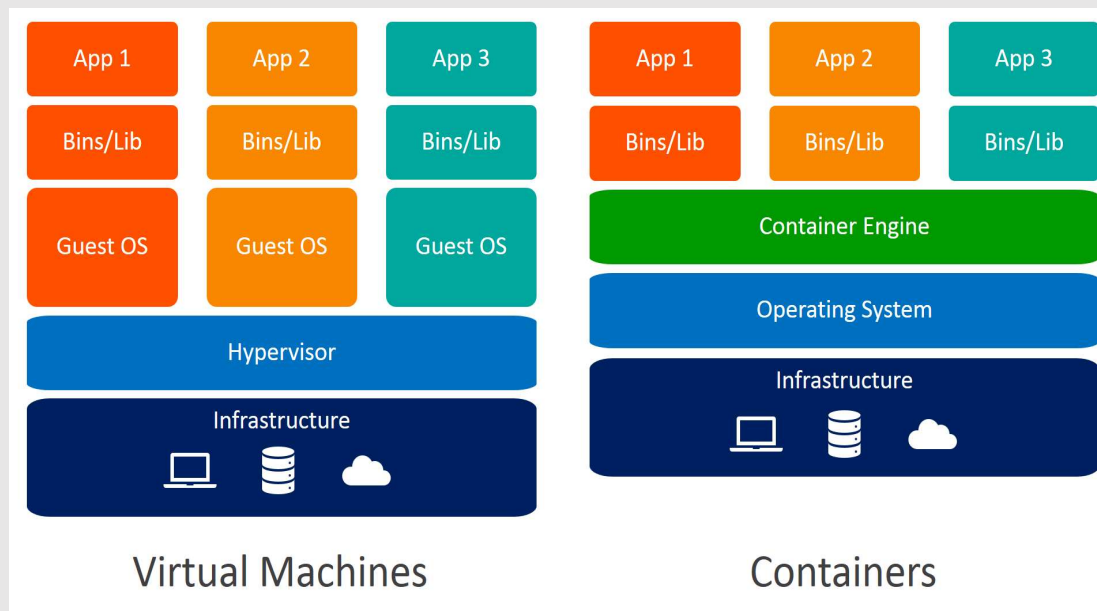


PROMETEO

Docker: ¿Cómo funciona?

De manera similar a cómo una MV virtualiza (elimina la necesidad de administrar directamente) el hardware del servidor, los contenedores virtualizan el SO de un servidor.

Docker se instala en cada servidor y proporciona comandos sencillos que puede utilizar para crear, iniciar o detener contenedores.



Docker: Instalación

Para instalar Docker en Linux (Debian/Ubuntu), lo único que tenemos que hacer es ejecutar el siguiente comando en una consola:

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
```

Y añadimos al grupo de permisos de docker nuestro usuario (si somos administradores esto no es necesario):

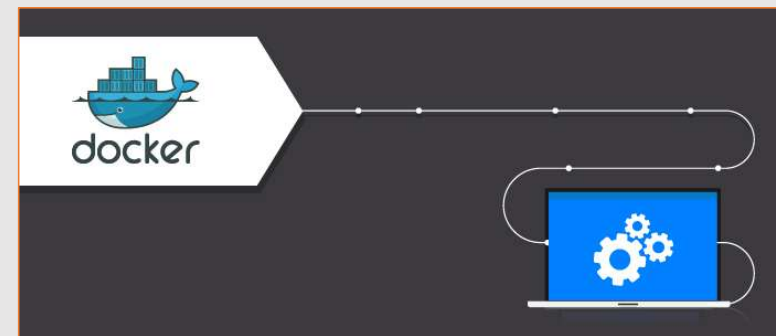
```
sudo usermod -aG docker "your-user"
```

- Instalación según las distribuciones de Linux:
<https://docs.docker.com/engine/install/ubuntu/>

Cómo instalarlo en Mac o en Windows:

- Instalación en Windows: <https://docs.docker.com/docker-for-windows/install/>
- Instalación en Mac: <https://docs.docker.com/docker-for-mac/install/>

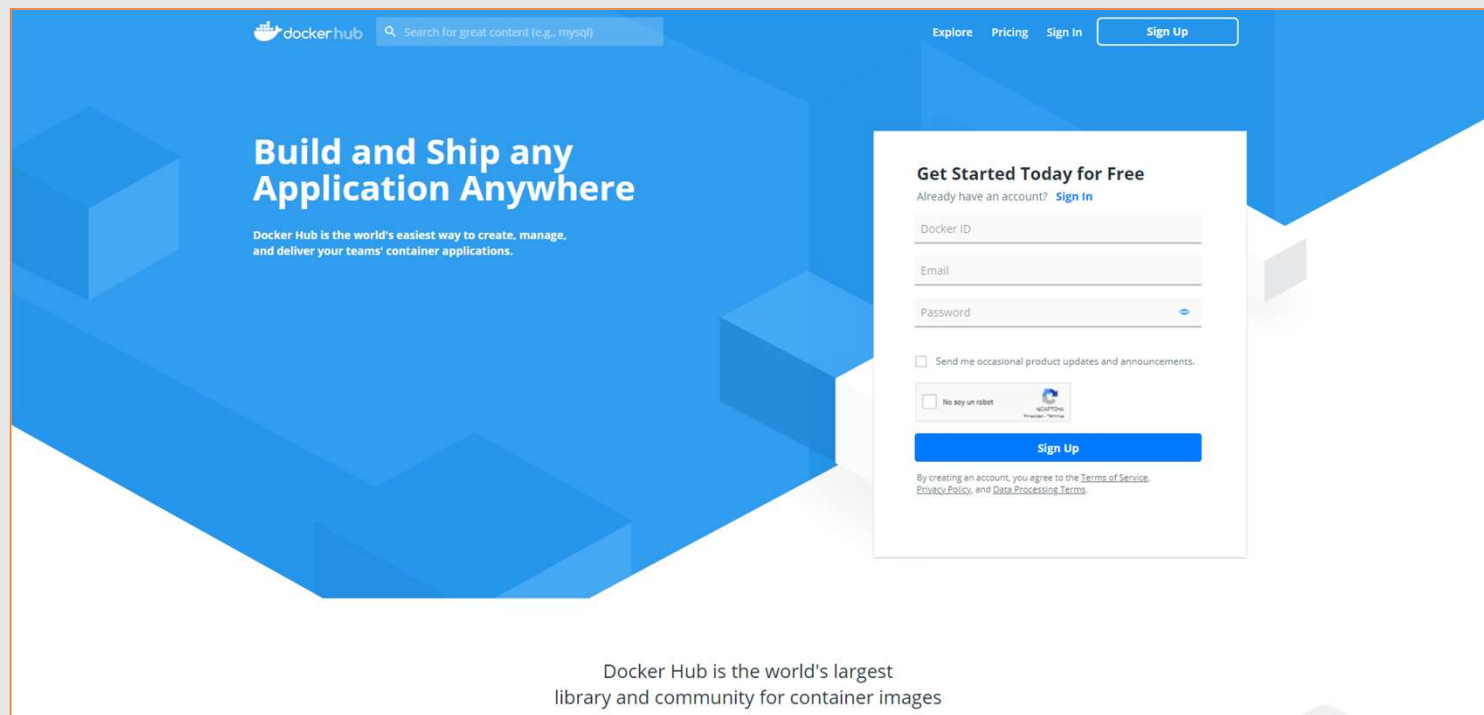
Aprovechando la instalación, vamos a instalar también docker-compose, veremos esta herramienta más adelante.
<https://docs.docker.com/compose/install/>



Docker: DockerHub

DockerHub es el repositorio de imágenes (una imagen es un objeto que contiene un sistema de archivos del SO, una aplicación y todas las dependencias de la aplicación)

<https://hub.docker.com/>



Docker

Mi primer contenedor

<https://hub.docker.com/r/misaogura/whalesay>

```

olga — -zsh — 80x24
Last login: Tue Feb  7 18:12:10 on ttys001
olga@MacBook-Pro-de-olga ~ % docker run docker/whalesay cowsay Hola Ingenieritos

WARNING: The requested image's platform (linux/amd64) does not match the detected
d host platform (linux/arm64/v8) and no specific platform was requested

-----
< Hola Ingenieritos >
-----
      \
       \
          ##
        ## ## ##      .
      ## ## ## ##    ==
    ## ## ## ##    ===
  /"oooooooooooooooo"____/ ===
 ~~~ {~~ ~~~~ ~~~ ~~~~ ~~~ ~ / ===== ~~~
     \_____o_____ /
      \      \      /
       \_____\_____/
olga@MacBook-Pro-de-olga ~ %

```

Docker: comandos básicos

- run: pone en funcionamiento un contenedor si especificamos la imagen a usar, si la imagen no está en el host, la intentará descargar de DockerHub
 - Ej.: `docker run nginx`
- ps: lista los contenedores activos, con la opción -a lista todos los contenedores, sea cuál sea su estado
 - Ej.: `docker ps` / `docker ps -a`
- stop: para el funcionamiento del contenedor especificado por nombre
 - Ej.: `docker stop silly_sammet`
- rm: elimina un contenedor especificado por nombre, para eliminar un contenedor, éste tiene que estar parado.
 - Ej.: `docker rm silly_sammet`
- images: muestra las imágenes instaladas en el host en el que nos encontramos
 - Ej.: `docker images`

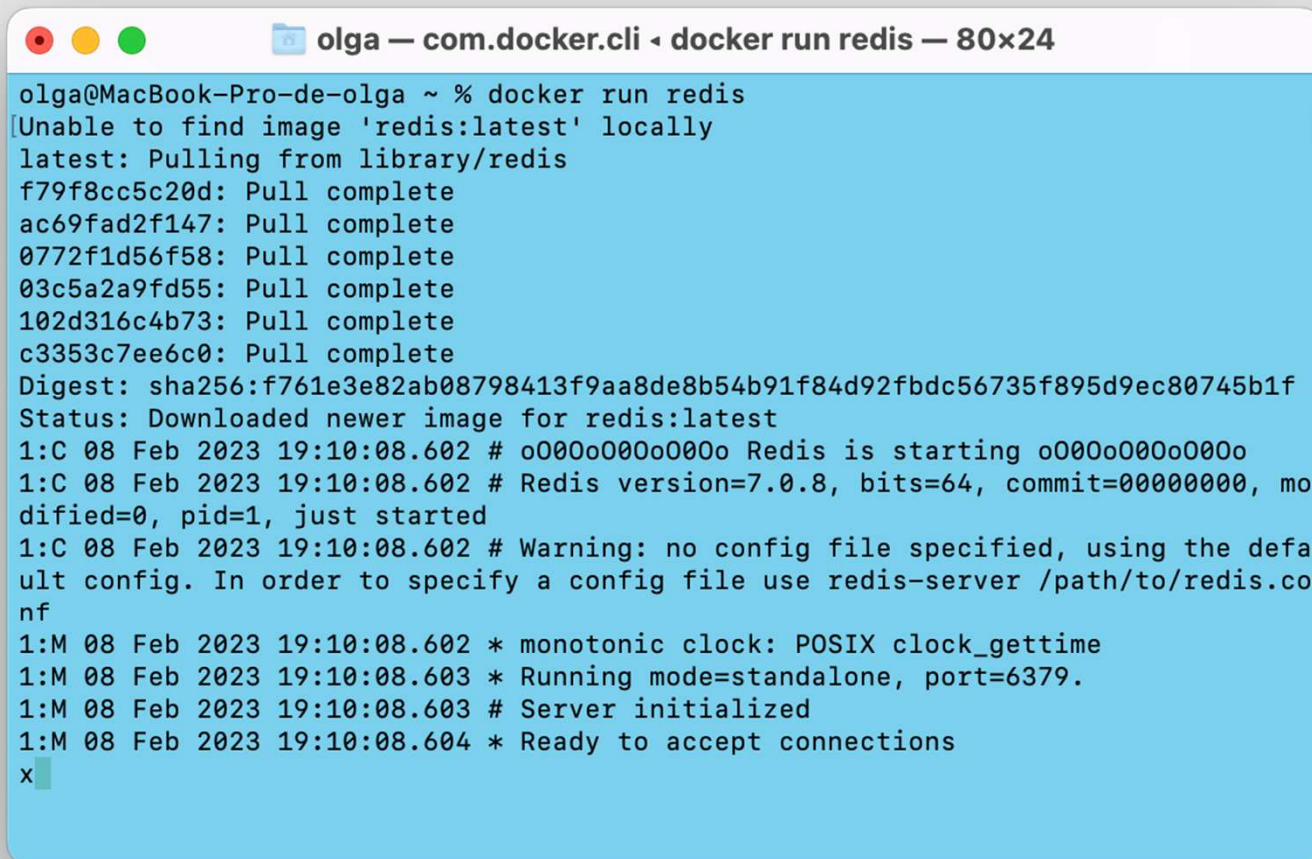
Docker: comandos básicos

- pull: descarga una imagen especificada si no está instalada en el host, por defecto conectará con el repositorio de DockerHub, pero también podemos descargar otras, aunque esto no es habitual.
 - Ej.: `docker pull ubuntu`
- exec: ejecuta una instrucción dentro del contenedor especificado por nombre
 - Ej.: `docker exec distracted_mcclintock cat /etc/hosts`
- attach y detach: la terminal se vincula a la terminal del contenedor
 - Ej: `docker run kodekloud/simple-webapp`
 - `docker run -d kodekloud/simple-webapp` (Devuelve un id)
 - `docker attach idAnterior`

Docker: el comando docker run

`docker run image:versionTag`

Por defecto la etiqueta es "latest"

A terminal window titled 'olga — com.docker.cli • docker run redis — 80x24'. The terminal shows the command 'olga@MacBook-Pro-de-olga ~ % docker run redis' being executed. The output indicates that the 'redis:latest' image was pulled from the library and then started. The Redis server logs show it is starting on port 6379 and is ready to accept connections. The terminal text is as follows:

```
olga@MacBook-Pro-de-olga ~ % docker run redis
[Unable to find image 'redis:latest' locally]
latest: Pulling from library/redis
f79f8cc5c20d: Pull complete
ac69fad2f147: Pull complete
0772f1d56f58: Pull complete
03c5a2a9fd55: Pull complete
102d316c4b73: Pull complete
c3353c7ee6c0: Pull complete
Digest: sha256:f761e3e82ab08798413f9aa8de8b54b91f84d92fdbc56735f895d9ec80745b1f
Status: Downloaded newer image for redis:latest
1:C 08 Feb 2023 19:10:08.602 # o000o000o000o Redis is starting o000o000o000o
1:C 08 Feb 2023 19:10:08.602 # Redis version=7.0.8, bits=64, commit=00000000, modified=0, pid=1, just started
1:C 08 Feb 2023 19:10:08.602 # Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
1:M 08 Feb 2023 19:10:08.602 * monotonic clock: POSIX clock_gettime
1:M 08 Feb 2023 19:10:08.603 * Running mode=standalone, port=6379.
1:M 08 Feb 2023 19:10:08.603 # Server initialized
1:M 08 Feb 2023 19:10:08.604 * Ready to accept connections
x
```


Docker: el comando docker run

`docker run image:versionTag` Por defecto la etiqueta es “latest”

Ejemplo 1: Ejecutar una versión determinada en un contenedor:

`docker run redis:6.0.9`

Ejemplo 2: Ejecutar la shell dentro de un nuevo contenedor:

`docker run -i -t debian /bin/bash`

Ejemplo 3: Ejecutar un shell en un nuevo contenedor con nombre de host “mydebian” y eliminar el directorio bin del SO

`docker run -h mydebian -i -t debian /bin/bash`

`rm -R /bin`

Docker: obtener información de contenedores

¿Cómo visualizar la dirección IP de un contenedor?

a) Iniciar un nuevo contenedor (los que están detenidos no tienen IP)

```
docker run -i -t debian /bin/bash
```

b) Iniciar un nuevo terminal y visualizar la lista de contenedores

```
docker -ps -a
```

c) Visualizar la dirección IP del contenedor

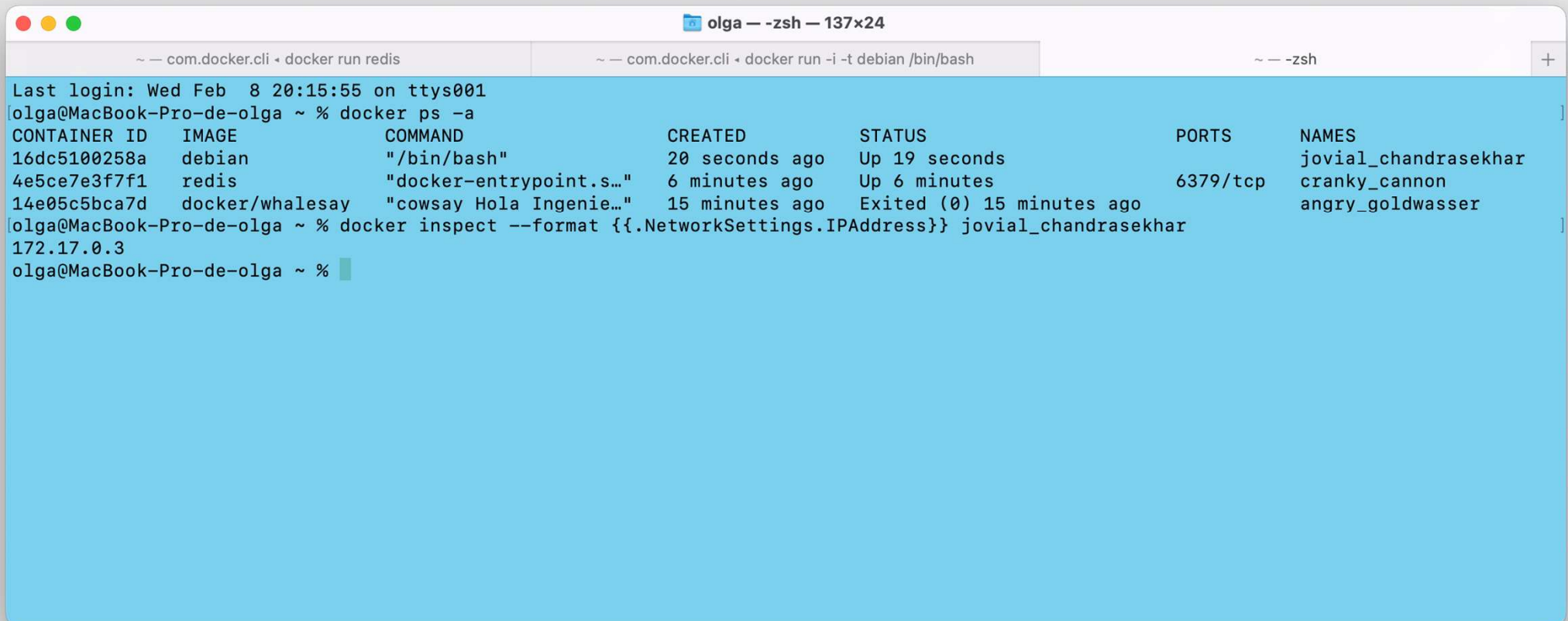
LINUX: `docker inspect --format {{.NetworkSettings.IPAddress}} <nombre o ID del contenedor>`

WINDOWS: `docker container inspect --format "{{.NetworkSettings.Networks.nat.IPAddress }}" <nombre o ID del contenedor>`

Para tener IP el contenedor tiene que estar ejecutándose.

Docker: obtener información de contenedores

¿Cómo visualizar la dirección IP de un contenedor?



The screenshot shows a terminal window with three tabs. The first tab, titled 'com.docker.cli - docker run redis', shows the command 'docker run redis' and its output. The second tab, titled 'com.docker.cli - docker run -i -t debian /bin/bash', shows the command 'docker run -i -t debian /bin/bash' and its output. The third tab, titled '-zsh', shows the command 'docker ps -a' and its output, which is a table of running containers. Below the table, the command 'docker inspect --format {{.NetworkSettings.IPAddress}} jovial_chandrasekhar' is shown, followed by the output '172.17.0.3'.

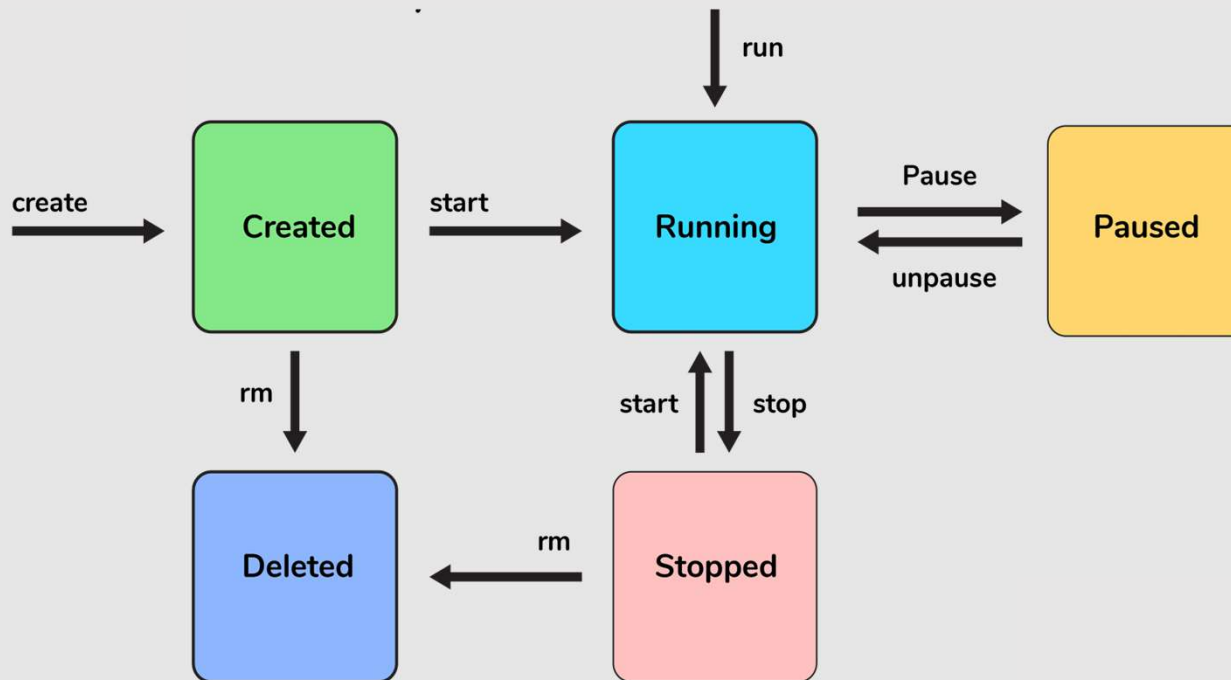
```
Last login: Wed Feb  8 20:15:55 on ttys001
olga@MacBook-Pro-de-olga ~ % docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
16dc5100258a   debian        "/bin/bash"            20 seconds ago Up 19 seconds          jovial_chandrasekhar
4e5ce7e3f7f1   redis         "docker-entrypoint.s..." 6 minutes ago  Up 6 minutes          cranky_cannon
14e05c5bca7d   docker/whalesay "cowsay Hola Ingenie..." 15 minutes ago Exited (0) 15 minutes ago angry_goldwasser
olga@MacBook-Pro-de-olga ~ % docker inspect --format {{.NetworkSettings.IPAddress}} jovial_chandrasekhar
172.17.0.3
olga@MacBook-Pro-de-olga ~ %
```

Para tener IP el contenedor tiene que estar ejecutándose. PROMETEO

Docker: estados de los contenedores

Un contenedor puede estar en varios estados

- Created
- Restarting
- Running
- Paused
- Exited (stopped)



Docker: estados de los contenedores

Un contenedor puede estar en varios estados

• Created Restarting Running Paused Exited (stopped)

- Un contenedor en **created** ha sido creado mediante el comando `create` pero aún no ha sido iniciado (no está en ejecución)
- El estado **restarting** es infrecuente y se da sólo cuando Docker intenta reiniciar un contenedor el falla en el intento
- El estado **running** lo tienen los contenedores que se encuentran en ejecución y tienen procesos activos en su interior
- El estado **paused** indica un contenedor cuya actividad está suspendida temporalmente
- El estado **stopped** (también llamado **exited** indica que no hay procesos en ejecución dentro del contenedor. Este estado también está a true en contenedores en estado **created** pero un contenedor en **stopped** ha sido iniciado al menos una vez
- Un contenedor alcanza el estado **stopped** cuando su proceso principal termina. Por tanto, puede ser reiniciado mediante el comando `start` del engine de Docker
- Un contenedor en estado **stopped** no es lo mismo que una imagen, las imágenes no tienen configuración en tiempo de ejecución ni dirección IP, los contenedores en **stopped** tampoco tienen dirección IP
- Un contenedor **stopped** retiene los cambios efectuados en su configuración, en sus metadatos y en su sistema de archivos

Docker: Acceder a contenedores existentes

- El comando run de Docker crea en primer lugar una capa de lectura y escritura encima del sistema de archivos de la imagen base, a continuación inicia el contenedor.
- Utilizar el comando run es equivalente a utilizar el comando create y luego el comando start.
- Ejemplo: Reiniciar un contenedor en estado exited (equivalente para iniciar un contenedor en estado created)

```
PS C:\Users\olga3emes> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS              PORTS          NAMES
c74aaf1727e3   debian   "/bin/bash"             5 minutes ago  Exited (0) 5 minutes ago           hardcore_thompson
PS C:\Users\olga3emes> docker start hardcore_thompson
hardcore_thompson
PS C:\Users\olga3emes> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS              PORTS          NAMES
c74aaf1727e3   debian   "/bin/bash"             5 minutes ago  Up 1 second         hardcore_thompson
PS C:\Users\olga3emes> docker attach hardcore_thompson
root@c74aaf1727e3:/#
root@c74aaf1727e3:/# exit
```

Docker: Acceder a contenedores existentes

- El comando attach de Docker sirve para vincularse a un contenedor existente en ejecución, o bien a su salida o de forma interactiva mediante una terminal
- El comando exec de Docker permite ejecutar un comando puntual sobre un contenedor existente en ejecución

```
PS C:\Users\olga3emes> docker exec hardcore_thompson cat /etc/hosts
127.0.0.1      localhost
::1           localhost ip6-localhost ip6-loopback
fe00::0       ip6-localnet
ff00::0       ip6-mcastprefix
ff02::1       ip6-allnodes
ff02::2       ip6-allrouters
172.17.0.2     c74aaf1727e3
PS C:\Users\olga3emes> █
```

Ejemplo

Ejecutar un comando en un contenedor existente en ejecución contar el número de palabras de una frase usando el comando `wc -w` dentro de un contenedor basado en Linux Debian. Hay que visualizar la respuesta.

A terminal window titled "olga — -zsh — 106x5" with three colored window control buttons (red, yellow, green) on the top left. The terminal has two tabs: the first is labeled "~ — com.docker.cli ◀ docker run redis" and the second is labeled "....docker.cli ◀ docker run -i -t debian /bin/bash". The active tab shows a prompt "olga@MacBook-Pro-de-olga ~ %" followed by the command "echo 'Dos palabras' | docker exec -i jovial_chandrasekhar wc -w". The output "2" is displayed on the next line. The terminal background is light blue.

```
olga@MacBook-Pro-de-olga ~ % echo "Dos palabras" | docker exec -i jovial_chandrasekhar wc -w
2
olga@MacBook-Pro-de-olga ~ %
```


Docker: eliminar contenedores

El comando **rm** de Docker permite eliminar un contenedor existente, esté o no en funcionamiento.



```
olga@MacBook-Pro-de-olga ~ % clear

olga@MacBook-Pro-de-olga ~ % docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
16dc5100258a	debian	"/bin/bash"	2 hours ago	Up 2 hours		jovial_chandrasekhar
4e5ce7e3f7f1	redis	"docker-entrypoint.s..."	2 hours ago	Up 2 hours	6379/tcp	cranky_cannon
14e05c5bca7d	docker/whalesay	"cowsay Hola Ingenie..."	2 hours ago	Exited (0) 2 hours ago		angry_goldwasser

```
olga@MacBook-Pro-de-olga ~ % docker rm angry_goldwasser
angry_goldwasser
olga@MacBook-Pro-de-olga ~ %
```

Docker: eliminar contenedores

Ejemplo: Eliminar todos los contenedores que estén en estado exited.

```
PS C:\Users\olga3emes> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS    NAMES
PS C:\Users\olga3emes> docker run debian echo "Hola1"
Hola1
PS C:\Users\olga3emes> docker run debian echo "Hola2"
Hola2
PS C:\Users\olga3emes> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS    NAMES
414429bb6689   debian    "echo Hola2"            4 seconds ago   Exited (0) 3 seconds ago   trusting_neumann
14f529b37e15   debian    "echo Hola1"            8 seconds ago   Exited (0) 7 seconds ago   vibrant_shtern
PS C:\Users\olga3emes> docker ps -a -f status=exited
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS    NAMES
414429bb6689   debian    "echo Hola2"            18 seconds ago   Exited (0) 18 seconds ago   trusting_neumann
14f529b37e15   debian    "echo Hola1"            22 seconds ago   Exited (0) 22 seconds ago   vibrant_shtern
PS C:\Users\olga3emes> docker ps -a -f status=running
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS    NAMES
PS C:\Users\olga3emes> docker ps -a -q -f status=exited | % { docker rm $_ }
414429bb6689
14f529b37e15
PS C:\Users\olga3emes>
```

En linux:
docker rm \$(docker ps -a -q -f status=exited)

PROMETEO

Docker: eliminar contenedores

Ejecutar un nuevo contenedor y eliminar cuando termine su proceso:

```
PS C:\Users\olga3emes> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\Users\olga3emes> docker run --rm debian echo "Hola1"
Hola1
PS C:\Users\olga3emes> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\Users\olga3emes>
```

Docker: uso de DockerHub

- **Registry:** Servicio web responsable de alojar y distribuir imágenes. El registro por defecto es DockerHub. <https://hub.docker.com/explore/>
- **Repository:** Colección de imágenes relacionadas entre sí. Usualmente diferentes versiones de la misma aplicación o servicio.
- **Tag:** Identificador alfanumérico asociados a las imágenes dentro de un repositorio (ej.: 7.4, stable, latest, ...).

Experimento 1

1. Explorar las imágenes disponibles en el registro DockerHub
2. Buscar imágenes en DockerHub desde línea de comandos. Usando:

`docker search <nombreDeLaImagen>`

3. Descargar la imagen oficial de mysql
4. Visualizar imágenes descargadas.

`docker images`

Experimento 2

Objetivo: observar las diferencias entre un contenedor y su imagen.

Crearemos un microservicio para una base de datos PostgreSQL, añadiremos una tabla de datos y comprobaremos los cambios añadidos al sistema de archivos del contenedor.

1. Crear y lanzar en segundo plano un contenedor con la imagen de PostgreSQL dándole contraseña de acceso “pingu”.
2. Acceder al intérprete cliente de PostgreSQL (psql) localmente desde dentro del microservicio.
3. Crear una tabla de datos dentro de psql.
4. Salir del intérprete de PostgreSQL dejando activo el microservicio (**Ctrl+D**)
5. Con el comando `docker diff` se observan todos los cambios realizados
6. sobre el sistema de archivos de la imagen base de un contenedor. (`docker diff pg`)

```
docker run --name pg -e POSTGRES_PASSWORD=pingu -d postgres
```

```
docker exec -it pg psql --user postgres
```

```
postgres=# create table Personas(
postgres=# PersonaID int,
postgres=# LastName varchar(255),
postgres=# FirstName varchar(255),
postgres=# Address varchar(255),
postgres=# City varchar(255)^C
postgres=# create table Personas(
postgres=# PersonaID int,
postgres=# Nombre varchar(255),
postgres=# Apellidos varchar(255),
postgres=# Direccion varchar(255),
postgres=# Ciudad varchar(255)
postgres=# );
CREATE TABLE
```

```
postgres=# \d Personas;
               Table "public.personas"
  Column      |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
personaid     | integer                |           |          |
nombre        | character varying(255) |           |          |
apellidos     | character varying(255) |           |          |
direccion     | character varying(255) |           |          |
ciudad        | character varying(255) |           |          |
postgres=#
```

Docker: publicación de imágenes en DockerHub

Crear una cuenta de usuario gratuita en DockerHub.

<https://hub.docker.com/signup>

Iniciar sesión en DockerHub a través de línea de comandos.

```
PS C:\Users\olga3emes> docker login -u olga3emes -p <password>
```

Luego hacemos un “commit” de la imagen del contenedor “pg” del ejercicio anterior.

- Debemos especificar el nombre del contenedor seguido de username/nombreImagen. La opción -a es para especificar el autor del commit.

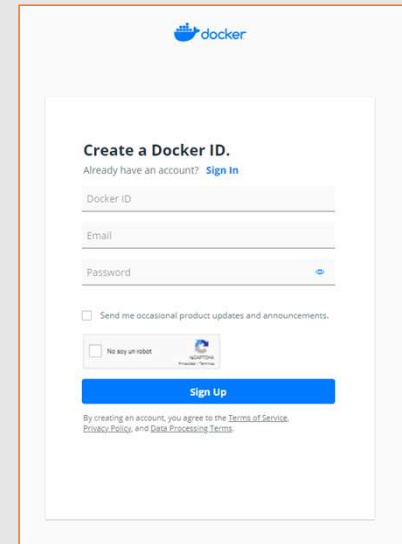
```
PS C:\Users\olga3emes> docker commit -a "Olga M. Moreno <olga3emes@gmail.com>" pg olga3emes/pg-prueba
sha256:6419b5a8e46597539d320a71f2b97b8c904e6780548a88dc0a214796045042d2
PS C:\Users\olga3emes> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
olga3emes/pg-prueba	latest	6419b5a8e465	13 seconds ago	315MB

- Para publicar la imagen creada debemos ejecutar la operación push + username/nombreImagen. Cuando termine la operación tendremos una url en DockerHub donde estará disponible públicamente nuestra imagen.

```
PS C:\Users\olga3emes> docker push olga3emes/pg-prueba
```

<https://hub.docker.com/repository/docker/olga3emes/pg-prueba>



Docker: volúmenes

Hay varias formas de transferir archivos entre un contenedor y el host. Las dos formas más comunes de transferir archivos son:

- 1ª: Mediante el comando `cp` de Docker.

Ejemplo: Publicar una página HTML ubicada en el host hacia un contenedor con Apache HTTP Server mediante `cp`.

```
docker run --name webserver -d -p 80:80 httpd
```

Esta instrucción lanza un contenedor con Apache HTTP Server ofreciendo sus servicios a través del puerto 80 (opción `-p`) hacia el host en el mismo puerto.

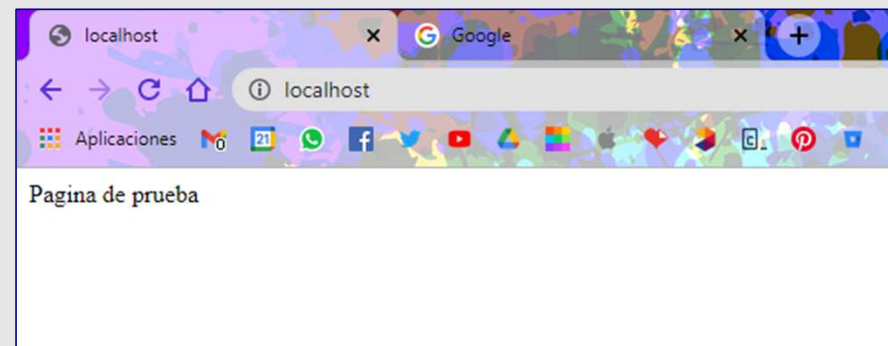
A continuación creamos una página HTML sencilla y la ubicamos en la carpeta de publicación web del contenedor.

```
PS C:\Users\olga3emes> echo "<html>Pagina de prueba</html>" > index.html
PS C:\Users\olga3emes> docker cp index.html webserver:/usr/local/apache2/htdocs
PS C:\Users\olga3emes> curl localhost

StatusCode      : 200
StatusDescription : OK
Content         : <html>Pagina de prueba</html>

RawContent      : HTTP/1.1 200 OK
                  Keep-Alive: timeout=5, max=100
                  Connection: Keep-Alive
                  Accept-Ranges: bytes
                  Content-Length: 64
                  Content-Type: text/html
                  Date: Tue, 13 Jul 2021 15:03:57 GMT
                  ETag: "40-5c7028aacdd80..."

Forms           : {}
Headers         : {[Keep-Alive, timeout=5, max=100], [Connection, Keep-Alive], [Accept-Ranges, bytes],
                  [Content-Length, 64]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 64
```



Docker: volúmenes

2ª Forma: Usando volúmenes de Docker.

Los volúmenes son ficheros o carpetas directamente montados en el host.

- No forman parte del sistema UFS.
- Pueden ser compartidos por varios contenedores.
- Todos los cambios se hacen sobre el sistema de archivos del host.
- Podemos declarar un fichero o carpeta como volumen:
 - Usando el argumento `-v` de `docker run`

Ejemplo: Las siguientes formas son equivalentes y permiten crear un volumen llamado `/data` dentro de un contenedor.

```
PS C:\Users\olga3emes> docker run -i -t -v /data debian /bin/bash
>>
root@7e032047565e:/# ls
bin boot data dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
```

Por defecto, los volúmenes son montados dentro del directorio de instalación de Docker (`/var/lib/docker`)

Es posible especificar la carpeta del host donde montar el volumen gracias al argumento `-v` de `docker run`:

`docker run -v <host dir>:<container dir> image`

Docker: volúmenes

Enlazar contenedores y/o sus volúmenes entre sí

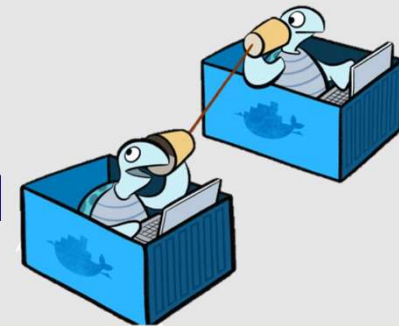
- Enlazar contenedores entre sí mediante Docker permite facilitar la comunicación en red entre los distintos microservicios en ejecución.
- Esto se puede realizar mediante el argumento `--link` de `docker run` o mediante Docker Compose (lo veremos en la siguiente sección).
- Pero, es una mejor práctica enlazar el sistema de archivos de un contenedor como volumen montado en otro. Esto facilita la transferencia de archivos entre los distintos microservicios.
- Para ello se utiliza el argumento `--volumes-from` de `docker run`.

El argumento `--volumes-from` de `docker run` tiene la siguiente sintaxis:

`docker run --volumes-from <nombre/ID contenedor> image`

Ejemplo con `--link`: `docker run --name redis_server -d redis`

```
PS C:\Users\olga3emes\Desktop> docker run --rm -it --link redis_server:redis redis bash
root@ecd9c9a70daf:/data# redis-cli -h redis -p 6379
redis:6379> ping
PONG
redis:6379> set "nombre" Olga
OK
redis:6379> get "nombre"
"Olga"
redis:6379> exit
root@ecd9c9a70daf:/data# exit
exit
```



Creación un microservicio de almacenamiento de datos en memoria basado en pares clave/valor y se hace uso de él desde otro microservicio.

PROMETEO

Docker: volúmenes

Ejemplo con `--volumes-from`: ejecutar un contenedor que permita hacer una copia de seguridad de los datos del microservicio redis anterior.

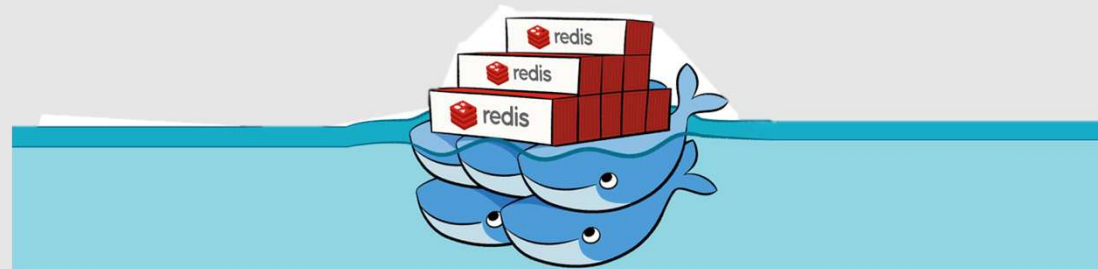
En primer lugar, hacemos una copia de seguridad de los datos que hemos insertado anteriormente.

```
PS C:\Users\olga3emes> docker run --rm -it --link redis_server:redis redis bash
root@1eb7f0fb4945:/data# redis-cli -h redis -p 6379
redis:6379> save
OK
redis:6379> exit
root@1eb7f0fb4945:/data# exit
exit
```

Luego, creamos un contenedor debian que nos permite enlazar un volumen con otro y extraer la copia de seguridad a nuestro archivo local.

```
docker run --rm --volumes-from redis_server -v ${pwd}/backup:/backup debian cp /data/dump.rdb /backup/
```

En Linux:
\$(pwd)

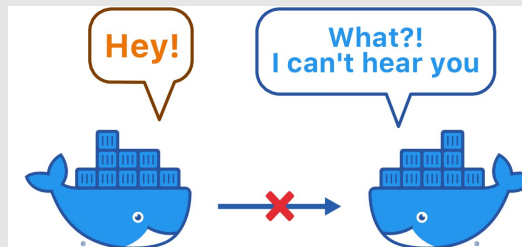


Docker : Redes

- Cuando instalamos Docker tenemos las siguientes redes predefinidas:

```
PS C:\Users\olga3emes> docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
4670a0d46273        bridge              bridge              local
045fe7b51b96        host                host                local
de99da1f19dc        none                null                local
PS C:\Users\olga3emes>
```

- Por defecto los contenedores que creamos se conectan a la red de tipo bridge llamada bridge (por defecto el direccionamiento de esta red es 172.17.0.0/16). Los contenedores conectados a esta red que quieren exponer algún puerto al exterior tienen que usar la opción **-p** para mapear puertos.
- Si conecto un contenedor a la red host, el contenedor será accesible usando la misma IP que máquina host.
- La red none no configurará ninguna IP para el contenedor y no tiene acceso a la red externa ni a otros contenedores. Tiene la dirección loopback y se puede usar para ejecutar trabajos por lotes.



Docker: Redes

La parte de redes de Docker está basada en la arquitectura llamada Container Network Model (CNM). Esta está implementada en una librería llamada libnetwork, que forma parte de Docker Engine. Por otro lado, cada SO que soporta Docker Engine tiene un conjunto de drivers que permiten a libnetwork crear la configuración de la red que corresponda en cada sistema operativo por nosotros. De manera nativa, estos traen consigo 4 tipos:

- **Bridge/NAT:** Se utiliza cuando tenemos un solo host de Docker y necesitamos que los contenedores hablen entre sí.
- **Overlay:** está disponible cuando trabajamos en un formato clúster, es decir que tenemos más de un host de Docker trabajando como uno solo, por lo que se necesita que los contenedores puedan hablar entre sí, independientemente de dónde estén ejecutándose dentro del clúster.
- **None:** este driver es la forma de decir a Docker que para un contenedor/es en concreto no queremos que estén conectados a ninguna red. Lo que hace en este caso es no asignarle una tarjeta de red.
- **Macvlan:** esta opción se utiliza en el caso de querer conectar los dos mundos: el dockerizado y el sin dockerizar. Se trataría de un entorno híbrido que hace que los contenedores tengan asignadas una IP dentro de la red donde está el host/s.

Docker: Redes

Bridge

- Este tipo es el más simple que ofrece Docker y es el que hemos estado usando todo este tiempo, sin saberlo.
- Se utiliza cuando sólo existe un host y solo es necesario comunicar los contenedores que viven en él entre ellos.
- En Linux se crean con el driver llamado bridge mientras que en Windows se utiliza el llamado nat, pero el funcionamiento es el mismo.
- Esta red es la utilizada por defecto, salvo que utilicemos el parámetro -network, que veremos después. Para comprobarlo, vamos a crear dos contenedores:

```
PS C:\Users\olga3emes> docker run -dit --name container-1 alpine ash
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
5843afab3874: Pull complete
Digest: sha256:234cb88d3020898631af0ccbbcca9a66ae7306ecd30c9720690858c1b007d2a0
Status: Downloaded newer image for alpine:latest
a24f5a42371d954a1440f47e54f53858ea9ad14213eb536f7bf5d17a63569862
PS C:\Users\olga3emes> docker run -dit --name container-2 alpine ash
87c0f66ec60476c2bac5d198ddee0c3ef4682313ffa413510288a9217d3036a2
```

- Si inspeccionamos la red bridge, veremos que existe un apartado llamado Containers donde aparecen los dos que tenemos ejecutándose:

```
PS C:\Users\olga3emes> docker network inspect bridge
```

```
Podemos comprobar que funciona con:
docker attach container-1
ping -c 2 172.17.0.X
```

Docker: Redes

- Cada vez que se ponga en marcha un contenedor la IP puede ser diferente, ya que de ninguna forma estas IPs se reservan en los contenedores. Esto puede suponer un problema si varios contenedores se comunican entre sí, y quiero añadir esta información como parte de mi aplicación.
- Es por ello que lo que sería ideal que pudiéramos comunicarnos con los contenedores a través de su nombre, en lugar de la IP, y es otra de las características que Docker nos brinda. Sin embargo, en esta red bridge generada por defecto no es posible usar este mecanismo.
- Para crear una red dentro de Docker es sencillo, solamente debemos utilizar el siguiente comando:

```
docker network create mynetwork
```

- Para probar la comunicación a través del nombre vamos a crear dos contenedores dentro de la misma, utilizando el parámetro -network:

```
PS C:\Users\olga3emes> docker container run -dit --name container-a --network mynetwork alpine ash  
dfcc2b662eacd419939307769c1df9da0627c40a6c5e94a807dae3acb8149449  
PS C:\Users\olga3emes> docker container run -dit --name container-b --network mynetwork alpine ash  
4f7f90b3e2d3fe14b7c2271960cd2e90f9e9cf50fd79653144c53f90076ab324
```

- Si ahora intentamos hacer ping desde el contenedor b al contenedor a:

```
PS C:\Users\olga3emes> docker attach container-b  
/ # ping -c 3 container-a  
PING container-a (172.18.0.2): 56 data bytes  
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.042 ms  
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.044 ms  
64 bytes from 172.18.0.2: seq=2 ttl=64 time=0.040 ms  
  
--- container-a ping statistics ---  
3 packets transmitted, 3 packets received, 0% packet loss  
round-trip min/avg/max = 0.040/0.042/0.044 ms  
/ #
```


Docker: Redes

Conectar un contenedor a dos redes

- Un contenedor conectarse a más de una red. Sin embargo, durante la creación solo es posible conectarse a una única red. Para conectarse a una segunda, tercera, etc. red, necesitas hacerlo a posteriori utilizando network connect:

```
docker network connect mynetwork container-1
```

- Ahora el container-1, que originalmente solo estaba en la red por defecto, llamada bridge, ahora también pertenece a la red llamada mynetwork, por lo que podría comunicarse tanto con container-a como con container-b:

```
PS C:\Users\olga3emes> docker network connect mynetwork container-1
PS C:\Users\olga3emes> docker attach container-1
/ # ping -c 2 container-a
PING container-a (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.042 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.043 ms

--- container-a ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.042/0.042/0.043 ms
/ #
```


Docker: Redes

Host

Otra de las redes que aparece en el listado de las redes por defecto es la llamada host. Lo que hace esta red es eliminar el aislamiento entre el host y los contenedores, por lo que un contenedor no recibe su propia IP, como vimos en los ejemplos anteriores, sino que utiliza la del host. Por lo tanto, si ejecutas un contenedor asociado a esta red escuchando por el puerto 80, la aplicación del contenedor estará disponible a través del puerto 80 de la IP del host. Este modo suele ser útil cuando necesitas mejorar el rendimiento de tu aplicación.

```
docker run --rm -d --network host --name my_nginx nginx
```

Para comprobar que funciona debemos acceder a <http://localhost:80> .

Sólo funciona en Linux y no está soportado en Docker Desktop, debido a que estamos trabajando con una máquina virtual.

Docker: Redes

None

- En el caso de que no quisiéramos que un contenedor tenga una red asignada, utilizaremos -network none como en el siguiente ejemplo:

```
PS C:\Users\olga3emes> docker run --rm -dit --network none --name no-net-alpine alpine ash 2d1d04ee3deb02a17485149340460860674485dee6890caf3652e4a4071b81e7
```

- Si ahora revisamos las tarjetas de red que tiene asignado este contenedor comprobarás que solo aparece la interfaz loopback, que es para que el contenedor pueda hablar consigo mismo, pero nada más.

```
PS C:\Users\olga3emes> docker exec no-net-alpine ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: sit0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN qlen 1000
    link/sit 0.0.0.0 brd 0.0.0.0
```

Overlay

- Para el tipo overlay, necesitamos trabajar en modo clúster. Si intentas crear una red de este tipo, sin estar en este modo, obtendrás el siguiente mensaje:

```
PS C:\Users\olga3emes> docker network create --driver overlay multihost-net
Error response from daemon: This node is not a swarm manager. Use "docker swarm init" or "docker swarm join" to connect this node to swarm and try again.
```

- Sin embargo, la experiencia es la misma: la comunicación entre los diferentes contenedores podrá hacerse tanto a través de su IP como de su nombre, actuando como una única red, aunque haya diferentes máquinas ejecutando nuestros contenedores.

Docker: Redes

Macvlan

- El modo llamado Macvlan que nos permite asignar IPs de una red a cada uno de los contenedores que viven en nuestro host. Sin embargo, para que este tipo de redes funcione necesitamos que la tarjeta de red del host esté en **modo promiscuo**, lo cual significa que necesita estar a la escucha de todos los paquetes que viajan por dicha red, simplemente para poder reconocer aquellos que potencialmente pueden ser para uno de sus contendores. Este modo muchas veces no está permitido en muchas organizaciones. La forma de configurar una red de este tipo sería así:

```
docker network create -d macvlan --subnet=172.16.86.0/24 --gateway=172.16.86.1 -o parent=eth0 my-macvlan-net
```

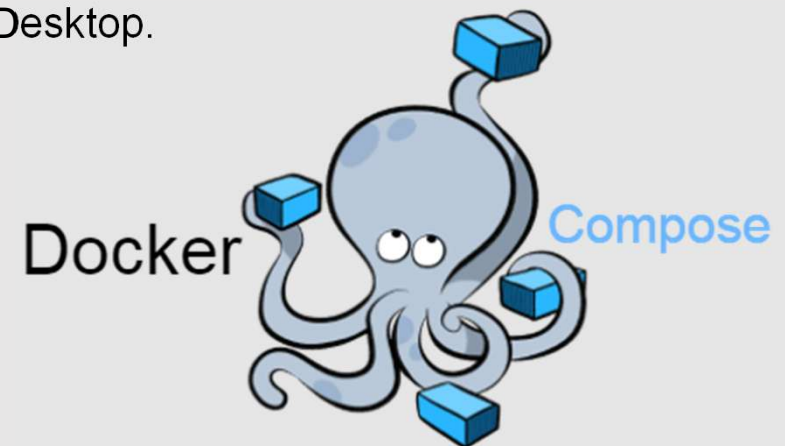
- Con ello conseguiríamos que los contenedores que estén dentro de esta puedan ser alcanzados por otras aplicaciones que vivan fuera de Docker.

Docker: despliegue orquestado

Despliegue orquestado mediante Docker Compose

- Docker Compose automatiza el despliegue de aplicaciones multicontenedor.
- Evita tener que crear y mantener nuestros propios scripts automatizados para crear, ejecutar contenedores y enlazarlos entre sí.
- En primer lugar, debemos instalar Docker Compose, pues es una herramienta independiente a la que hemos usado hasta ahora (Docker Engine).
- Guía de instalación de Docker Compose:
<https://docs.docker.com/compose/install/>
 - En Mac y Windows se instala a la vez con Docker Desktop.
 - Para verificar la instalación:

```
PS C:\Users\olga3emes> docker-compose --version
docker-compose version 1.29.1, build c34c88b2
PS C:\Users\olga3emes>
```



Antes de seguir... ¿Qué es YAML?

YAML es un lenguaje de serialización de datos que suele utilizarse en el diseño de archivos de configuración.

Para algunas personas, YAML significa otro lenguaje de marcado más; para otras, es un acrónimo recursivo que quiere decir YAML no es un lenguaje de marcado, lo que enfatiza la idea de que se utiliza para los datos, no para los documentos.

Además, es un lenguaje de programación popular debido a que las personas pueden comprenderlo fácilmente y utilizarlo con otros lenguajes. Gracias a la flexibilidad y la accesibilidad que lo caracterizan, la herramienta de automatización de Ansible lo utiliza para crear procesos de automatización en forma de playbooks de Ansible.

Lo vamos a utilizar para generar nuestros scripts de automatización con Docker-compose.

Sintaxis de YAML

YAML cuenta con funciones de Perl, C, XML, HTML y otros lenguajes de programación, y se basa en JSON, lo cual genera que los archivos JSON sean compatibles con YAML.

En YAML, se usa la sangría al estilo Python para indicar la incorporación de un elemento de código dentro de otro. No se admiten los caracteres de tabulación, así que se usan los espacios en blanco, ni tampoco hay símbolos comunes de formato, como las llaves, los corchetes, las etiquetas de cierre o las comillas.

Los archivos YAML usan las extensiones `.yaml` o `.yml`.

La estructura de los archivos puede ser un mapa o una lista.

Los mapas permiten asociar pares de clave-valor. Cada clave debe ser única y el orden no importa, tal como ocurre en un diccionario de Python o la asignación de variables en un script de Bash.

Es necesario resolver los mapas para poder cerrarlos, entonces se origina uno nuevo. Es posible crear mapas nuevos aumentando el nivel de sangría o resolviendo el mapa anterior y comenzando uno adyacente.

Sintaxis de YAML

Las listas incluyen valores en un orden específico y pueden contener cualquier cantidad de elementos según sea necesario. Las secuencias de las listas empiezan con un guion (-) y un espacio, y se utiliza la sangría para separarlas del elemento principal, de manera similar a las listas de Python o los conjuntos en Bash o en Perl. Las listas se pueden incorporar a los mapas.

YAML también contiene valores sencillos (escalares), que son datos arbitrarios (codificados en Unicode) que se pueden utilizar como cadenas, números enteros, fechas, números o booleanos.

Al crear un archivo YAML, es necesario asegurarse de que sea válido y siga las reglas sintácticas. Un “linter” es una herramienta que se usa para verificar la sintaxis de los archivos. El comando `yamllint` sirve para asegurarse de haber creado un archivo YAML válido antes de enviarlo a una aplicación.

<https://www.redhat.com/sysadmin/yaml-beginners>

Ejemplo de sintaxis de YAML

A continuación aparece el ejemplo de un archivo YAML simple en comparación con la sintaxis de XML y JSON:

XML	JSON	YAML
<pre><Servers> <Server> <name>Server1</name> <owner>John</owner> <created>123456</created> <status>active</status> </Server> </Servers></pre>	<pre>{ Servers: [{ name: Server1, owner: John, created: 123456, status: active }] }</pre>	<pre>Servers: - name: Server1 owner: John created: 123456 status: active</pre>

Docker: despliegue orquestado

Ejemplo 1: Lanzar un servidor web con WordPress usando MariaDB.

```
docker-compose -f file.yml up
```

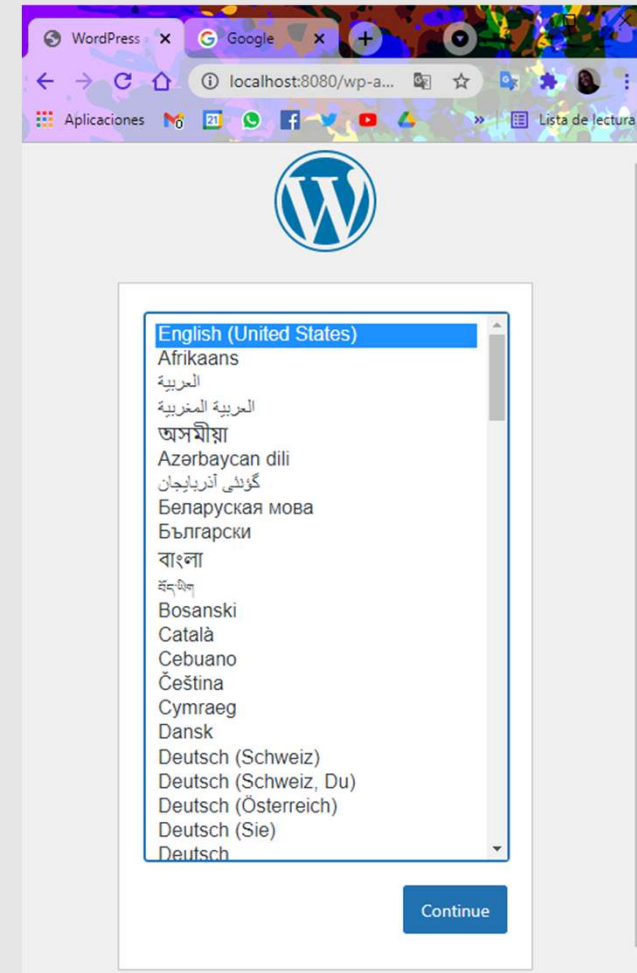
docker-compose.yml

```
version: '3.1'

services:
  wordpress:
    image: wordpress
    restart: always
    ports:
      - 8080:80
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: exampleuser
      WORDPRESS_DB_PASSWORD: examplepass
      WORDPRESS_DB_NAME: exampledb
    volumes:
      - wordpress:/var/www/html

  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_DATABASE: exampledb
      MYSQL_USER: exampleuser
      MYSQL_PASSWORD: examplepass
      MYSQL_RANDOM_ROOT_PASSWORD: '1'
    volumes:
      - db:/var/lib/mysql

volumes:
  wordpress:
  db:
```



Docker: despliegue orquestado

Ejemplo 1: Lanzar un servidor web con WordPress usando MariaDB.

- Visualizar los servicios en ejecución gestionados por Docker Compose.

```
PS C:\Users\olga3emes> docker ps
```

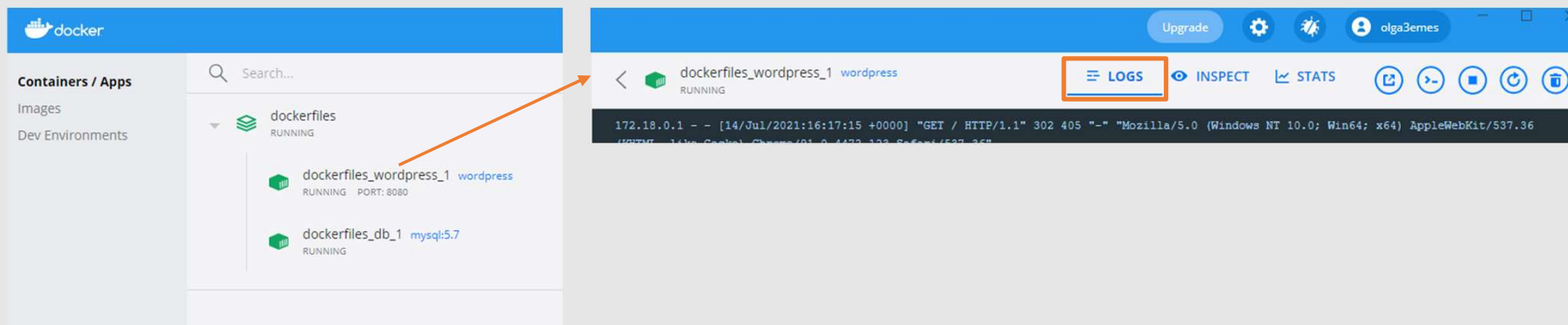
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ee049335d414	wordpress	"docker-entrypoint.s..."	15 hours ago	Up 50 seconds	0.0.0.0:8080->80/tcp, :::8080->80/tcp	dockerfiles_wordpress_1
b8e92a63fbf4	mysql:5.7	"docker-entrypoint.s..."	15 hours ago	Up 50 seconds	3306/tcp, 33060/tcp	dockerfiles_db_1

- Visualizar los mensajes de logs de los servicios en ejecución de Docker Compose.

```
PS C:\Users\olga3emes\Desktop> cd .\dockerfiles\  
PS C:\Users\olga3emes\Desktop\dockerfiles> docker-compose logs
```

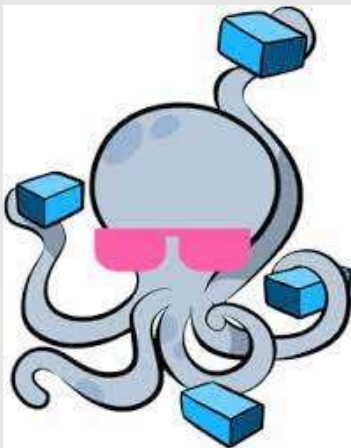
Attaching to dockerfiles_wordpress_1, dockerfiles_db_1

```
db_1 | 2021-07-14 16:12:15+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 5.7.32-1debian10 started.  
db_1 | 2021-07-14 16:12:15+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'  
db_1 | 2021-07-14 16:12:15+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 5.7.32-1debian10 started.  
db_1 | 2021-07-14 16:12:15+00:00 [Note] [Entrypoint]: Initializing database files
```



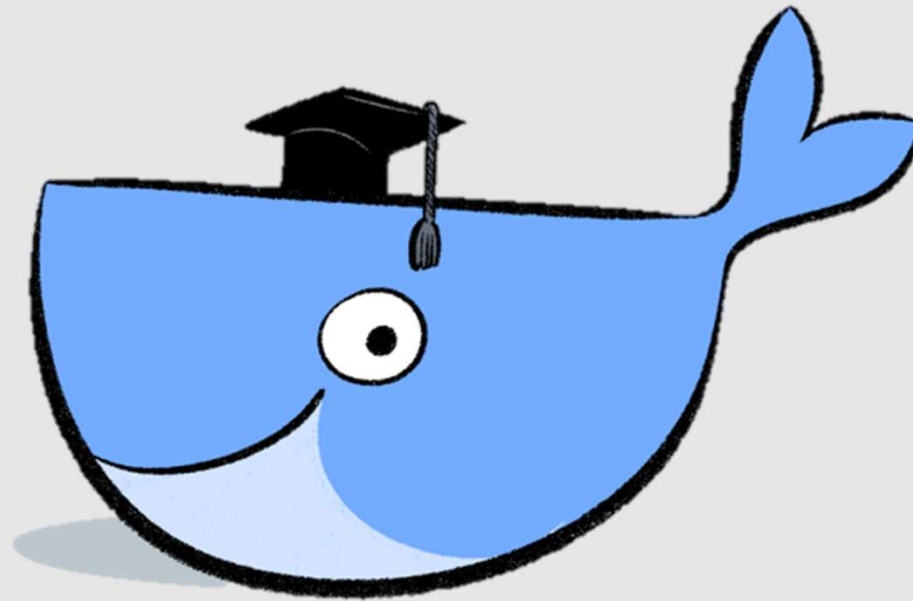
Docker: despliegue orquestado

Comandos de Docker Compose: <https://docs.docker.com/compose/reference/overview/>



Commands:	
build	Build or rebuild services
bundle	Generate a Docker bundle from the Compose file
config	Validate and view the Compose file
create	Create services
down	Stop and remove containers, networks, images, and volumes
events	Receive real time events from containers
exec	Execute a command in a running container
help	Get help on a command
images	List images
kill	Kill containers
logs	View output from containers
pause	Pause services
port	Print the public port for a port binding
ps	List containers
pull	Pull service images
push	Push service images
restart	Restart services
rm	Remove stopped containers
run	Run a one-off command
scale	Set number of containers for a service
start	Start services
stop	Stop services
top	Display the running processes
unpause	Unpause services
up	Create and start containers
version	Show the Docker-Compose version information

FIN



Bibliografía/Webgrafía

<https://www.docker.com/>

<https://hub.docker.com/>

<https://www.redhat.com/es/topics/automation/what-is-yaml>

<https://www.redhat.com/sysadmin/yaml-beginners>

<https://docker-curriculum.com/>

<https://docs.docker.com/get-started/>



¿Preguntas?

Espacio abierto para dudas, comentarios o reflexiones sobre lo visto hoy

PROMETEO

Unidad 2





2

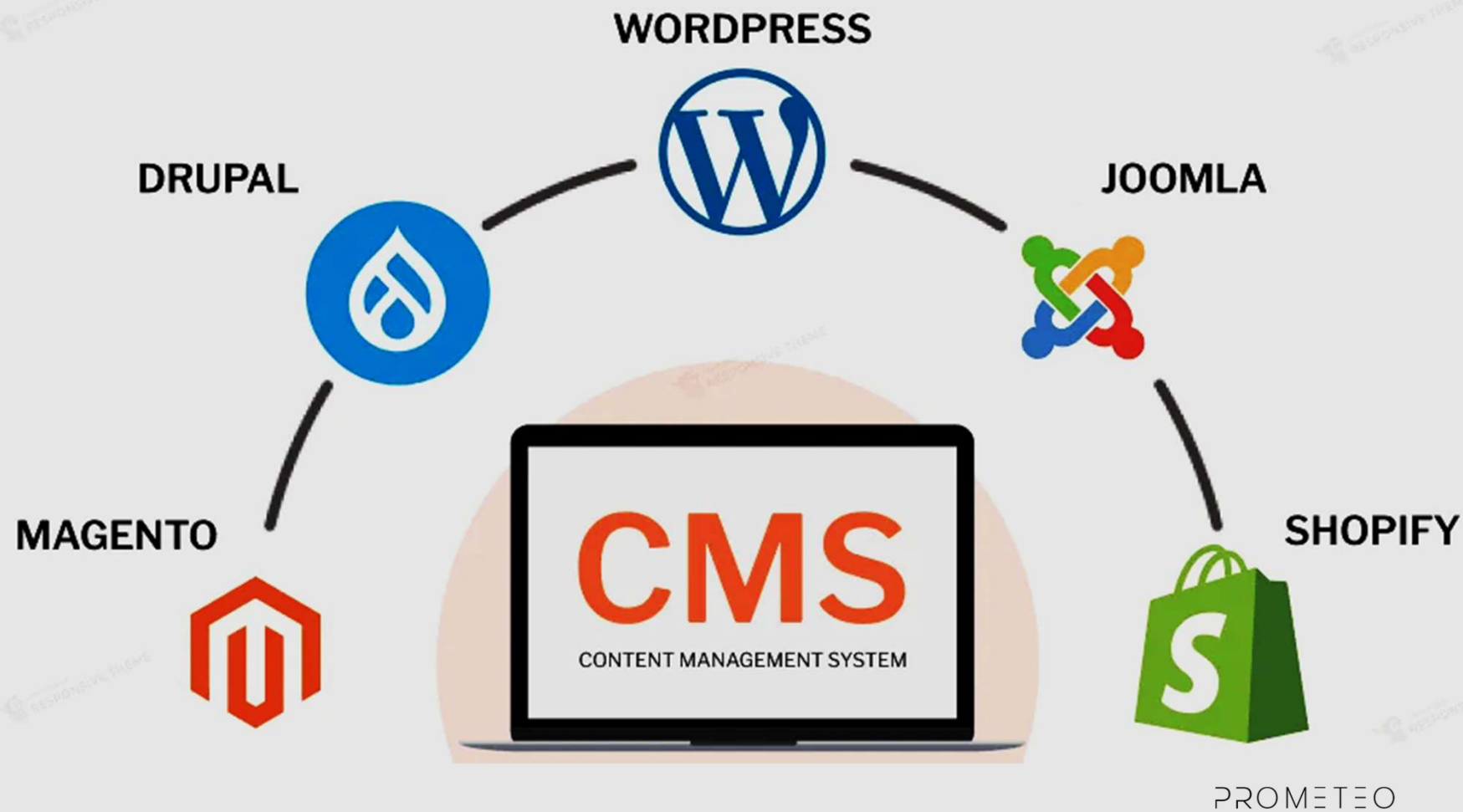
Tema 2: Instalación, administración y personalización

PROMETEO

¿Qué es un CMS?

RESPONSIVE THEME

RESPONSIVE THEME



RESPONSIVE THEME

RESPONSIVE THEME

Definición de CMS

Un sistema de gestión de contenidos o CMS es un programa informático que permite crear un entorno de trabajo para la creación y administración de contenidos, principalmente en páginas web, por parte de los administradores, editores, participantes y demás usuarios.

[Visitar para más información](#)

¿Por qué un CMS?

Razones para usar un CMS

- Más sencillo.
- Menos fácil de implementar.
- Menos recursos.
- Menos conocimientos técnicos específicos.
- Más rápido.
- Más autogestionable.
- Open Source.
- MÁS BARATO.

Web Technology Usage Trends

Web and Internet Technology Usage Statistics

BuiltWith Trends in Spain

View a selection of the most popular web technologies in use across Spain.

Technology Trends in Spain

Technology Groups

- Advertising ▼
- Analytics and Tracking ▼
- Audio / Video Media ▼
- Content Delivery Network
- Content Management System ▼
- Copyright
- Document Encoding
- Document Standards ▼
- Domain Parking

Popular Technologies

 **Shopify**
5,102,201 live websites

 **Magento**
630,747 live websites

 **WordPress**
33,155,586 live websites

 **WooCommerce**
5,481,315 live websites

 **Magento Enterprise**
65,606 live websites

 **Drupal**
1,631,122 live websites

 **Ruby on Rails**
2,286,648 live websites

 **Salesforce**
594,172 live websites

 **Cart Functionality**
7,427,065 live websites

 **Angular JS**
3,567,283 live websites

 **Amazon**
24,618,183 live websites

 **Google AdSense**
41,479,218 live websites

 **Magento 1.7**
10,406 live websites

 **Magento 1.9**
127,918 live websites

 **Taboola**
428,817 live websites

 **Umbraco**
105,439 live websites

 **Pardot**
442,643 live websites

 **Liferay**
104,035 live websites

Introducción a Wordpress



PROMETEO

Docker: despliegue orquestado

Ejemplo 1: Lanzar un servidor web con WordPress usando MariaDB.

```
docker-compose -f file.yml up
```

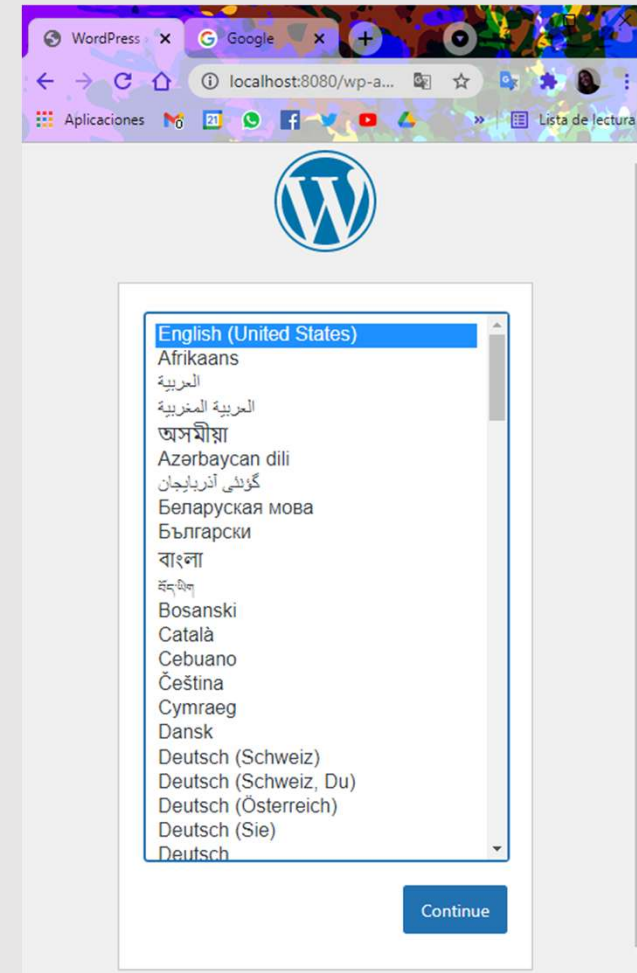
docker-compose.yml

```
version: '3.1'

services:
  wordpress:
    image: wordpress
    restart: always
    ports:
      - 8080:80
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: exampleuser
      WORDPRESS_DB_PASSWORD: examplepass
      WORDPRESS_DB_NAME: exampledb
    volumes:
      - wordpress:/var/www/html

  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_DATABASE: exampledb
      MYSQL_USER: exampleuser
      MYSQL_PASSWORD: examplepass
      MYSQL_RANDOM_ROOT_PASSWORD: '1'
    volumes:
      - db:/var/lib/mysql

volumes:
  wordpress:
  db:
```



Google Chrome browser window showing a search for "descargar wordpress".

Search bar: descargar wordpress

Results: Aproximadamente 46.600.000 resultados (0,32 segundos)

Search results for "descargar wordpress":

- [https://es.wordpress.org > download](https://es.wordpress.org/download)
- Descarga | WordPress.org España** (highlighted with a red box and a red arrow pointing to it)
- Descarga hoy **WordPress** y empieza a crear tu web con una de las más potentes, populares y personalizables plataformas del mundo.
- Gratis
- [Versiones](#) · [Obtener WordPress](#) · [Conseguí WordPress](#) · [Consigue WordPress](#)
- <https://es.wordpress.org>
- WordPress.org**
- Software de código abierto que puedes usar para crear fácilmente una bonita web, blog o aplicación.

Instalación de WordPress x Descargar - WordPress.org x +

es.wordpress.org/download/ Finalizar actualización

Campus Séneca Index of /Ejemplos... LM DWEC DWES Interfaces Apple Teacher Programa - Word... BOES DUAL Cursos

Hola, olga3emes

Consigue WordPress

Todo lo que necesitas para configurar tu sitio tal y como tú quieras.

Descárgalo e instálalo por tu cuenta

Para cualquiera que se sienta a gusto contratando su propio alojamiento y dominio.

[Descarga WordPress 6.6.2](#) [Guía de instalación](#)

Recomendado PHP 7.4 o superior y MySQL 8.0 o MariaDB versión 10.5 o superior.

[Versiones](#) · [Nocturnas](#) · [Contador](#) · [Origen](#)

Configúralo con un proveedor de alojamiento

Para cualquiera que busque la manera más sencilla de empezar.

[Ver todos los alojamientos recomendados](#)

Ver ficheros de Wordpress



< > wordpress Atrás/Adelante	
Nombre	
>  wp-admin	
>  wp-content	
>  wp-includes	
 license.txt	
 index.php	
 wp-activate.php	
 wp-blog-header.php	
 wp-comments-post.php	
 wp-config-sample.php	
 wp-cron.php	
 wp-links-opml.php	
 wp-load.php	
 wp-login.php	
 wp-mail.php	
 wp-settings.php	
 wp-signup.php	
 wp-trackback.php	
 xmlrpc.php	
 readme.html	

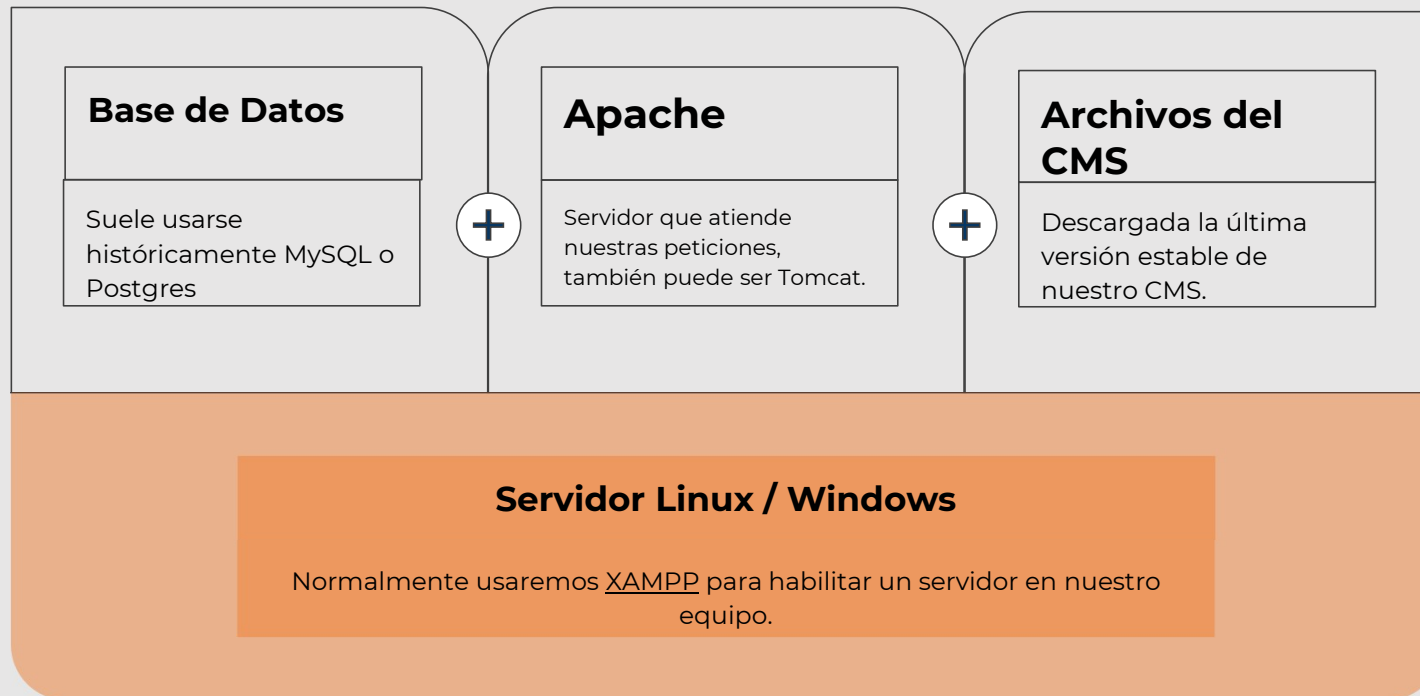
< > wp-admin Atrás/Adelante	
Nombre	
>  css	
>  images	
>  includes	
>  js	
>  maint	
>  network	
>  user	
 freedoms.php	
 edit-form-blocks.php	
 site-editor.php	
 about.php	
 credits.php	
 options.php	
 options-reading.php	
 menu.php	
 plugin-editor.php	
 theme-editor.php	
 user-edit.php	
 site-health.php	
 load-scripts.php	
 load-styles.php	
 themes.php	

< > wp-content Atrás/Adelante	
Nombre	
 index.php	
>  languages	
>  plugins	
>  themes	

< > wp-includes Atrás/Adelante	
Nombre	
>  assets	
>  block-patterns	
>  block-supports	
>  blocks	
>  certificates	
>  css	
>  customize	
>  fonts	
>  ID3	
>  images	
>  IXR	
>  js	
>  php-compat	
>  PHPMailer	
>  pomo	
>  random_compat	
>  Requests	
>  rest-api	
>  SimplePie	
>  sitemaps	
>  sodium_compat	
>  Text	

¿Qué necesita WP para funcionar?

Stack técnico básico

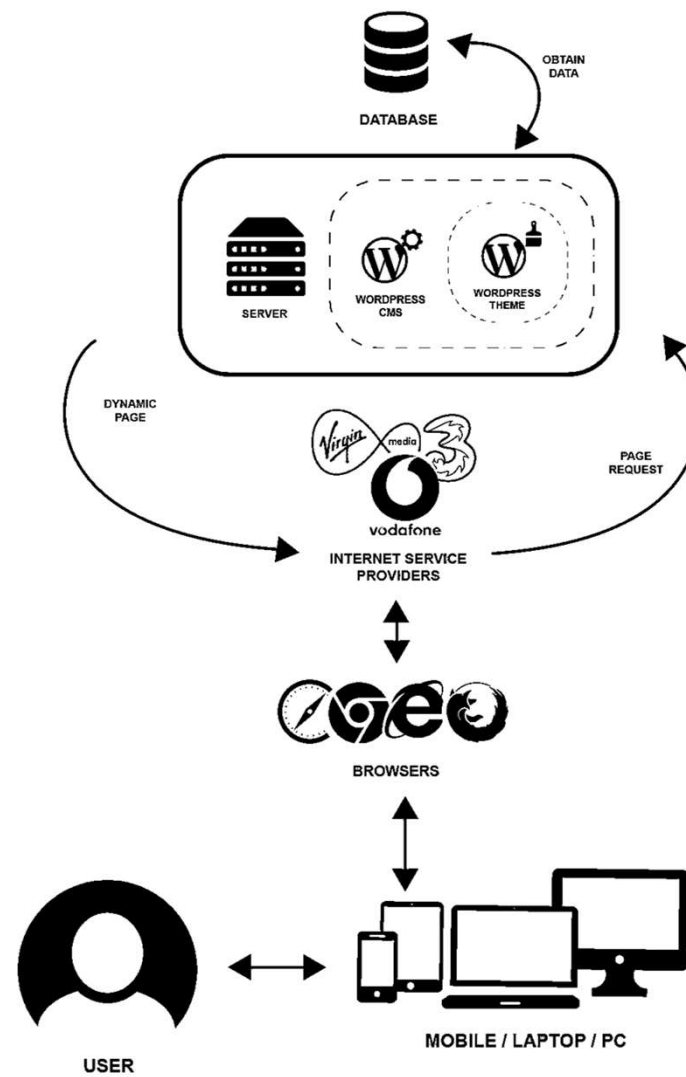


PROMETEO

Arquitectura WP

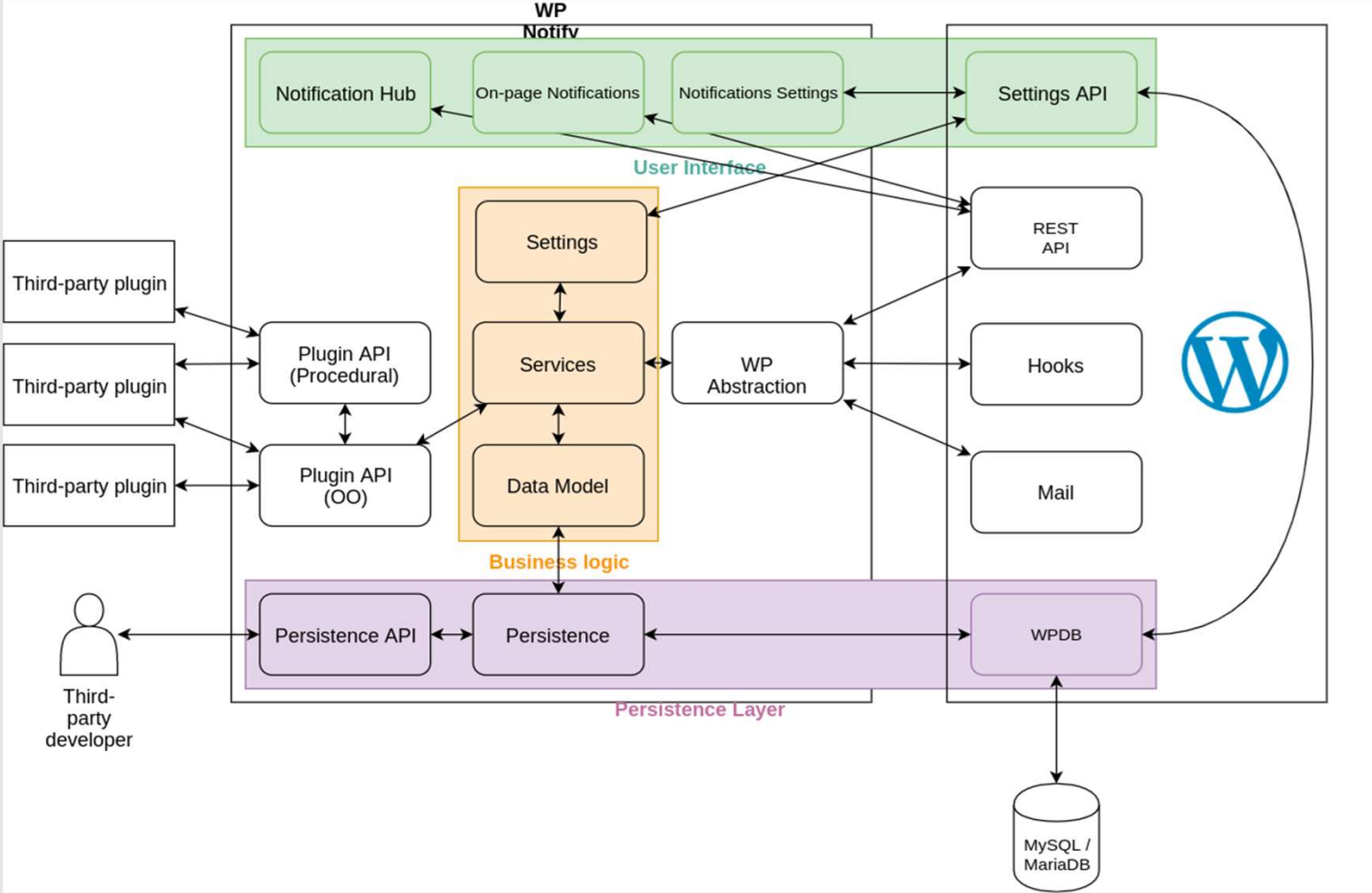
PROMETEO

Esquema General



PROMETEO

Arquitectura Plugin





WordPress first looks
for files in your child
theme's folder



Anything it does not
find, it loads from the
parent theme



style.css
footer.php
screenshot.png
page-nosidebar.php



style.css
~~footer.php~~
~~screenshot.png~~

modifies and adds to

Proceso de Instalación de WP

Crear una Base de Datos para WordPress

TU PROFE TE AYUDARÁ SI NO SABES a veces este paso no es necesario

Abrir phpMyAdmin o MySQL Workbench:

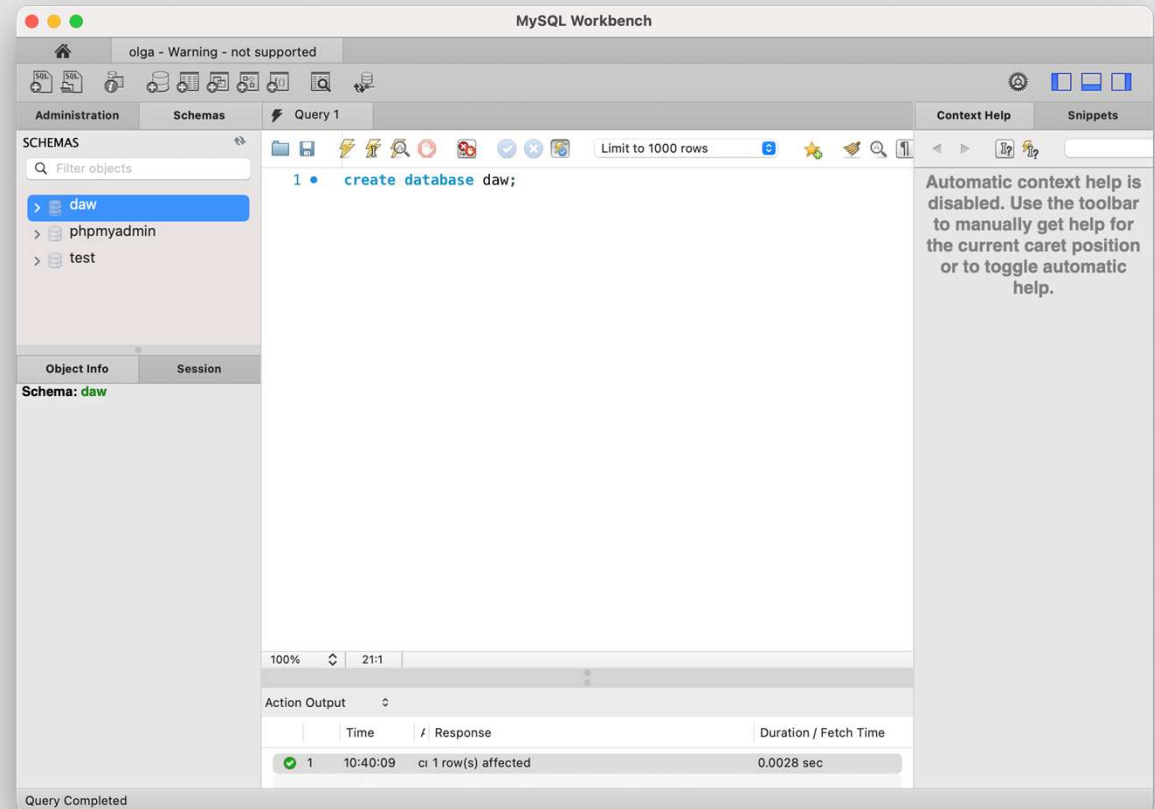
Si abrimos phpMyAdmin: Ir a <http://localhost/phpmyadmin>.

Para utilizar MySQL Workbench (descargarlo).

Crear Base de Datos:

Crear una nueva base de datos con un nombre adecuado (por ejemplo, wordpress).

Configurar Usuarios: Configurar el usuario y los permisos si es necesario.

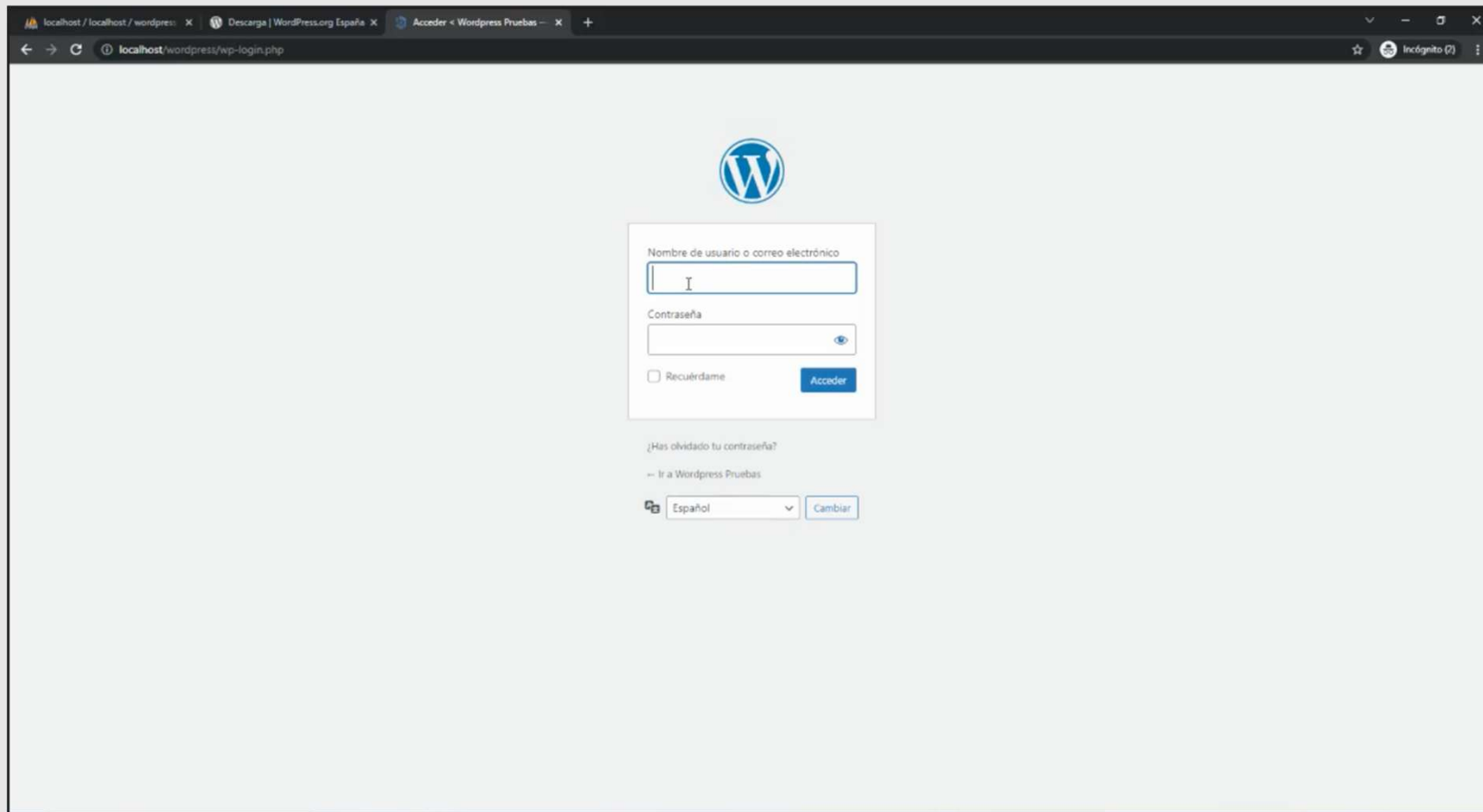


Crear una Base de Datos para WordPress

Si ya tenemos una base de datos creada:

Editar wp-config.php:

- Cambiar el nombre del archivo wp-config-sample.php a wp-config.php y editarlo para añadir la información de la base de datos:
- `define('DB_NAME', 'wordpress');`
- `define('DB_USER', 'root');`
- `define('DB_PASSWORD', '');`
- `define('DB_HOST', 'localhost');`
- **Guardar los Cambios:** Guardar el archivo wp-config.php con la configuración actualizada.



Back-End de WP

← → ↻ localhost/wordpress/wp-admin/ Incógnito (2) Hola, admin

WordPress Pruebas 0 + Añadir

Escritorio

Inicio Actualizaciones

Entradas Medios Páginas Comentarios Apariencia Plugins Usuarios Herramientas Ajustes Cerrar menú

¡Bienvenido a WordPress!

[Aprende más sobre la versión 5.9](#)

5.9 Descartar

 **Crea contenido rico con bloques y patrones**

Los patrones de bloques son diseños de bloques preconfigurados. Úsalos para inspirarte o crear nuevas páginas en un instante.

[Añadir una nueva página](#)

 **Personaliza todo tu sitio con temas de bloques**

Diseña todo en tu sitio — Desde la cabecera hasta el pie de página. Todo usando bloques y patrones.

[Abrir el editor del sitio](#)

 **Cambia la apariencia de tu sitio con los estilos**

¡Retoca tu sitio o dale un aspecto completamente nuevo! Sé creativo — ¿Qué tal una nueva paleta de color o una nueva fuente?

[Editar los estilos](#)

Estado de salud del sitio

Aún no hay información...

Las pruebas de salud del sitio se ejecutarán automáticamente de forma periódica para obtener información sobre tu sitio. También puedes [visitar ahora la pantalla de salud del sitio](#) para obtener información sobre tu sitio.

Borrador rápido

Título

Contenido

¿En qué estás pensando?

[Guardar borrador](#)

De un vistazo

1 entrada 1 página 1 comentario

Arrastra aquí las cajas

Arrastra aquí las cajas

PROMETEO

 Escritorio

 Entradas

Todas las entradas

Añadir nueva

Categorías

Etiquetas

 Medios

 Medios

Biblioteca

Añadir nuevo

 Páginas

Todas las páginas

Añadir nueva

  PlanDigital.io  0  Añadir WPForms

 Escritorio

 Entradas

 Medios

 Páginas

 **Comentarios**

 Elementor

 Plantillas

Comentarios

Todos (1) | Míos (0) | Pendientes (0) | A

Acciones en lote

Aplicar

☐ Autor

☐  Un comentarista de WordPress

wordpress.org

wapuu@wordpress.example

Apariencia

Temas

Personalizar

Widgets

Menús

Plantillas de inicio

Opciones de Astra

Editor de archivos de temas

Plugins 1

Plugins instalados

Añadir nuevo

Editor de archivos de plugins

Usuarios

Todos los usuarios

Añadir nuevo

Perfil



Herramientas

Herramientas disponibles

Importar

Exportar

Salud del sitio

Exportar los datos personales

Borrar los datos personales



Ajustes 4

Generales

Escritura

Lectura

Comentarios

Medios

Enlaces permanentes

Privacidad

Y mucho más...

PROMETEO



PROMETEO

PROMETEO

Unidad 3



Licencia



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Introducción

- ¿Qué entendemos por “nube” y “productividad web”?
- **Ventajas:** trabajar desde cualquier dispositivo, colaborar en tiempo real, acceder a archivos compartidos, menos dependencias de infraestructura local.
- En esta sesión vamos a ver 4 herramientas potentes que permiten todo eso, y ver cómo pueden ayudar tanto en educación como en el entorno profesional.

¿Qué es Google Workspace?

- Es la suite de productividad de Google que incluye servicios como Gmail, Calendar, Drive, Docs, Sheets, Slides, Meet, Chat.
- Permite: correo personalizado, almacenamiento en la nube, colaboración en documentos en tiempo real, videoconferencias, etc.

Enlace oficial: workspace.google.com

Puntos para los alumnos:

- Crear un documento colaborativo con otros.
- Compartir carpetas y asignar permisos.
- Usar Google Meet para clases o reuniones virtuales.

¿Qué es Microsoft 365?

- Suite de productividad de Microsoft que incluye Word, Excel, PowerPoint, Outlook, OneDrive, Teams y otros servicios en la nube.
- Adicionalmente: almacenamiento en la nube, colaboración en tiempo real, opciones para educación.

Enlace oficial: microsoft.com/microsoft-365 Microsoft

Puntos para los alumnos:

- Editar un archivo de Word online con 2 o más compañeros.
- Guardar automáticamente en OneDrive y acceder desde cualquier lugar.
- Usar Teams para videollamadas, chat de clase, intercambio de archivos.

¿Qué es Trello?

Herramienta de gestión visual de tareas, basada en tableros “kanban”, listas y tarjetas. Permite ver de un vistazo qué tareas están pendientes, en curso o terminadas. Ideal para proyectos, grupos de trabajo, gestionar tareas académicas.

Enlace oficial: trello.com trello.com

Puntos para los alumnos:

- Crear un tablero para un trabajo de grupo: listas “Por hacer”, “Haciendo”, “Hecho”.
- Añadir tarjetas con subtareas, fechas de entrega, asignar responsables.
- Integrar con calendario, adjuntar archivos.

Tip educativo: Puede usarse con la versión gratuita; anima a los alumnos a usarlo para organizar sus entregas y estudiar.

¿Qué es Asana?

- Plataforma de gestión del trabajo en equipo: ayuda a planificar, organizar, seguir tareas y proyectos. [Asana](#)
- Muy útil en entornos donde hay varias fases, equipos, responsabilidades, plazos.

Puntos para los alumnos:

- Crear un “Proyecto” para clase o para una iniciativa extra-escolar.
- Añadir tareas, fechas límite, asignar a miembros del equipo.
- Usar vistas de lista, tablero, cronograma para ver cómo progresa.
- Ideal para trabajos más complejos que un simple tablero de tareas; puede ser una buena “práctica laboral”.

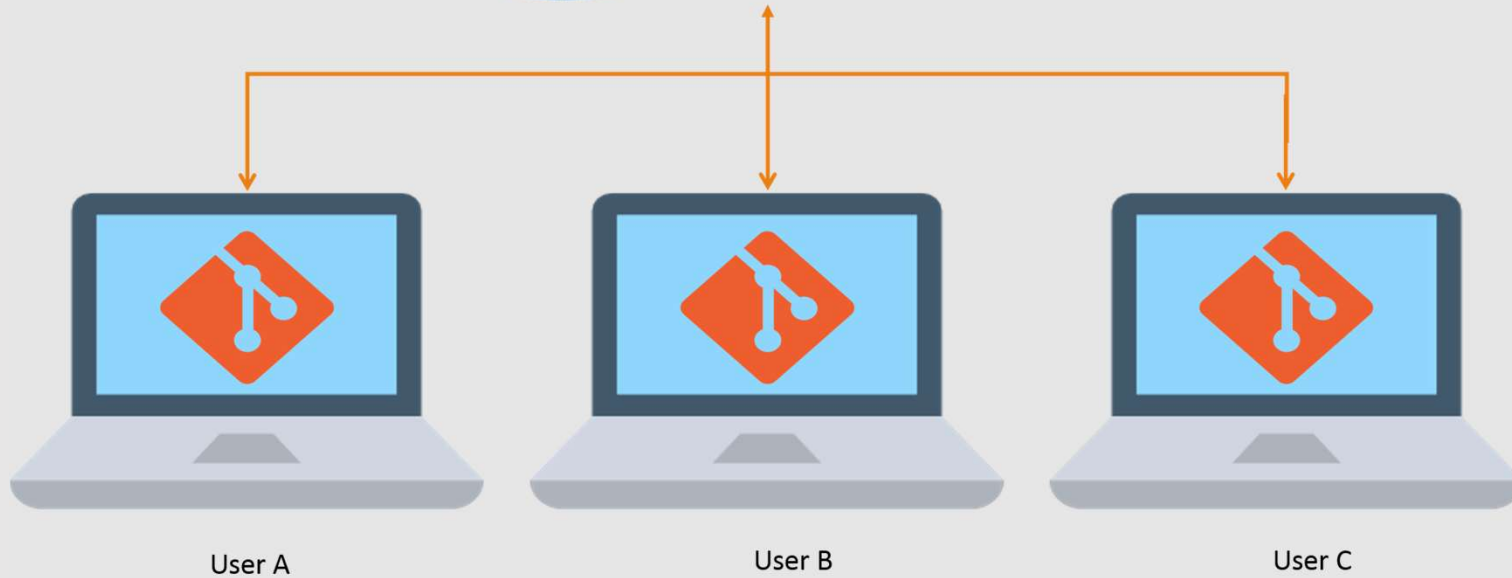
Extra: Git y GitHub

Sistemas de Control de Versiones Distribuido

GitHub: Gestor de proyectos

01

Git y GitHub



PROMETEO

01 Git



¿Por qué Git?

- **Git es un sistema libre.** No puede haber limitaciones contractuales sobre el uso de Git. Sea cual sea el número de colaboradores, de proyectos o actualizaciones, Git siempre será gratuito.
- Git posee también todos los beneficios derivados de los **DVCS**. Es decir, que es posible trabajar en modo desconectado con su repositorio manteniendo las características avanzadas de colaboración cuando se está conectado.
- Git tiene un **rendimiento excelente**. Git incorpora muchas herramientas internas para mejorar su rendimiento (en particular para el uso de las ramas).
- Git es actualmente el sistema más **popular** y gana en popularidad cada año. Es difícil contar con estadísticas precisas sobre los sistemas de gestión de versiones instalados y usados en las empresas.

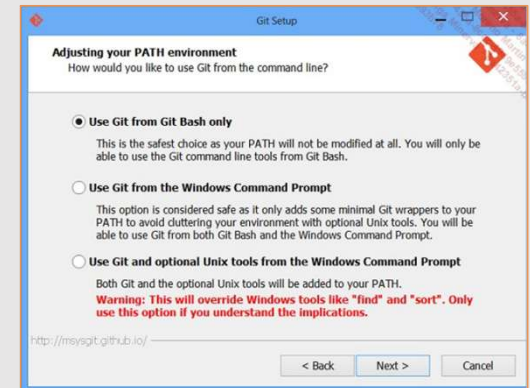
01 Git

Instalación en Linux

- Para instalar Git en un sistema Debian a partir de paquetes preexistentes, se puede utilizar el gestor de paquetes apt-get. Después de haber abierto el CLI, debemos utilizar el siguiente comando para instalar Git: `sudo apt-get install git`
- Una vez que la instalación ha terminado, para probar la instalación y obtener la versión de Git que fue instalada, debemos utilizar el siguiente comando: `git --version`.

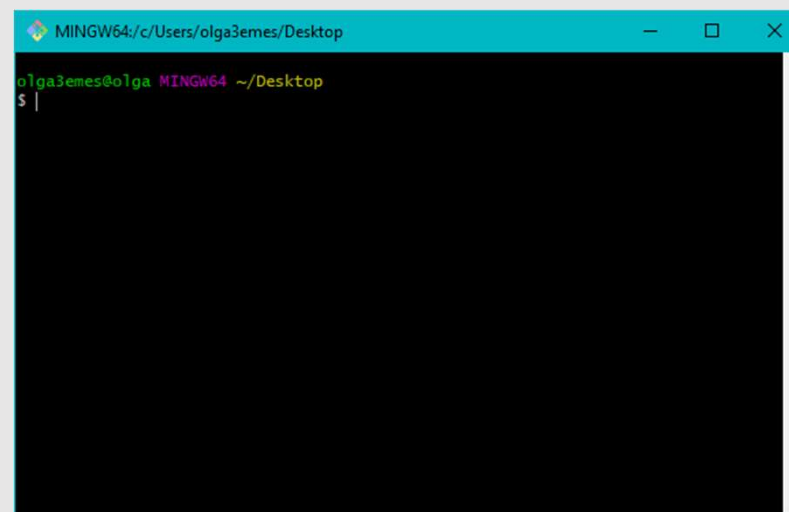
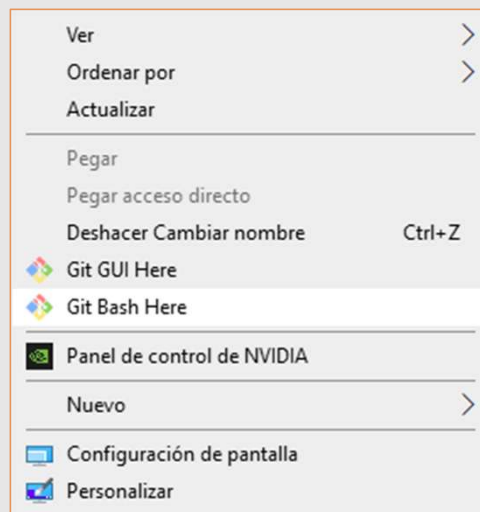
Instalación en Mac y Windows

- Para descargar Git debemos ir al sitio oficial, en la dirección : <https://git-scm.com/>
- A partir de la información enviada por el navegador, detecta la versión que se ha de instalar y ofrece un enlace de descarga en la parte inferior derecha en una imagen de Mac o Windows. Hay que descargar la aplicación. Y seguir el instalador.
- Con Mac también podemos usar brew .



01 Git

Una vez instalado Git, en nuestro sistema Windows aparecerá algo parecido a:



Mientras que en Linux y Mac no tiene por qué aparecer, dado que sus terminales ya soportan los comandos necesarios del kernel de Linux.

01

Git y GitHub

¿Qué es Github?

Plataforma para la gestión de repositorios Git y desarrollo colaborativo.

Para hacer uso de Github, nos basta con registrarnos en:
<https://github.com/>

Al crear una cuenta debemos verificar nuestro correo electrónico.

Se recomienda encarecidamente el uso de una dirección de correo electrónico profesional.

<https://learn.microsoft.com/es-es/shows/visual-studio-code/using-git-with-visual-studio-code-official-beginner-tutorial>



Comandos básicos

Después de haber instalado Git, es muy útil saber cómo acceder a su ayuda. Puede ser necesario obtener más información sobre un comando y para ello necesitaremos la ayuda. Para mostrar la ayuda general que lista los comandos principales de Git, hay que utilizar el siguiente comando:

```
git --help
```

Este comando permite tener una lista y una breve explicación (en inglés) de los comandos principales de Git.

Para obtener una ayuda más completa sobre un comando específico de Git, hay que utilizar la siguiente sintaxis:

```
git comando --help
```

Git propone una serie de comandos que se dividen en dos categorías:

- Los comandos de porcelana (porcelain command): Estos son los comandos de alto nivel. Son los comandos que se utilizan a diario y para los que Git propone una serie de controles. Ej.: git commit o git add.
- Los comandos de fontanería (plumbing commands): Estos son los comandos de bajo nivel, es decir, que manipulan directamente la información que constituye el núcleo de Git. Estos comandos pueden ser destructivos para el repositorio y suelen ser aplicados por usuarios que dominan Git.

Para ver todos hay que usar:

```
git help -a
```

Comandos básicos

Configuración requerida

Existen dos parámetros de configuración que son necesarios antes de cualquier uso de Git. Estos parámetros de configuración son la dirección de e-mail del usuario del repositorio y su nombre.

1. Configurar el nombre de usuario

```
git config --global user.name "Nombre Apellido"
```

2. Configurar el e-mail de usuario

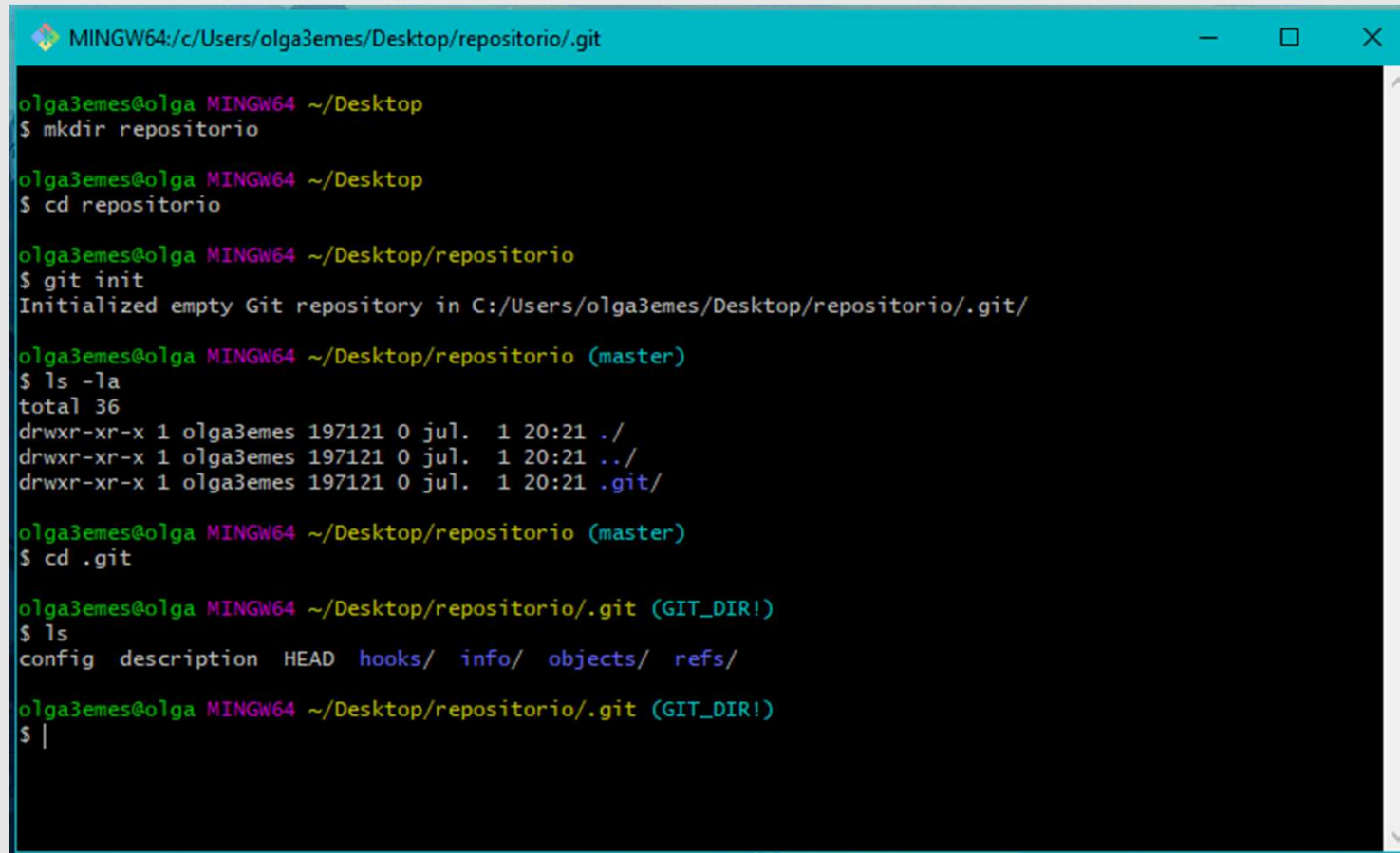
```
git config --global user.email email@dominio.com
```

Ya estamos listos para inicializar un repositorio local, usaremos:

```
git init
```

¡AVISO! Si tu ordenador está compartido con otros compañeros es muy posible que si usas Windows, necesites dirigirte al Administrador de credenciales y borrar los datos de aquellos usuarios que no estén en este momento utilizando el PC.

Comandos básicos



```
MINGW64:/c/Users/olga3emes/Desktop/repositorio/.git

olga3emes@olga MINGW64 ~/Desktop
$ mkdir repositorio

olga3emes@olga MINGW64 ~/Desktop
$ cd repositorio

olga3emes@olga MINGW64 ~/Desktop/repositorio
$ git init
Initialized empty Git repository in C:/Users/olga3emes/Desktop/repositorio/.git/

olga3emes@olga MINGW64 ~/Desktop/repositorio (master)
$ ls -la
total 36
drwxr-xr-x 1 olga3emes 197121 0 jul.  1 20:21 ./
drwxr-xr-x 1 olga3emes 197121 0 jul.  1 20:21 ../
drwxr-xr-x 1 olga3emes 197121 0 jul.  1 20:21 .git/

olga3emes@olga MINGW64 ~/Desktop/repositorio (master)
$ cd .git

olga3emes@olga MINGW64 ~/Desktop/repositorio/.git (GIT_DIR!)
$ ls
config  description  HEAD  hooks/  info/  objects/  refs/

olga3emes@olga MINGW64 ~/Desktop/repositorio/.git (GIT_DIR!)
$ |
```

Comandos básicos

Concepto básico: "Las 3 zonas de un repositorio Git"

Working Copy | Staging Area | Repository

working copy: la carpeta de nuestro proyecto

staging area: donde pondremos los archivos a "trackear" (también se le conoce por index o caché)

repository: donde se almacena toda la información de los cambios

Comparar entre las 3 zonas:

git diff (Working Copy → Staging Area)
git diff --staged (Staging Area → Repositorio)
git diff HEAD (Working Copy → Repositorio)



Extensión para ver los cambios:

<https://marketplace.visualstudio.com/items?itemName=mhutchie.git-graph>

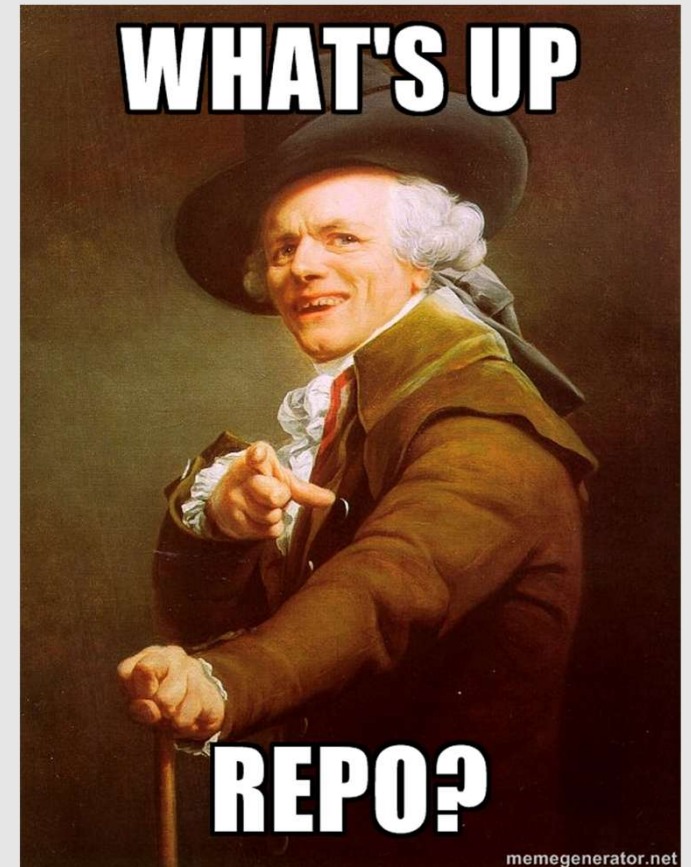
comandos básicos

¿CÓMO ESTÁ MI REPOSITORIO?

git status

Nos resume el estado de nuestro repositorio:

- qué archivos están en el staging area y cuales no
- qué archivos son "trackeados"
- qué archivos han sido modificados



Comandos básicos

DE WORKING COPY A STAGING AREA

`git add`

Añade el archivo al staging area

`git add <filename>`

Añade el directorio al staging area

`git add <folder>`

Añade los archivos .md al staging area

`git add *.md`

¿Y si me equivoco?

`git reset HEAD o <filename> o <folder> o *.md`

`git checkout --<filename>`

Para que sea más fácil:

Añadimos al staging area todos los cambios.

`git add -A / --all`

Añadimos los cambios y nuevos archivos, pero no los eliminados.

`git add .`

Añadimos los archivos modificados y eliminados, pero no los nuevos.

`git add -u`

Comandos básicos

DEL STAGING AREA AL REPOSITORIO:

Hacemos un commit

¿Qué es un commit? Hacer un commit es como “hacer una foto”

Vale..., pero ¿qué es un commit?

- Un commit es un paquete que contiene:
- Uno o más «hunks» (el resultado de un diff)
- Un mensaje que describe qué cambios van en este commit
- Un hash SHA para identificar el commit (ID)
- Un enlace con su "commit padre"

`git commit`

Nos abrirá nuestro editor de texto para escribir un mensaje descriptivo del commit. Guardando el comentario como si fuera un archivo, creará el commit.

Hace el commit sin abrir el editor de texto usando el mensaje tras -m.

`git commit -m "Mensaje rápido para el commit"`

¡ATENCIÓN! Sólo se guardará lo que esté en el staging area



Comandos básicos

DESHACIENDO UN COMMIT

Para deshacer el último commit usaremos:

`git reset`

Dos estrategias:

- Deshacer el último commit pero mantener lo que había en mi working copy. Nuestro staging area queda vacío.

`git reset HEAD~1`

(o el ID del commit para retroceder más)

- Deshacer el último commit y lo que había en mi working copy de manera que todo quede como estaba antes. Nuestro staging area queda vacío.

`git reset --hard HEAD~1`

(o el ID del commit para retroceder más)

comandos básicos

VER LOS COMMITS

¿Cómo puedo ver los commits de mi repositorio?

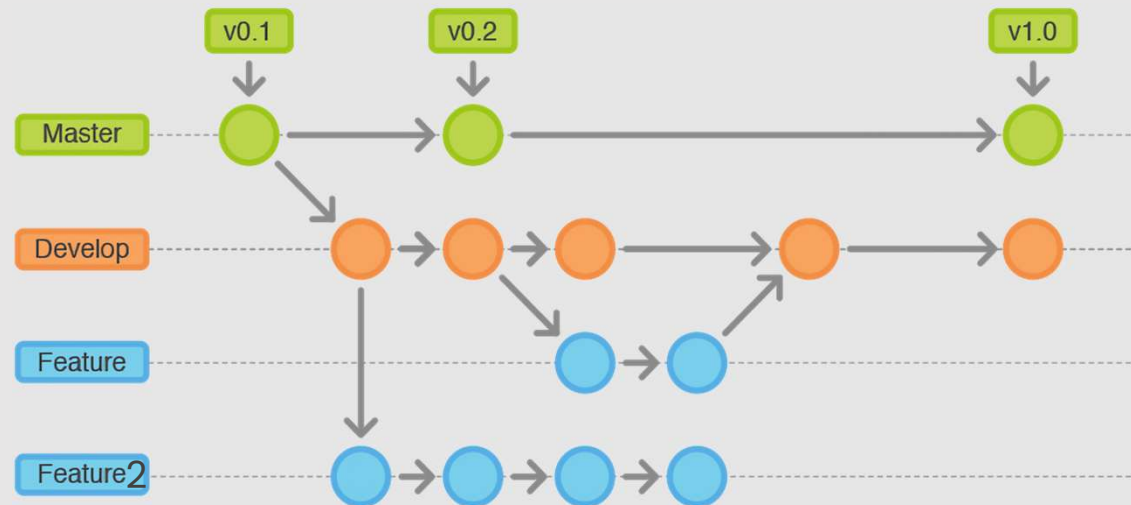
```
git log --graph --decorate --pretty=oneline  
git reflog
```



Ramas

RAMAS O BRANCHES, PARA LOS AMIGOS

- ¿Qué son?
Son directorios virtuales. Universos paralelos con un punto en común.
- ¿Para qué sirven?
Diferentes puntos de entrada al grafo.
Nos permiten desarrollar diferentes cosas en paralelo.



Ramas



¿CÓMO TRABAJAMOS CON LAS RAMAS?

- Ver las ramas existentes:

`git branch`

- Para crear una rama nueva usamos:

`git branch <NombreDeLaRama>`

¡El contenido de working copy y staging area no varía!

- Cambiar de una rama a otra:

`git checkout <NombreDeLaRama>`

¡El contenido de working copy varía!

- Eliminar una rama:

`git branch -D <NombreDeLaRama>`

¡El contenido de working copy y staging area no varía!

- No es más que eliminar un puntero del grafo (un punto de acceso al grafo)
- ¡Ojo! podemos dejar commits “inalcanzables”
- No podemos eliminar la rama en la que estamos

Ramas

MERGING: UNIENDO RAMAS

Para unir dos ramas usaremos:

`git merge <NombreDeLaRama>`

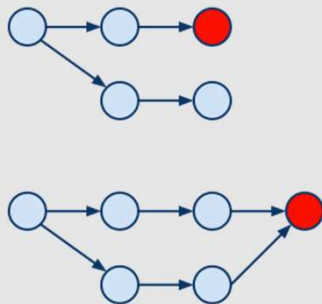
¿Quién absorbe a quién?

- La rama en la que estamos, absorbe la que le indicamos con git merge

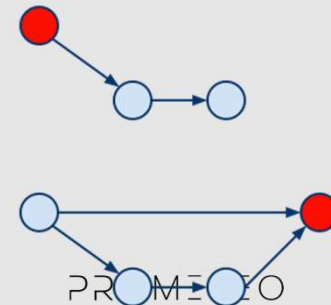
- Ejemplo:

Si estamos en 'carneAsada' y queremos hacer merge con 'salsa'
`git merge salsa`

- Cuando las ramas forman una lista. Hay dos opciones:
Con fast-forward (por defecto)



Sin fast-forward (`--no-ff`)



Ramas

RESOLVER CONFLICTOS

Cuándo hago git pull o git merge, únicamente, mi repositorio dice que hay un conflicto, es decir, da error... ¿Ahora qué hago?

1. Editar cada uno de los archivos en conflicto, quedándonos con el código que realmente nos interesa.
2. Hacer git add de esos archivos al staging-area.
3. Hacer git commit con los cambios.
4. Actualizar el repositorio remoto.

Para cancelar el merge que estábamos haciendo dejando todo como estaba.

`git merge --abort`



Trabajando en remoto

Para **CREAR UN REPOSITORIO REMOTO** podemos usar **Github**.

- Una vez creado podemos clonarlo con:
`git clone <repoUrl>`
- Es decir, copiamos un repositorio remoto para trabajar en nuestra máquina.
- Nos crea una carpeta con el mismo nombre que el repositorio.
- Ésta es la opción más cómoda
- También podemos crear un repositorio remoto usando:
`git remote add origin <UrlRepo>`
Añade <NombreRepo> como repositorio remoto



Olga M. Moreno Martín

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner *

Repository name *

olga3emes / reponame ✓

Great repository names are short and memorable. Need inspiration? How about [jubilant-chainsaw](#)?

Description (optional)

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☒ Add a README file

This is where you can write a long description for your project. [Learn more.](#)

☒ Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: Laravel

☒ Choose a license

A license tells others what they can and can't do with your code. [Learn more.](#)

License: Creative Commons Z...

This will set `master` as the default branch. Change the default name in your [settings](#).

Grant your Marketplace apps access to this repository

You are subscribed to 1 Marketplace app

☒ LeanBoard

Whiteboard with sticky notes, connected with your GitHub issue

Create repository

PROMETEO

123

Trabajando en remoto

Para **PUBLICAR NUESTROS CAMBIOS** en un repositorio remoto usaremos:

```
git push  
git push <NombreRepo> <branch>
```

Sube nuestros cambios en la rama <branch> a <NombreRepo>
Si nuestro repo no está actualizado, se rechazará

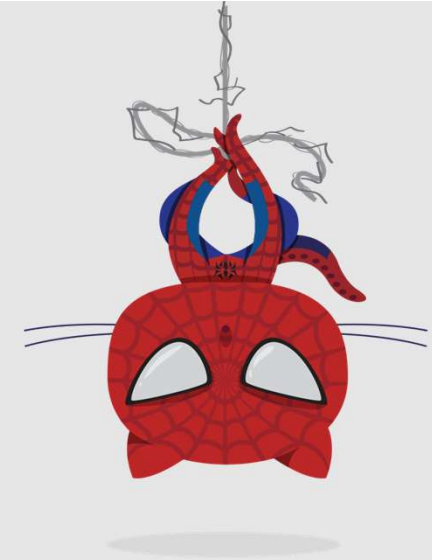
¿Cómo actualizo el repositorio?

```
git fetch  
git fetch <remote>
```

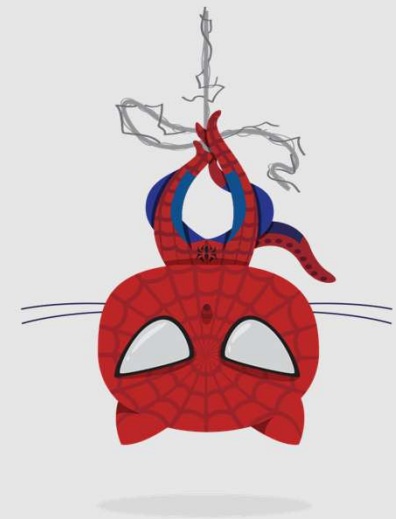
- Descarga todos los cambios de todas las ramas de <remote> desde la última vez que descargamos. Y crea nuevas ramas si es **necesario**.

```
git fetch <remote> <NombreRamaRemota>
```

- Descarga los cambios de la rama <NombreRamaRemota> de <remote> desde la última vez que descargamos. Crea una nueva rama si no la tenemos.



Trabajando en remoto



DESCARGAR Y APLICAR LOS CAMBIOS REMOTOS

```
git pull  
git pull <remote>
```

git pull = git fetch + git merge

Perfecto para hacer actualizaciones directas

Entonces, el orden de las operaciones más corrientes queda:

- 1) `git add --all`
- 2) `git commit -m`
- 3) `git pull`
- 4) `git push`

Etiquetas: Tags y versiones

ETIQUETANDO

- Git tiene la posibilidad de etiquetar puntos específicos del historial como importantes.
- Esta funcionalidad se usa típicamente para marcar versiones de lanzamiento.
- Una etiqueta se une a un commit.

- Listar las etiquetas disponibles en Git es sencillo:

`git tag`

- Crear una etiqueta:

`git tag -a <nombreEtiqueta> -m "mensajito"`

Ejemplo:

```
$ git tag -a v1.4 -m 'my version 1.4'
```

```
$ git tag
```

```
v0.1
```

```
v1.3
```

```
v1.4
```

- La opción `-m` especifica el mensaje de la etiqueta, el cual es guardado junto con ella. Si no especificas el mensaje de una etiqueta anotada, Git abrirá el editor de texto para que lo escribas.

- Eliminar una etiqueta:

`git tag --delete <nombreEtiqueta>`



GitHub: Releases – Tags

Tag

v3.4.5

Target: main

Excellent! This tag will be created from the target when you publish this release.

Release title

WritePreview

HBI<>@↵

Previous tag: auto

Generate release notes

Describe this release

Important release

Latest

This releases important things ⚡

▼ Assets2

Source code (zip)

Nov 15, 2019

Source code (tar.gz)

Nov 15, 2019

ReleasesTags

Pre-release

v1.51.0-beta0

Ba41332

Compare

1.51.0-beta0

atom-build released this 16 days ago

Notable Changes

- #20899 - Improve the performance of getting the list of all available atom packages

All Changes

Assets18

Latest release

v1.50.0

c8e5286

Compare

1.50.0

atom-build released this 16 days ago

Notable Changes

- #20804 - Fix multi cursor correct setting
- #20910 - Add async and await keywords to Rust highlighter

PROMISEO

GitHub: Issues

File Edit View Repository Branch Help

Current repository
SER421

Current branch
master

Fetch origin
Last fetched 7 minutes ago

Issues

Filters

Labels Milestones

New issue

436 Open ✓ 3,039 Closed

Author Labels Projects Milestones Assignee Sort

Clone A Repository Dialog's Statefulness Is Bad UX
#5937 opened 6 hours ago by dsanders11 ||| New Issues

2

can't push to repository in app (publish repository modal displayed) bug priority-2
#5936 opened 6 hours ago by FaithInMotion ||| New Issues

8

File selection of disabled entries possible with space help wanted priority-3
#5934 opened 13 hours ago by biolauri ||| New Issues

1

open in external editor visual studio code more-information-needed
#5933 opened 14 hours ago by edgale ||| New Issues

1

It's unclear which branch is being merged into another in the merge dialog enhancement
good first issue help wanted
#5930 opened a day ago by iAmWillShepherd 1.4.4 ||| New Issues

 1

Add a button to the branch selector enhancement good first issue help wanted
#5929 opened a day ago by iAmWillShepherd 1.4.4 ||| New Issues

 1

lib/git/commit functions could return SHA on success enhancement help wanted tech-debt
#5923 opened a day ago by outofambit ||| New Issues

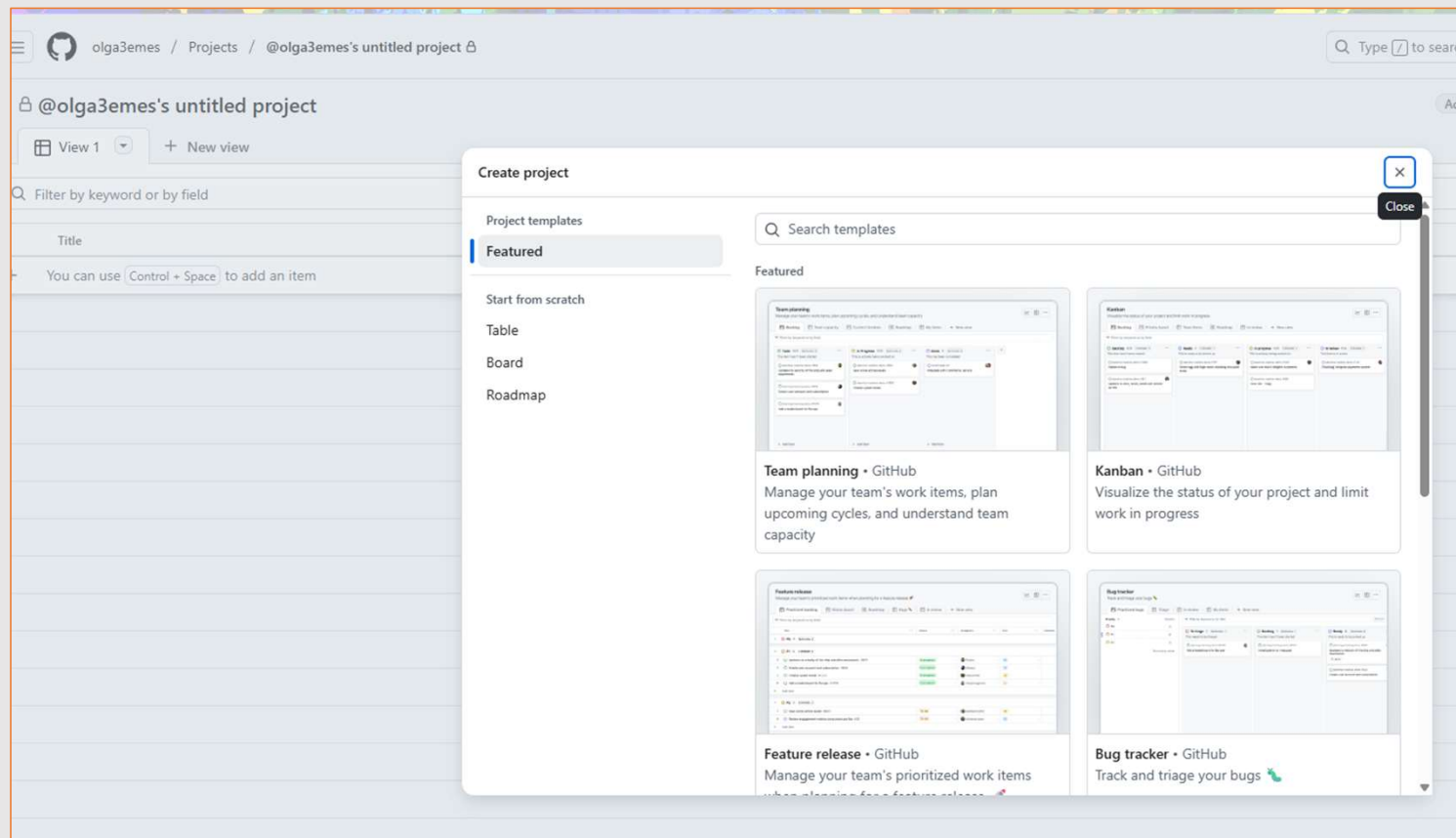
3

command line more-information-needed
#5922 opened 2 days ago by wlooseven ||| New Issues

1

GitHub : Gestor de Proyectos

- <https://learn.microsoft.com/es-es/training/paths/manage-project-lifecycle-github/>



Uso de Git y GitHub de forma profesional

Apuntes extra de cómo utilizar Git – GitHub con SSH

Configuración con clave SSH

Verifica si ya existe una clave SSH

- Primero, revisa si ya tienes generadas claves SSH para tu maquina. Abre la terminal e ingresa el siguiente comando:

`ls -al ~/.ssh`

- Si ya generaste claves SSH, deberías ver una respuesta similar a esta:

```
-rw----- 1 user_name user_name 1766 Jul 7 2018 id_rsa
-rw-r--r-- 1 user_name user_name 414 Jul 7 2018 id_rsa.pub
-rw----- 1 user_name user_name 12892 Feb 5 18:39 known_hosts
```

- Si tus claves ya existen, avanza a la diapositiva “Copiar tu clave SSH pública”

Configuración con clave SSH

- Si no ves ninguna respuesta o recibes un mensaje de que el archivo o directorio no existe (No such file or directory), entonces ejecuta en la terminal:

```
mkdir $HOME/.ssh
```

- Luego genera un nuevo conjunto de claves con:

```
ssh-keygen -t rsa -b 4096 -C your@email.com
```

- Ahora revisa que tus claves existan con el comando `ls -al ~/.ssh` y asegúrate de que la salida de la terminal es similar a la mostrada mas arriba.

- Las claves SSH siempre son generadas como un par de claves públicas (id_rsa.pub) y privadas (id_rsa). Es extremadamente importante que nunca reveles tu clave privada, y solo uses tu clave pública para cosas como la autenticación en GitHub.

Agrega tu clave SSH al ssh-agent

`ssh-agent` es un programa que se inicia cuando inicias sesión y almacena tus claves privadas. Para que funcione correctamente, necesita estar ejecutándose y tener una copia de tu clave privada.

- Primero, asegúrate de que `ssh-agent` esta ejecutándose con:
`eval "$(ssh-agent -s)"` # for Mac and Linux

o:

`eval `ssh-agent -s``

`ssh-agent -s` # for Windows

- Luego, añade tu clave privada al `ssh-agent` con:
`ssh-add ~/.ssh/id_rsa`

Copia tu clave SSH pública

A continuación, debes copiar tu clave SSH pública en el portapapeles:

- Para Linux o Mac, imprime el contenido de tu clave pública a la consola con:

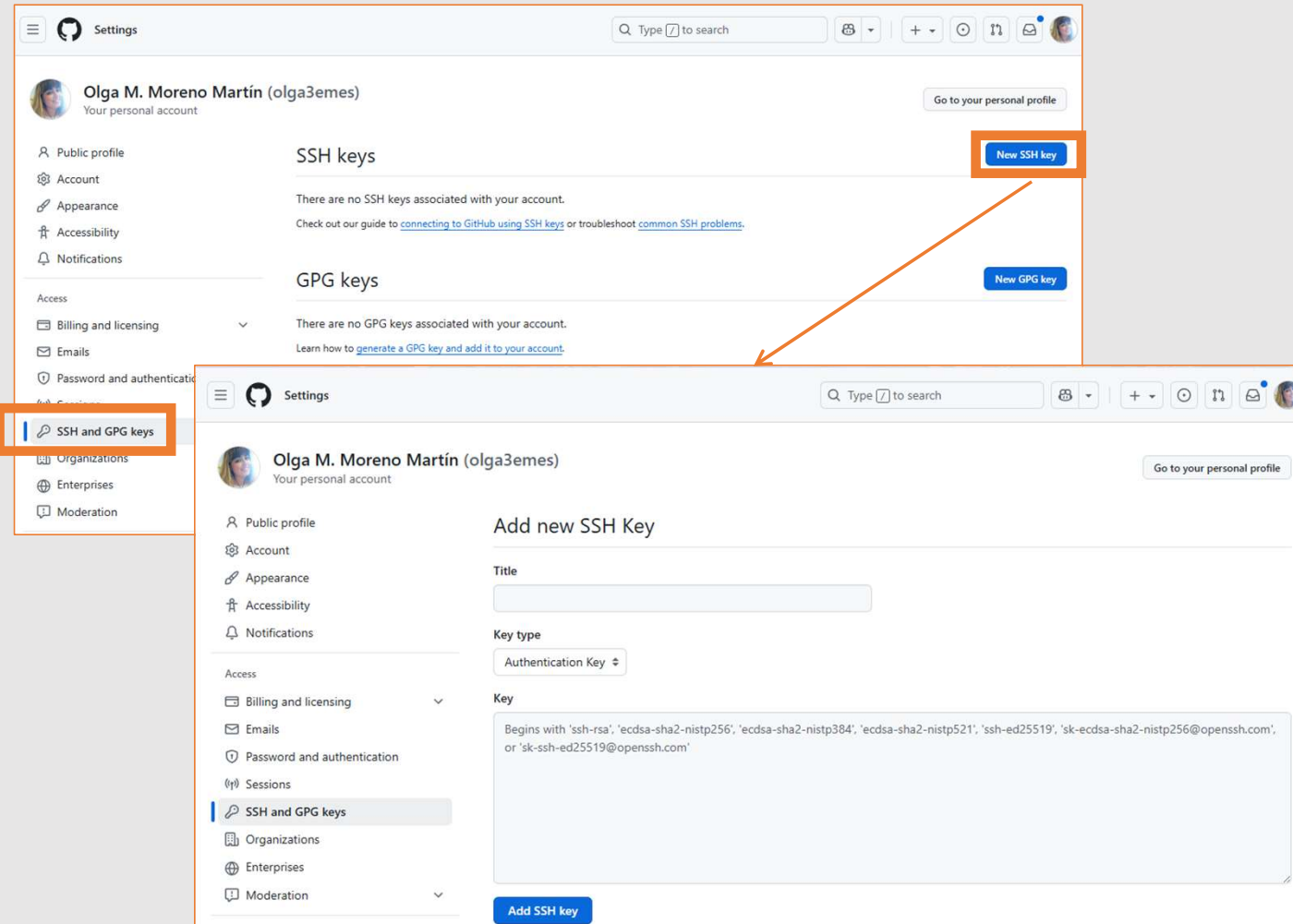
```
cat ~/.ssh/id_rsa.pub # Linux
```

- Luego selecciona y copia la respuesta.
- O para Windows simplemente ejecuta:

```
clip < ~/.ssh/id_rsa.pub # Windows
```

Añade tu clave pública SSH a GitHub

- Ve a la página de ajustes (Settings) de GitHub, click en SSH and GPG keys y en dar al botón "New SSH key":
- Luego ingresa un nombre reconocible y pega tu clave pública (id_rsa.pub) :



Comprueba la autenticación

- Finalmente, en la terminal vamos a ver si todo ha ido bien con:

```
ssh -T git@github.com
```

- Si seguiste correctamente todos los pasos, deberías ver este mensaje:

```
Hi your_user_name! You've successfully authenticated,  
but GitHub does not provide shell access.
```

¡Ya hemos terminado!





PROMETEO



PROMETEO

Unión práctica Unidades 1,2,3

Licencia



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

¿Cómo crear un entorno local de WordPress para Git?

- Antes de utilizar GitHub, crea una versión local de tu sitio WordPress. Este sitio de desarrollo local es para probar cambios, incluyendo nuevos temas, plugins o diseños.
- Cualquier modificación en tu sitio local de WordPress no afectará al sitio web en vivo hasta que la despliegues.
- Herramientas de WordPress local
- Usa una herramienta de desarrollo web local para crear un entorno WordPress en tu máquina. Según sus funciones, existen dos tipos de herramientas: las de propósito general y las de desarrollo de WordPress.
- Las herramientas de propósito general te permiten crear cualquier entorno de desarrollo web. Son compatibles con todos los Sistemas de Gestión de Contenidos (CMS), pero debes instalar el CMS manualmente.
- Aunque puedes utilizar herramientas de uso general para el desarrollo de WordPress, te recomendamos las específicas para WordPress. Están preinstaladas con WordPress, lo que las hace más prácticas.

¿Cómo crear un entorno local de WordPress para Git?

- Para este tutorial, utilizaremos **Local WP**. Esta herramienta de desarrollo tiene una interfaz gráfica de usuario (GUI), lo que la hace fácil de usar e ideal para desarrolladores web principiantes. Sigue estos pasos para configurar Local:
- Visita la página web de Local llamada "Releases" donde se encuentran últimos cambios de sus productos.
- Descarga la última versión de Local para tu sistema operativo.
- <https://localwp.com/releases/>
- Ejecuta el instalador local. Sigue la guía de instalación según tu sistema operativo.
- Una vez finalizado el proceso de instalación de WordPress Local, crea un sitio de desarrollo local
- Abrir Local. Por ahora, sáltate el paso de creación de la cuenta haciendo clic en el botón X.
- Y crea tu sitio web con Wordpress.

Local

What's your site's name?

miblog

Advanced options ^

Local site domain

miblog.local

Local site path

~\Local Sites\miblog

Browse

Go back

Continue

1 Set up site

2 Set up environment

3 Set up WordPress

Choose your environment



Preferred

Custom

PHP 8.2.27 | Web server nginx | MySQL Version 8.0.35

Go back

Continue

✓ Set up site

2 Set up environment

3 Set up WordPress

Set up WordPress



WordPress username

user

WordPress password

••••



WordPress e-mail

olga.moreno@thepowr

Advanced options ^

Select Language

Español



Is this a WordPress Multisite?

No



Go back

Add Site



Set up site



Set up environment



Set up WordPress

Blog

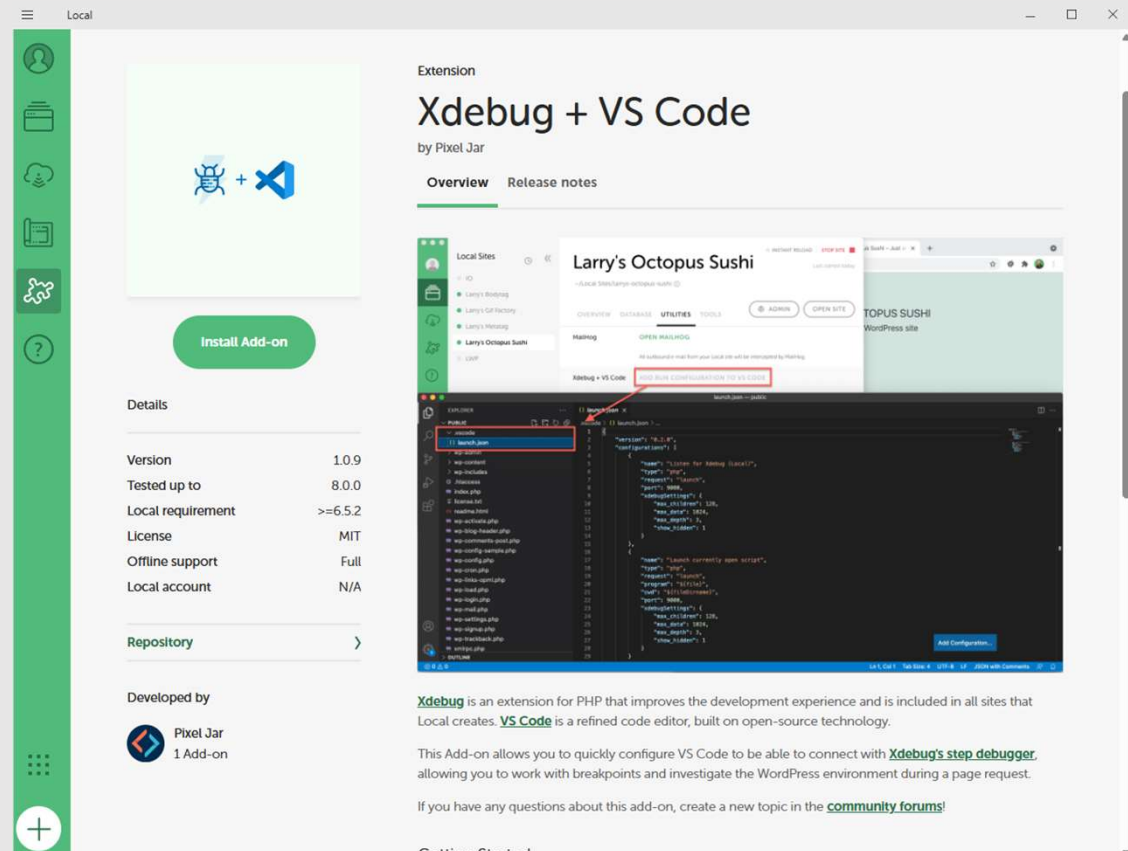
¡Hola, mundo!

Te damos la bienvenida a WordPress. Esta es tu primera entrada. Edítala o bórrala, ¡luego empieza a escribir!

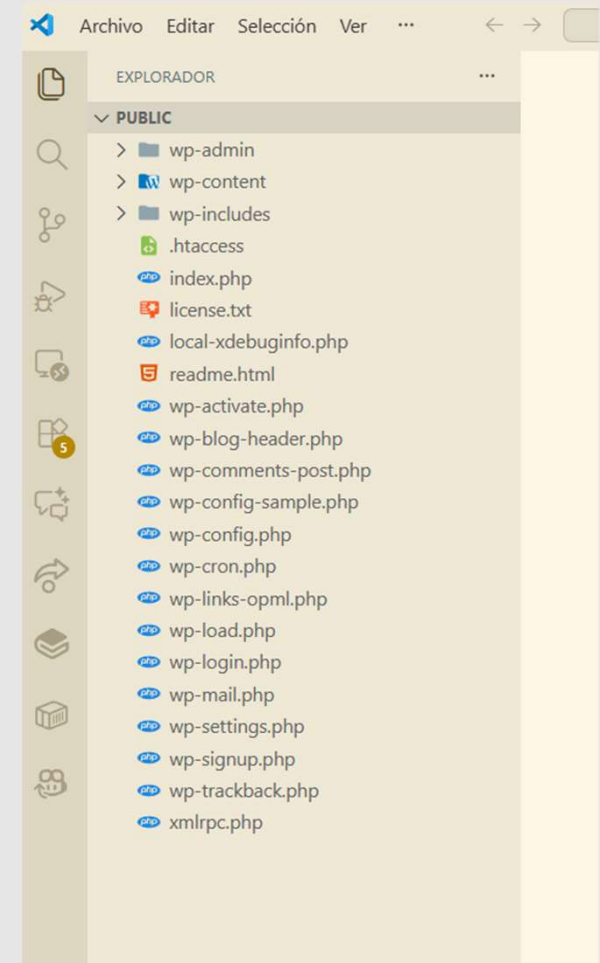
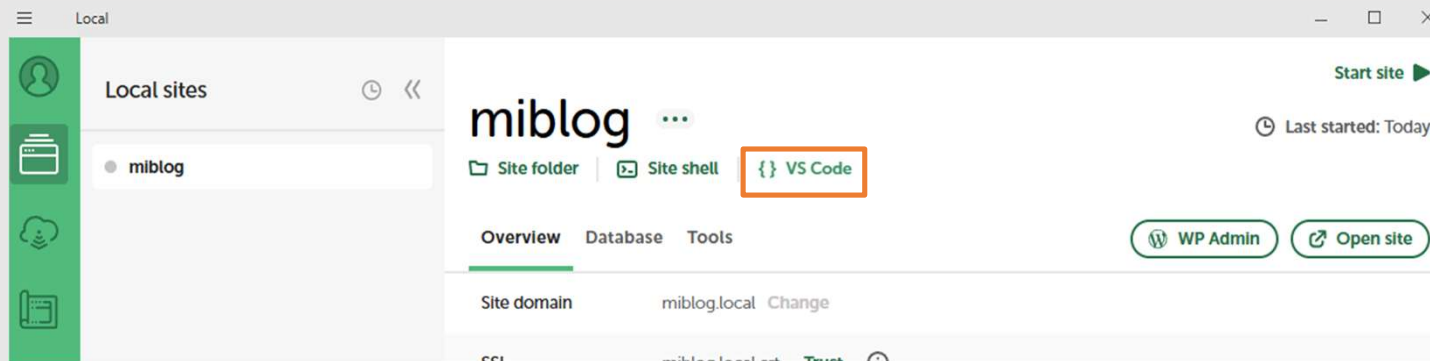
17 de noviembre de 2025

¿Cómo crear un entorno local de WordPress para Git?

- Una vez que tu entorno local esté listo, instala un editor de texto para cambiar los datos de tu sitio. Con Local, puedes instalar VSCode directamente desde su panel de control:
- Desde el panel de control de Local, abre el menú de **complementos** representado por el icono del rompecabezas en la barra lateral izquierda.
- Aparecerá la ventana de instalación. Haz clic en **Install Add-on** (Instalar complemento).
- Espera a que Local termine de instalarse y se reinicie.



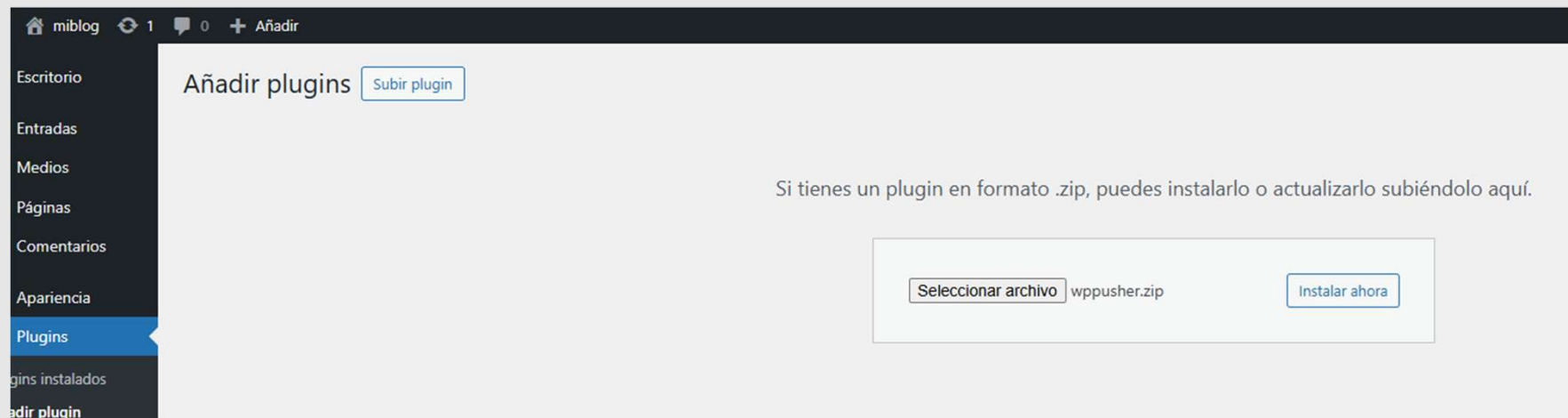
Cómo utilizar GitHub para el desarrollo de WordPress



PROMETEO

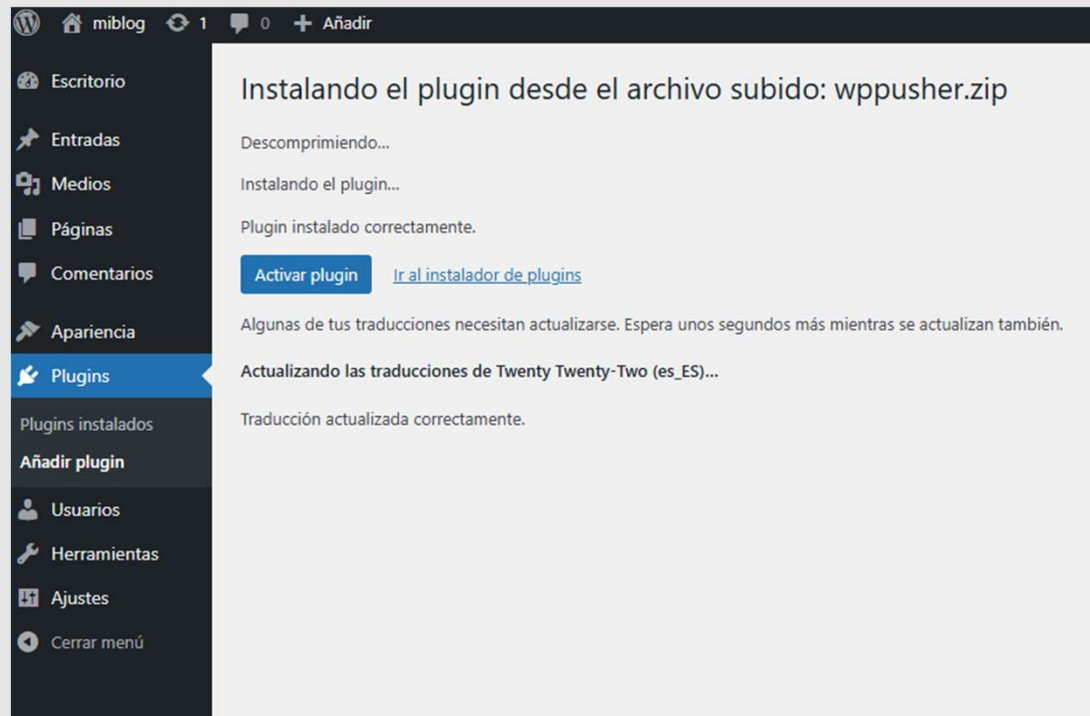
Cómo utilizar GitHub para el desarrollo de WordPress

- Después de preparar todas las herramientas, puedes empezar a utilizar GitHub para trabajar en el desarrollo de tu sitio.
- Nosotros vamos a utilizar un plugin WP Pusher . Lo puedes descargar aquí: <https://wppusher.com/>



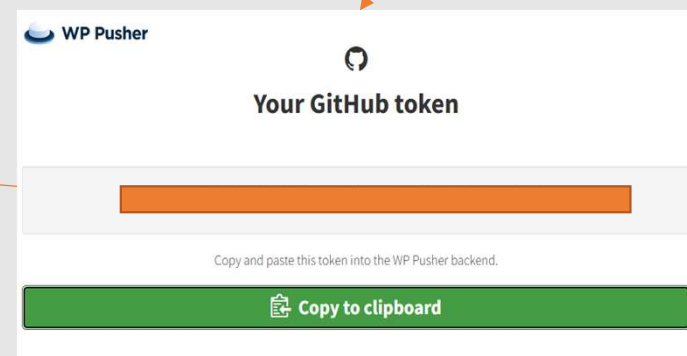
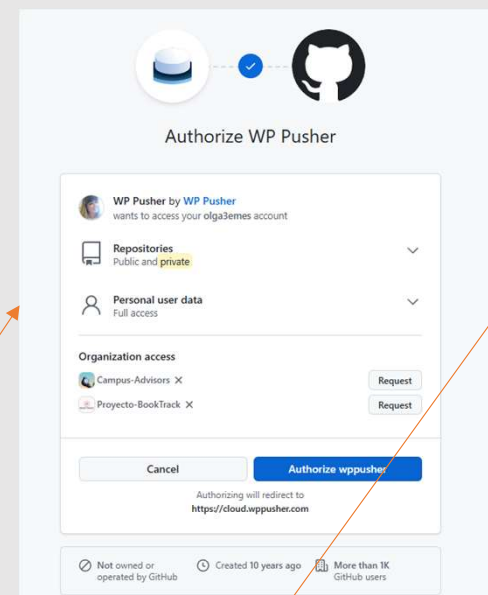
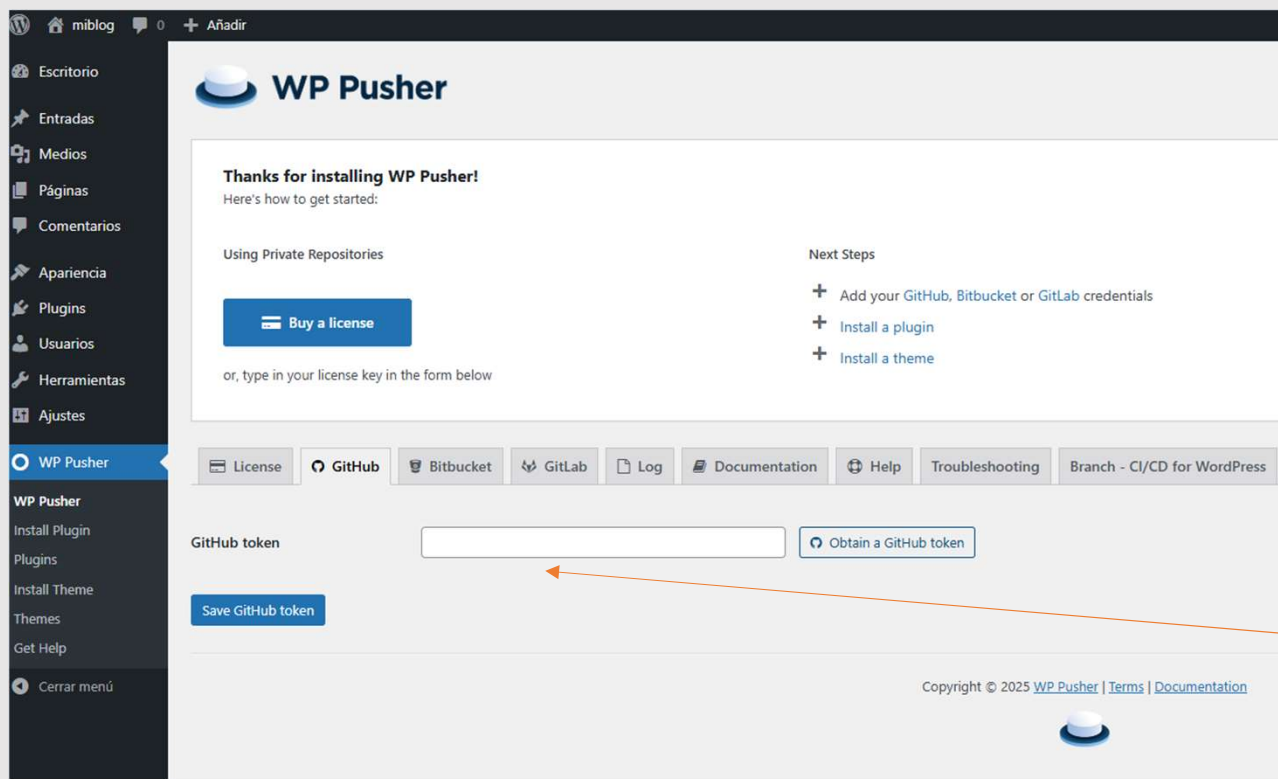
Cómo utilizar GitHub para el desarrollo de WordPress

- Y lo activamos



Cómo utilizar GitHub para el desarrollo de WordPress

- En tu barra lateral de WordPress, navega hasta **WP Pusher**.
- Ve a la **pestaña GitHub** y haz clic en **Obtener un token de GitHub**.



Cómo utilizar GitHub para el desarrollo de WordPress

- Ve a la barra lateral de WordPress → WP Pusher → Instalar tema o Instalar plugin. En este ejemplo, instalaremos un tema.
- Rellena el campo Repositorio de temas con el nombre de tu repositorio de GitHub.
- En el campo Rama del repositorio, introduce main.
- Haz clic en el botón Instalar tema.
- Tras la instalación, ve a Apariencia → Temas → Activar el tema recién instalado.
- Siempre que introduzcas cambios en los repositorios, ve a WP Pusher → Plugins o Temas → Actualizar. De lo contrario, tus cambios no aparecerán en el sitio activo.
- <https://github.com/ColorlibHQ/Activello>
- Podemos ver cómo es aquí:
- <https://colorlib.com/wp/themes/activello/>

¿Y si queremos utilizar un servidor?

- Recordatorio: pila LAMP o LEMP. (Podemos usar Docker o un servidor con FTP)
- Código en GitHub
- Base de datos MySQL (posibilidad de exportar/importar)



¿Preguntas?

Espacio abierto para dudas, comentarios o reflexiones sobre lo visto hoy

PROMETEO

Unidad 4



Licencia



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

HTML – CSS – JS

Diapositivas de repaso en la plataforma

PROMETEO



PROMETEO

PROMETEO

Unidad 5



Licencia



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice

1. PHP
2. Código Embebido
3. Generando Contenido
4. Comentarios
5. Errores
6. Variables
7. Constantes
8. Operadores
9. Trabajando con formularios
10. Condiciones
11. Bucles
12. Arrays
13. Funciones

PHP

- Acrónimo de **P**ersonal **H**ome **P**age
- Lenguaje de propósito general, aunque su fuerte es el desarrollo web.
- Sintaxis similar a C / Java
- El código se ejecuta en el servidor (en Apache mediante mod_php)
- El cliente recibe el resultado generado tras interpretar el código en el servidor.
- El código se almacena en archivo con extensión .php.
- La **última versión es la 8.5**, de 2025. La versión 8.0 salió en Diciembre de 2020. Además de numerosas nuevas funcionalidades que iremos viendo durante el curso, tiene más de dos veces mejor rendimiento que PHP5.
- Su documentación es extensa y está traducida: <https://www.php.net/manual/es/>.



Código embebido

- Los bloques de código se escriben entre `<?php` y `?>`, mientras que las sentencias se separan mediante `;`

ej1

EJ1

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content=
"IE=edge">
  <meta name="viewport" content=
"width=device-width, initial-scale=1.0">
  <title>PHP</title>
</head>
<body>
  <!-- Muestra una frase con HTML -->
  Hola mundo<br>
  <!-- Muestra una frase con PHP -->
  <?php echo "Es muy fácil programar en PHP."; ?>
</body>
</html>
```



Sólo etiquetas de apertura

Si nuestro código sólo va a contener código PHP y nada de html, como por ejemplo, cuando codifiquemos clases o interfaces, sólo pondremos la etiqueta de apertura, para así indicar que es un archivo de php puro.

Generando contenido

Tenemos tres posibilidades a la hora de generar contenido en nuestros documentos PHP:

- `echo expresión;`
- `print (expresión);`
- `<?= expresión ?>`

Las que vamos a utilizar son **echo** cuando lo hagamos dentro de un bloque de instrucciones y **<?=** cuando sólo vayamos a mostrar el valor de una variable dentro de un fragmento HTML.

Comentarios

Podemos utilizar comentarios de una línea o de bloque:

ej3

EJ3

```
<?php
// Este es un comentario de una sola línea
/*
Este es
un comentario
que ocupa
varias líneas
*/
?>
```

Errores

- Si hay un error de ejecución, se produce un Fatal Error.

```
1 | Fatal error: Uncaught Error: Call to undefined function plint() in  
  | C:\xampp\htdocs\202echo.php:11  
2 | Stack trace:  
3 | #0 {main}  
4 |   thrown in C:\xampp\htdocs\202echo.php on line 11
```

- Desde PHP 5 se lanzan como una excepción. Más adelante veremos el uso de try / catch.

Variables

- No es necesario declararlas previamente.
- Comienzan por \$, por ejemplo \$nombre. Tras el \$, el siguiente carácter debe ser una **letra en minúscula** (recomendación) o guión bajo `_`. Luego ya se pueden poner números.
- Son case sensitive: `$var != $vAR`
- No se declara su tipo, el tipado es dinámico. Se asigna en tiempo de ejecución dependiendo del valor asignado.
- Conveniente inicializarlas, sino dan error.

Aunque a priori no hay tipos de datos, internamente PHP trabaja con cuatro tipos escalares: boolean, integer, float y string y cuatro tipos compuestos: array, object, callable e iterable. Existe un tipo especial para null.

(Más información en <http://php.net/manual/es/language.types.null.php>).

EJ4

Constantes

Son variables cuyo valor no varían. Existen dos posibilidades:

- `define(NOMBRE, valor);`
- `const NOMBRE; // PHP > 5.3`
- Se declaran siempre en **MAYÚSCULAS**
- Hay un conjunto de constantes ya predefinidas, también conocidas como magic constants:
<https://www.php.net/manual/es/language.constants.predefined.php>

EJ5

Operadores aritméticos

Ejemplo	Nombre	Resultado
-\$a	Negación	Opuesto de \$a
\$a + \$b	Suma	Suma de \$a y \$b
\$a - \$b	Resta	Diferencia de \$a y \$b
\$a * \$b	Multiplicación	Producto de \$a y \$b
\$a / \$b	División	Cociente de \$a y \$b
\$a % \$b	Módulo/ Resto	Resto de \$a dividido por \$b
\$a ** \$b	Potencia	Resultado de \$a elevado a \$b. PHP >= 5.6

En el caso de cadenas, si queremos concatenarlas, se utiliza el operador .

EJ6

Operadores de comparación

Ejemplo	Nombre	Resultado
<code>\$a == \$b</code>	Igual	true si \$a es igual a \$b tras conversión de tipos
<code>\$a === \$b</code>	Idéntico, Comparación Estricta	true si \$a es igual a \$b, y son del mismo tipo de dato
<code>\$a != \$b</code> <code>\$a <> \$b</code>	Diferente	true si \$a no es igual a \$b después de la conversión de tipos
<code>\$a !== \$b</code>	No idéntico	true si \$a no es igual a \$b, o si no son del mismo tipo
<code>\$a < \$b</code>	Menor que	true si \$a es estrictamente menor que \$b
<code>\$a > \$b</code>	Mayor que	true si \$a es estrictamente mayor que \$b
<code>\$a <= \$b</code>	Menor o igual que	true si \$a es menor o igual que \$b
<code>\$a >= \$b</code>	Mayor o igual que	true si \$a es mayor o igual que \$b
<code>\$a <=> \$b</code>	Nave espacial	Devuelve -1, 0 ó 1 cuando \$a es respectivamente menor, igual o mayor que \$b PHP <code>>= 7</code>
<code>\$a ?? \$b</code> <code>?? \$c</code>	Fusión de null	El primer operando de izquierda a derecha que exista y no sea null. null si no hay valores definidos y no son null. PHP <code>>= 7</code>

Operadores lógicos

Ejemplo	Nombre	Resultado
<code>\$a and \$b</code> <code>\$a && \$b</code>	And (y)	true si tanto \$a como \$b son true
<code>\$a or \$b</code> <code>\$a \$b</code>	Or (o inclusivo)	true si cualquiera de \$a o \$b es true
<code>\$a xor \$b</code>	Xor (o exclusivo)	true si \$a o \$b es true, pero no ambos
<code>!\$a</code>	Not (no)	true si \$a no es true

Operadores de asignación

Ejemplo	Nombre	Resultado
<code>\$a = \$b</code>	Asignación	Asigna a \$a el valor de \$b
<code>\$a += \$b</code>	Asignación de la suma	Le suma a \$a el valor de \$b. Equivalente a <code>\$a = \$a + \$b</code>
<code>\$a -= \$b</code>	Asignación de la resta	Le resta a \$a el valor de \$b. Equivalente a <code>\$a = \$a - \$b</code>
<code>\$a *= \$b</code>	Asignación del producto	Asigna a \$a el producto de \$a por \$b. Equivalente a <code>\$a = \$a * \$b</code>
<code>\$a /= \$b</code>	Asignación de la división	Asigna a \$a el cociente de \$a entre \$b. Equivalente a <code>\$a = \$a / \$b</code>
<code>\$a %= \$b</code>	Asignación del resto	Asigna a \$a el resto de dividir \$a entre \$b. Equivalente a <code>\$a = \$a % \$b</code>
<code>\$a .= \$b</code>	Concatenación	Concatena a \$a la cadena \$b. Equivalente a <code>\$a = \$a . \$b</code>
<code>\$a++</code>	Incremento	Incrementa \$a en una unidad. Equivalente a <code>\$a = \$a + 1</code>
<code>\$a--</code>	Decremento	Decrementa \$a en una unidad. Equivalente a <code>\$a = \$a - 1</code>

Prioridad de los operadores

Recuerda la prioridad. Primero los paréntesis, luego la negación (!), productos/divisiones, sumas/restas, comparaciones, lógicos y por último se realiza la asignación. Más información en

<https://www.php.net/manual/es/language.operators.precedence.php>

Trabajando con formularios

- Los datos se envían vía URL con el formato `var1=valor1&var2=valor2...`

Por ejemplo: `ejemplo.php?nombre=Bruce+apellido1=Wayne`

- Se divide en dos pasos:

1. Generar un formulario con `action='archivo.php' method='GET'`
2. En el archivo `.php` leer los datos con `$_GET['nombreVar']`

Vamos a separar siempre que podamos el código HTML del de PHP. Por ejemplo, el formulario lo colocamos en `saluda.html`.

El trabajo con formularios lo estudiaremos en profundidad en la UD4, y veremos que además de GET, podemos enviar los datos con POST.



EJ7

Condiciones

- La **condición simple** se realiza mediante la instrucción **if**. Entre paréntesis se pone la condición que se evalúa a true o false. Si no se ponen llaves, en vez de abrir un bloque, se ejecutará sólo la siguiente instrucción.

ej8 - if.php

```
<?php
$hora = 8; // La hora en formato de 24 horas
if ($hora === 8) {
    echo "Suenas el despertador.";
}
echo "<br>";
if ($hora === 8)
    echo "Suenas el despertador.";
?>
```

Siempre llaves

Es recomendable poner llaves siempre aunque en el momento de codificar sólo haya una única instrucción. De este modo, se queda preparado para añadir más contenido en el futuro sin provocar bugs.

Condiciones

- Las condiciones compuestas mediante if-else:

ej8: if-else.php

```
<?php
$hora = 17; // La hora en formato de 24 horas
if ($hora <= 12) {
    echo "Son las " . $hora . " de la mañana";
}
else {
    echo "Son las " . ($hora - 12) . " de la tarde";
}
?>
```

Condiciones

- Las condiciones anidadas mediante if- else if -else:

```
ej8: if - else if - else.php

<?php
$hora = 14; // La hora en formato de 24 horas
if ($hora === 8) {
    echo "Es la hora de desayunar.";
}
else if ($hora === 14) {
    echo "Es la hora de la comida.";
}
else if ($hora === 21) {
    echo "Es la hora de la cena.";
}
else {
    echo "Ahora no toca comer.";
}
?>
```

Condiciones

- La **sentencia switch** también permite trabajar con **condiciones múltiples**:

ej8: switch.php

```
<?php
$hora = 14; // La hora en formato de 24 horas
switch ($hora) {
    case 9:
        echo "Es la hora de desayunar.";
        break;
    case 14:
        echo "Es la hora de la comida.";
        break;
    case 21:
        echo "Es la hora de la cena.";
        break;
    default:
        echo "Ahora no toca comer";
}??>
```



No olvides el break

Un error muy común es olvidar la instrucción break tras cada caso. Si no lo ponemos, ejecutará el siguiente caso automáticamente.

Condiciones

- Finalmente, también tenemos el operador ternario `condición ? valorTrue : valorFalse`:

ej8: ternario.php

```
<?php
$hora = 14;
$formato = ($hora > 12) ? 24 : 12;
echo "El formato es de $formato horas"
?>
```

- Si queremos comprobar si una variable tiene valor y si no darle un valor determinado, usaremos el operador `?:` (se conoce como el operador Elvis https://en.wikipedia.org/wiki/Elvis_operator) con la sintaxis `expresión ? : valorSiVacio`:

ej8: elvis.php

```
<?php
$nombre = $_GET['nombre'] ?: "desconocido"
?>
```

Bucles

- Mediante la instrucción **while**:

ej9: while.php

```
<?php
$i = 1;
while ($i <= 10) {
    echo "Línea " . $i;
    echo "<br>";
    $i++;
}
?>
```

- Mediante la instrucción **do-while**:

ej9: do-while.php

```
<?php
do {
    $dado = rand(1, 6);
    // rand() devuelve un valor aleatorio
    echo "Jugando al parchís...";
    echo "Tirando el dado... ";
    echo "ha salido un " . $dado . ".";
    echo "<br>";
} while ($dado != 5);
echo "¡Bien! Saco una ficha de casa.";
?>
```

Bucles

- Mediante la instrucción **for**:
- Más adelante estudiaremos el **bucle foreach** para recorrer arrays.

ej9: for.php

```
<?php
// Bucle ascendente
for ($i = 1; $i <= 10; $i++) {
    echo "Línea " . $i;
    echo "<br>";
}
// Bucle descendente
for ($i = 10; $i >= 0; $i--) {
    echo "Línea " . $i;
    echo "<br>";
}
?>
```


Bucles

- PHP, del mismo modo que Java y C, permite romper los bucles mediante la instrucción `break`. A su vez, `continue` permite saltar a la siguiente iteración.

Si puedes, evita `break` y `continue`:

- Personalmente, no me gusta su uso. Prefiero el uso de variables flags/auxiliares para controlar la salida de los bucles. Por ejemplo:

ej9: flags.php

```
<?php
$salir = false;
for ($i = 1; $i <= 10 && !$salir; $i++) {
    if ($i === 5) {
        echo "Salgo cuando i=5";
        $salir = true;
    }
}
?>
```

Arrays

- Para almacenar datos compuestos, podemos utilizar tanto arrays sencillos como arrays asociativos (similares a un mapa). En realidad todos los arrays son mapas ordenados compuestos de pares clave-valor.
- **Cuidado con mezclar tipos:** Como el tipado es dinámico, nuestros arrays pueden contenedor datos de diferentes tipos. No se recomienda mezclar los tipos.
- Del mismo modo que Java, se definen mediante **corchetes**, son **0-index**, y se puede asignar un valor a un posición determinada
- Podemos obtener el tamaño del array mediante la función **count(array)**. Para recorrer el array haremos uso de un **bucle for**.
- Otra forma de recorrer los arrays, incluso más elegante, es hacer uso de **foreach**. Su sintaxis es **foreach (array as elemento)**

EJ10

Arrays asociativos

- Cada elemento es un par clave-valor. En vez de acceder por la posición, lo hacemos mediante una clave. Así pues, para cada clave se almacena un valor.
- A la hora de recorrer este tipo de arrays, mediante **foreach** separamos cada elemento en una pareja **clave => valor**:

ej10: arraysAsociativos.php

```
<?php
$capitales = ["Italia" => "Roma", "Francia" => "Paris", "Portugal" => "Lisboa"];
$capitalFrancia = $capitales["Francia"];
// se accede al elemento por la clave, no la posición
$capitales["Alemania"] = "Berlín"; // añadimos un elemento
echo "La capital de Francia es $capitalFrancia <br />";
echo "La capital de Francia es {$capitales["Francia"]} <br />";

$capitales[] = "Madrid";
// se añade con la clave 0 !!! ¡¡¡No asignar valores sin clave!!!

foreach ($capitales as $valor) {
    // si recorremos un array asociativo, mostraremos los valores
    echo "$valor <br />";
}
foreach ($capitales as $pais => $ciudad) {
    // separamos cada elemento en clave => valor
    echo "$pais : $ciudad <br />";
}
```

Arrays: operaciones

Las operaciones más importantes que podemos realizar con arrays son:

- `print_r($array)`: muestra el contenido de todo el `$array`. Si queremos mostrar el contenido con un formato determinado, hemos de recorrer el array con `foreach`.
- `var_dump($mixed)`: muestra el contenido del elemento recibido. Muestra más información que `print_r`.
- `$elem = array_pop($array)`: elimina el último \$elemento
- `array_push($array, $elem)`: añade un \$elemento al final
- `$booleano = in_array($elem, $array)`: averigua si \$elem está en el \$array
- `$claves = array_keys($array)`: devuelve las claves del \$array asociativo
- `$tam = count($array)`: devuelve el tamaño de \$array
- `sort($array)`: ordena los elementos del \$array
- `isset($array[elemento])`: indica si existe/tiene valor elemento dentro del array
- `unset($array[elemento])`: elimina el elemento del array (deja un hueco)
- Existen muchísimas más funciones para trabajar con arrays. Puedes consultar toda la información en la documentación oficial.
<https://www.php.net/manual/es/ref.array.php>



Al asignar un array a otro se realiza una copia. Cuidado con esta operación que puede consumir muchos recursos.

Arrays bidimensionales

- Consiste en un **array de arrays**, ya sean arrays secuenciales o asociativos. Puede haber **N dimensiones**.

ej10: arraysBidimensionales.php

```
<?php
$persona["nombre"] = "Bruce Wayne";
$persona["telefonos"] = ["966 123 456", "636 636 636"];
// array de arrays ordinarios
$persona["profesion"] =
["dia" => "filántropo", "noche" => "caballero oscuro"];
// array de arrays asociativos
echo $persona['nombre'] . " por la noche trabaja de " .
    $persona['profesion']['noche'];
```

Funciones

- Al no declararse los tipos de datos, los parámetros de las funciones no tienen tipo ni se indica el tipo de dato que devuelven. El paso de parámetros se realiza por valor, es decir, se realiza una copia de la variable.

```
<?php
function nombreFuncion($par1, $par2, ...) {
    // código
    return $valor;
}
$resultado = nombreFuncion($arg1, $arg2, ...);
?>
```

ej11: funciones.php

```
<?php
function diaSemana()
{
    $semana = ["lunes", "martes", "miércoles",
               "jueves", "viernes", "sábado", "domingo"];
    $dia = $semana[rand(0, 6)];
    return $dia;
}
$diaCine = diaSemana();
echo "El próximo $diaCine voy al cine.";
?>
```

Funciones: parámetros por referencia

- Si queremos pasar un parámetro por referencia, en la declaración de la función, indicaremos los parámetros mediante el operador `&` para indicar la dirección de memoria de la variable.

ej11: parametrosReferencia.php

```
<?php
function duplicarPorValor($argumento) {
    $argumento = $argumento * 2;
    echo "Dentro de la función: $argumento.<br>";
}
function duplicarPorReferencia(&$argumento) {
    $argumento = $argumento * 2;
    echo "Dentro de la función: $argumento.<br>";
}

$numero1 = 5;
echo "Antes de llamar: $numero1.<br>";
duplicarPorValor($numero1);
echo "Después de llamar: $numero1.<br>";
echo "<br>";

$numero2 = 7;
echo "Antes de llamar: $numero2.<br>";
duplicarPorReferencia($numero2);
echo "Después de llamar: $numero2.<br>";
?>
```

Antes de llamar: 5.
Dentro de la función: 10.
Después de llamar: 5.

Antes de llamar: 7.
Dentro de la función: 14.
Después de llamar: 14.

Funciones: parámetros por defecto/opcionales

- Permiten asignar valores en la declaración, y posteriormente, dejar el argumento en blanco.

ej11: parametrosOpcionales.php

```
?php
function obtenerCapital($pais = "todos")
{
    $capitales = array("Italia" => "Roma",
        "Francia" => "Paris",
        "Portugal" => "Lisboa");

    if ($pais == "todos") {
        return array_values($capitales);
    }
    else {
        return $capitales[$pais];
    }
}

print_r(obtenerCapital());
echo "<br/>";
echo obtenerCapital("Francia");
```

ej11: parametrosOpcionales.php

```
function saluda($nombre, $prefijo = "Sr") {
    echo "Hola ".$prefijo." ".$nombre;
}

saluda("Héctor", "Mr");
saluda("Héctor");
saluda("Olga", "Srta");
```


Funciones: parámetros variables

Podemos tener funciones donde en la declaración no indiquemos la cantidad de datos de entrada.

- `$arrayArgs = func_get_args();` → Obtiene un array con los parámetros
- `$cantidad = func_num_args();` → Obtiene la cantidad de parámetros recibidos
- `$valor = func_get_arg(numArgumento);` → Obtiene el parámetro que ocupa la posición numArgumento.
- Estas funciones no se pueden pasar como parámetro a otra función (como funciones variable, que veremos más adelante). Para ello, debemos guardar previamente la función en una variable.

ej11: parametrosVariables.php

```
<?php
function sumaParametros() {
    if (func_num_args() == 0) {
        return false;
    } else {
        $suma = 0;

        for ($i = 0; $i < func_num_args(); $i++) {
            $suma += func_get_arg($i);
        }

        return $suma;
    }
}

echo sumaParametros(1, 5, 9); // 15
?>
```

Funciones: parámetros variables

- Desde PHP 5.6, se puede utilizar el **operador ... (variadics)** el cual "disfraza" los parámetros como un array:

```
ej11: variadics.php

<?php
function sumaParametrosMejor(...$numeros) {
    if (count($numeros) == 0) {
        return false;
    } else {
        $suma = 0;

        foreach ($numeros as $num) {
            $suma += $num;
        }

        return $suma;
    }
}

echo sumaParametrosMejor(1, 5, 9); // 15
?>
```

```
ej11: variadics.php

<?php
function suma($a, $b) {
    return $a + $b;
}

echo suma(...[1, 5])."<br />";

$a = [1, 5];
echo suma(...$a);
?>
```

- También se puede utilizar para dividir un array en variables separadas para proporcionar argumentos.

Funciones: Argumentos con nombre

- Desde PHP 8.0 podemos pasar los argumentos con el nombre (además de por posición, como hemos hecho hasta ahora). Los argumentos con nombre se pasan poniendo el nombre como prefijo del parámetros separado por dos puntos:

`$resultado = funcion( arg1 : valor1, arg2 : valor2);`

- Esta característica complementa los parámetros opcionales permitiéndonos saltar su valor:
- Tanto los parámetros opcionales como los obligatorios pueden tener nombre, pero lo argumentos con nombre se tienen que poner después de los que no lo tienen.

ej11: argumentosNombre.php

```
<?php
function funcionArgumentosNombre($a, $b = 2, $c = 4) {
    echo "$a $b $c";
}
funcionArgumentosNombre(c: 3, a: 1); // "1 2 3"

funcionArgumentosNombre(1, c: 3); // "1 2 3"
```

Funciones tipadas

- Desde PHP7 en las funciones, tanto los parámetro como su devolución, permiten la definición de tipos. Esto se conoce como **strict_types (tipificación estricta)** y hay que definirlo en la primera línea de cada archivo .php para que el propio intérprete PHP compruebe los tipos y lance errores si los tipos son incorrectos, mediante la sentencia `<?php declare(strict_types=1);`
- Así pues, vamos a definir los tipos de los parámetros y de los valores devueltos mediante los tipos: int, float, string, bool, object y array.
- Si una función no devuelve nada se indica mediante el tipo void.

```
functionesTipadas.php

<?php
declare(strict_types=1);

function suma(int $a, int $b) : int {
    return $a + $b;
}

$num = 33;
echo suma(10, 30); //40
echo suma(10, $num); //43
echo suma("10", 30);
// error por tipificación estricta, sino daría 40
?>
```

Funciones variables

- Permite asignar una función a una variable.
- Nombre de la función entre comillas.
- Si una variable va seguida de paréntesis, PHP buscará una función con su valor.

ej11: funcionesVariables.php

```
<?php
$miFuncionSuma = "suma";
echo $miFuncionSuma(3,4); // invoca a la función suma
?>
```

Funciones anónimas

- PHP permite la definición y uso de funciones anónimas, es decir, funciones que no tienen nombre, y se utilizan principalmente para gestionar los callbacks. Este tipo de funciones se utiliza mucho en Javascript para gestionar los eventos y promesas.
- Desde PHP 7.4 se han introducido las funciones flecha (arrow functions) para simplificar su definición y uso.
- Tenéis más información sobre funciones anónimas y flecha en el siguiente artículo (en inglés): [Funciones anónimas y flecha en PHP](#)

Alcance

- Las variables definidas fuera de las funciones tienen alcance **global**: accesibles desde cualquier función. Los parámetros de una función y las variables declaradas dentro de una función (se conocen como variables locales) sólo son accesibles desde dentro de la misma función → alcance de **función**.
- En caso de conflicto, tienen prioridad las variables locales. Para evitar el conflicto, dentro de la función, podemos declarar la variable como global.

ej11: alcanceLocal.php

```
<?php
function miCiudad()
{
    $ciudad = "Elche";
    echo "Dentro de la función: $ciudad.<br>";
}

$ciudad = "Alicante";
echo "Antes de la función: $ciudad.<br>";
miCiudad();
echo "Después de la función: $ciudad.<br>"
?>
```

ej11: alcanceGlobal.php

```
<?php
function miCiudad()
{
    global $ciudad;
    $ciudad = "Elche";
    echo "Dentro de la función: $ciudad.<br>";
}

$ciudad = "Alicante";
echo "Antes de llamar: $ciudad.<br>";
miCiudad();
echo "Después de llamar: $ciudad.<br>"
?>
```

Por favor, hay que evitar el uso de variables globales dentro de las funciones. En el caso de necesitarlas, es mejor pasarlas como parámetro a las funciones.

Biblioteca de funciones

Podemos agrupar un conjunto de funciones en un archivo, para permitir su reutilización. Posteriormente, se incluye con:

- `include(archivo);` / `include_once(archivo);`
- `require(archivo);` / `require_once(archivo);`
- Si no encuentra el archivo, `require` lanza un error fatal, `include` lo ignora.
- Las funciones `_once` sólo se cargan una vez, si ya ha sido incluida previamente, no lo vuelve a hacer, evitando bucles.

ej12: biblioteca.php

```
<?php
function suma(int $a, int $b) : int {
    return $a + $b;
}

function resta(int $a, int $b) : int {
    return $a - $b;
}
?>
```

ej12: include.php

```
<?php
include_once("biblioteca.php");
echo suma(10,20);
echo resta(40,20);
?>
```

Plantillas mediante include

Mediante el uso de la instrucción **include** también podemos separar fragmentos de código PHP/HTML que queramos **reutilizar** en nuestros sitios web y crear un sistema muy sencillo de **plantillas**. Por ejemplo, vamos a separar una página en tres partes, primero la parte superior en `cabecera.php`.

La parte de abajo, por ejemplo, solo va a contener HTML y la colocamos en `pie.html`.

Y luego nos centramos únicamente en el contenido que cambia en `pagina.php`:

ej13: cabecera.php

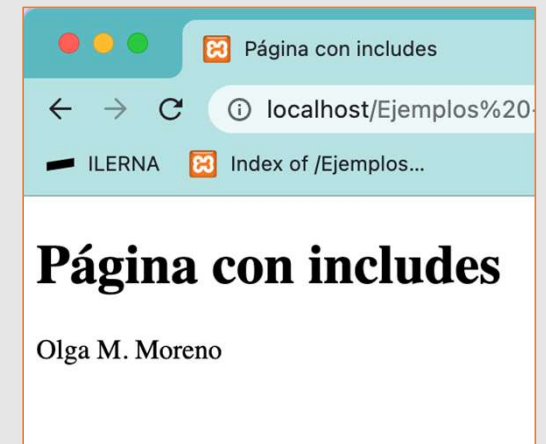
```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content=
"width=device-width, initial-scale=1.0">
<title><?= $titulo ?></title>
</head>
<body>
```

ej13: pagina.php

```
<?php
$titulo = "Página con includes";
include("cabecera.php");
?>
<h1><?= $titulo ?></h1>
<?php
include("pie.html");
?>
```

ej13: pie.html

```
<footer>Olga M. Moreno</footer>
</body>
</html>
```



Funciones predefinidas: cadenas

El lenguaje ofrece un abanico de funciones ya definidas, agrupadas por su funcionalidad:

<https://www.php.net/manual/es/funcref.php>

CADENAS

- Hemos visto que se pueden crear con comillas simples ('', sin interpretación) o comillas dobles ("", interpretan el contenido y las secuencias de escape \n, \t, \\$, {, ... - magic quotes)
- Se acceden a los caracteres como si fuera un array.
- Además de **echo**, podemos mostrar las cadenas mediante la función **printf**. Esta función viene heredada del lenguaje C, y en la cadena se indica el tipo de dato a formatear y genera una salida formateada. Si quiero guardar el resultado en una variable, podemos utilizar **sprintf**.

Tenéis muchos más ejemplos en:

https://www.w3schools.com/php/func_string_printf.asp

EJ14

```
<?php
$nombre= "Olga";
$moneda="moneda";
echo "Me llamo $nombre <br>";
echo "Son 30 ${moneda}s <br>";

$cadena = "Yo soy Batman";
$ygriega = $cadena[0];

echo $cadena;
echo "<br>";
echo $ygriega;
echo "<br>";

$num = 221;
$nombre = "Baker Street";
printf("Sherlok Holmes vivía en el número %db de %s", $num,
$nombre);
echo "<br>"; // %d -> número decimal, %s -> string
$frase = sprintf("Sherlok Holmes vivía en el número %db de
%s", $num, $nombre);
echo $frase;
```

Funciones predefinidas: cadenas

OPERACIONES BÁSICAS

- Todas las funciones se pueden consultar en <https://www.php.net/manual/es/ref.strings.php>

Las más importantes son:

- **strlen**: obtiene la longitud de una cadena y devuelve un número entero
- **substr**: devuelve una subcadena de la cadena original
- **str_replace**: reemplaza caracteres en una cadena
- **strtolower** y **strtoupper**: Transforman una cadena de caracteres en la misma cadena en minúsculas o mayúsculas respectivamente.

Si queremos trabajar con caracteres ASCII de forma individual, son útiles las funciones:

- **chr**: obtiene el carácter a partir de un ASCII
- **ord**: obtiene el ASCII de un carácter

Si queremos limpiar cadenas, tenemos las funciones:

- **trim**: elimina los espacios al principio y al final
- **ltrim** / **rtrim** o **chop**: Elimina los espacios iniciales / finales de una cadena.
- **str_pad**: rellena la cadenas hasta una longitud especificada y con el carácter o caracteres especificados.

EJ14

Funciones predefinidas: cadenas

COMPARANDO Y BUSCANDO

La comparación de cadenas puede ser con conversión de tipos mediante `==` o estricta con `===`. También funcionan los operadores `<` y `>` si ambas son cadenas. Al comparar cadenas con valores numéricos podemos utilizar:

- `strcmp`: 0 iguales, `<0` si `a<b` o `>0` si `a>b`
- `strcasecmp`: las pasa a minúsculas y compara
- `strncmp` / `strncasecmp`: compara los N primeros caracteres
- `strnatcmp`: comparaciones naturales

Si lo que queremos es buscar dentro de una cadena, tenemos:

- `strpos` / `strrpos`: busca en una cadena y devuelve la posición de la primera/última ocurrencia.
- `strstr` / `strchr` (*alias*): busca una cadena y devuelve la subcadena a partir de donde la ha encontrado
- `stristr`: ignora las mayúsculas

Funciones predefinidas: cadenas

Si queremos averiguar que contiene las cadenas, tenemos un conjunto de funciones de comprobaciones de tipo, se conocen como las funciones **ctype** que devuelven un booleano:

- **ctype_alpha** → letras
- **ctype_alnum** → alfanuméricos
- **ctype_digit** → dígitos
- **ctype_punct** → caracteres de puntuación, sin espacios
- **ctype_space** → son espacios, tabulador, salto de línea

ej14: operacionesBasicas.php

```
$prueba1 = "hola";  
$prueba2 = "hola33";  
$prueba3 = "33";  
$prueba4 = ",.()[]";  
$prueba5 = " ,.()[]";  
  
echo ctype_alpha($prueba1)."<br>"; // true  
echo ctype_alnum($prueba2)."<br>"; // true  
echo ctype_digit($prueba3)."<br>"; // true  
echo ctype_punct($prueba4)."<br>"; // true  
echo ctype_space($prueba5)."<br>"; // false  
echo ctype_space($prueba5[0])."<br>"; // true
```

Funciones predefinidas: cadenas

TRABAJANDO CON SUBCADENAS

Si queremos romper las cadenas en trozos, tenemos:

- **explode**: convierte en array la cadena mediante un separador.
- **implode** / **join**: pasa un array a cadena con un separador
- **str_split** / **chunk_split**: pasa una cadena a una array/cadena cada X caracteres

Si queremos trabajar con tokens:

- **strtok(cadena, separador)** y dentro del bucle: **strtok(separador)**

Finalmente, para separarla en base al formato:

- **sscanf**: al revés que sprintf, crea un array a partir de la cadena y el patrón.

Finalmente, otras operaciones que podemos realizar para trabajar con subcadenas son:

- **substr_count**: número de veces que aparece la subcadena dentro de la cadena
- **substr_replace**: reemplaza parte de la cadena a partir de su posición, y opcionalmente, longitud.

ej14: subcadenas

```
<?php
$frase = "Quien busca encuentra, eso dicen, a veces";
$partes = explode(",", $frase);

$ciudades = ["Elche", "Valencia", "Alicante"];
$cadenaCiudades = implode(">", $ciudades);

$partes3cadena = chunk_split($frase, 3);
// Qui
// en
// bus
// ca
// ...
$partes3array = str_split($frase, 3);
// ["Qui", "en ", "bus", "ca ", "enc", ...]
?>
```

ej14: subcadenas.php

```
$batman = "Bruce Wayne es Batman";
$empresa = substr($batman, 6, 5); // Wayne
$bes = substr_count($batman, "B"); // 2
// Bruce Wayne es camarero
$camarero1 = substr_replace($batman, "camarero", 15);
$camarero2 = substr_replace($batman, "camarero", -6);
// quita 6 desde el final
// Bruno es Batman
$bruno = substr_replace($batman, "Bruno", 0, 11);
```

Funciones predefinidas: cadenas

También disponemos de una serie de funciones que facilitan las codificaciones desde y hacia HTML:

- **htmlentities**: convierte a entidades HTML, por ejemplo, á por ´, ñ por ñ, < por <, etc..
- **htmlspecialchars**: ídem, pero solo con los caracteres especiales (&, ", ', <, >, ...)
- **striptags**: elimina etiquetas HTML.
- **nl2br**: cambia saltos de línea por
.
- **rawurlencode** / **rawurldecode**: codifica/decodifica una URL (espacios, ...).

Funciones predefinidas: matemáticas

Disponemos tanto de constantes como funciones ya definidas para trabajar con operaciones matemáticas: <https://www.php.net/manual/es/ref.math.php>

- Constantes ya definidas:

`M_PI`, `M_E`, `M_EULER`, `M_LN2`, `M_LOG2E`, `PHP_INT_MAX`, `PHP_FLOAT_MAX`

- Funciones de cálculo:

`pow`, `sqrt`, `log`, `decbin`, `bindec`, `decoct`, `dechex`, `base_convert`, `max`, `min`

- Funciones trigonométricas:

`sin`, `cos`, `tan`, `deg2rad`, `rad2deg`

- Funciones para trabajar con números aleatorios:

`rand`, `mt_rand` (más rápida)

Aunque la mayoría de ellas son muy específicas de problemas matemáticos / estadísticos, es muy común que tengamos que redondear y/o formatear los cálculos antes de mostrarlos al usuario.

Funciones predefinidas: matemáticas

- Mediante la función `number_format(numero, cantidadDecimales, separadorDecimales, separadorMiles)` podremos pasar números a cadena con decimales y/o separadores de decimales y/o de miles.
- Para redondear, tenemos `abs` para el valor absoluto y `round` para redondear, `ceil` para aproximación por exceso y `floor` por defecto.

```
ej15: matematicas.php

<?php
$nf = 1234.5678;
echo number_format($nf, 2); // 1,234.57
echo number_format($nf, 2, "M", "#"); // 1#234M57

$num = 7.7;
$siete = floor($num);
$ocho = ceil($num);

$otro = 4.49;
$cuatro = round($otro);
$cuatrocinco = round($otro, 1);
$cinco = round($cuatrocinco);
?>
```


Funciones predefinidas: tipos de datos

Finalmente, para realizar conversiones de datos o si queremos trabajar con tipos de datos, tenemos las siguientes funciones:

- **floatval**, **intval**, **strval**: devuelve una variable del tipo de la función indicada
- **settype**: fuerza la conversión
- **gettype**: obtiene el tipo
- **is_int**, **is_float**, **is_string**, **is_array**, **is_object**, **is_nan**: devuelve un booleano a partir del tipo recibido

ej16: tiposDeDatos.php

```
<?php
$uno = 1;
var_dump(is_int($uno)); // true
$unofloat = floatval($uno);
settype($uno, "string");
var_dump(is_int($uno)); // false
var_dump(is_string($uno)); // true
settype($uno, "float");
var_dump(is_int($uno)); // false
var_dump(is_float($uno)); // true
var_dump(is_int(intval($uno))); // true
?>
```

Bibliografía

- Páginas web que han ayudado en la construcción de estas diapositivas:
- <https://github.com/olga3emes/PHP-Web/wiki>
- <http://phpdevenezuela.github.io/php-the-right-way/>
- <https://www.mclibre.org/consultar/php/>
- Ediciones ENI: PHP (Acceso restringido)

Programación Orientada a Objetos

Índice

1. Clases y Objetos
2. Encapsulación
3. Recibiendo y enviando objetos
4. Constructores
5. Clases estáticas
6. Introspección
7. Herencia
8. Clases Abstractas
9. Clases Finales
10. Interfaces
11. Métodos encadenados
12. Métodos mágicos
13. Espacios de nombres
14. Autoload
15. Gestión de errores
16. Excepciones

Clases y objetos

PHP sigue un paradigma de programación orientada a objetos (POO) basada en clases.

Un clase es un plantilla que define las propiedades y métodos para poder crear objetos. De este manera, un objeto es una instancia de una clase.

Tanto las propiedades como los métodos se definen con una visibilidad (quien puede acceder):

- Privado - **private**: Sólo puede acceder la propia clase.
- Protegido - **protected**: Sólo puede acceder la propia clase o sus descendientes.
- Público - **public**: Puede acceder cualquier otra clase.

Para declarar una clase, se utiliza la palabra clave **class** seguido del nombre de la clase. Para instanciar un objeto a partir de la clase, se utiliza **new**:



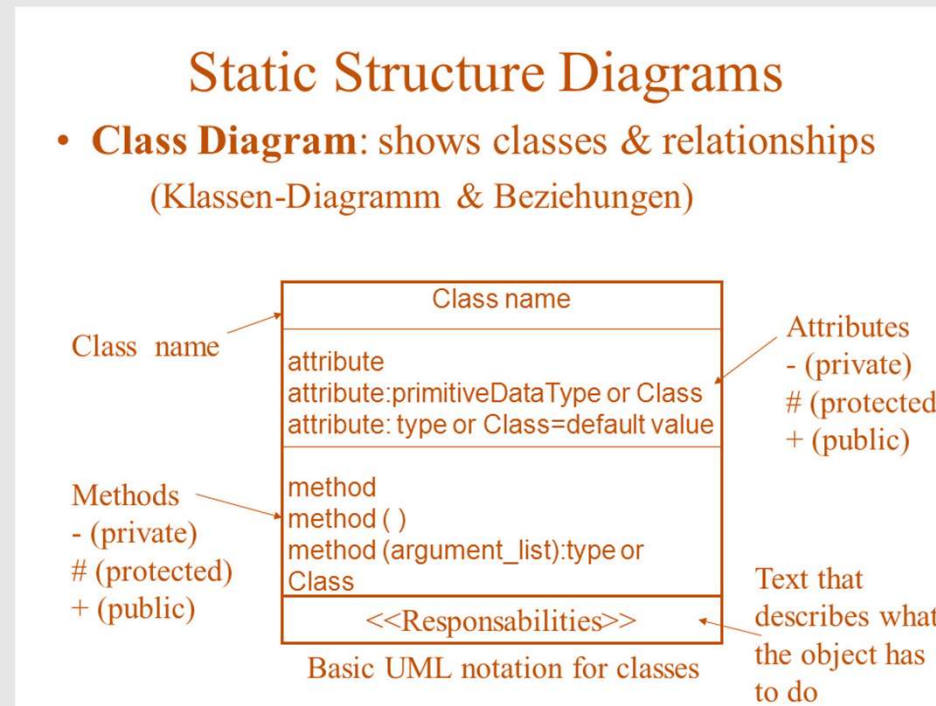
Clases con mayúscula
Todas las clases empiezan por letra mayúscula.

ej1

```
<?php
class NombreClase {
    // propiedad
    $/ y
    métodos
    $ob = new NombreClase();
```

UML

Cuando un proyecto crece, es normal modelar las clases mediante UML. Las clases se representan mediante un cuadrado, separando el nombre, de las propiedades y los métodos:



Clases y Objetos

- Una vez que hemos creado un objeto, se utiliza el operador `->` para acceder a una propiedad o un método:

```
$objeto->propiedad;  
$objeto->método(parámetros);
```

- Si desde dentro de la clase, queremos acceder a una propiedad o método de la misma clase, utilizaremos la referencia `$this`:

```
$this->propiedad;  
$this->método(parámetros);
```

- Como ejemplo, codificaríamos una persona en el fichero `Persona.php`:
- Aunque se pueden declarar varias clases en el mismo archivo,
- es una mala práctica. Así pues, cada fichero contendrá
- una sola clase, y se nombrará con el nombre de la clase.

ej2: Persona.php

```
<?php  
class Persona{  
  
    private String $nombre;  
  
    public function setNombre(String $nom){  
        $this -> nombre=$nom;  
    }  
  
    public function imprimir(){  
        echo $this->nombre;  
        echo "    ";  
    }  
}  
  
$hector = new Persona();  
$hector->setNombre("Héctor");  
$hector->imprimir();
```

Encapsulación

Las propiedades se definen privadas o protegidas (si queremos que las clases heredadas puedan acceder).

Para cada propiedad, se añaden métodos públicos (**getter/setter**):

```
public setPropiedad(tipo $param)
public getPropiedad() : tipo
```

Las constantes se definen públicas para que sean accesibles por todos los recursos.

ej3:

```
<?php
class MayorMenor{
    private int $mayor;
    private int $menor;
    public function getMayor(){
        return $this->mayor;
    }
    public function setMayor($mayor){
        $this->mayor = $mayor;
        return $this;
    }
    public function getMenor(){
        return $this->menor;
    }
    public function setMenor($menor){
        $this->menor = $menor;
        return $this;
    }
}
```


Recibiendo y Enviando objetos

Es recomendable indicarlo en el tipo de parámetros. Si el objeto puede devolver nulos se pone `?` delante del nombre de la clase.



Objetos por referencia

Los objetos que se envían y reciben como parámetros siempre se pasan por referencia.

ej4.php

```
<?php
include_once("../ej3/MayorMenor.php");
function mayormenor(array $nums) : ?MayorMenor{
    $a = max($nums);
    $b = min($nums);

    $res = new MayorMenor();
    $res->setMayor($a);
    $res->setMenor($b);
    return $res;
}

$resultado= mayormenor([2,58,6,389,35,39,27,42,22]);
echo "<br> Mayor: ". $resultado->getMayor();
echo "<br> Menor: ". $resultado->getMenor();
```

Constructor

El constructor de los objetos se define mediante el método mágico `__construct`. Puede o no tener parámetros, pero sólo puede haber un constructor.

```
ej5: Persona.php

<?php
class Persona{

    private String $nombre;

    public function __construct(String $nom)
    {
        $this->nombre= $nom;
    }

    public function imprimir(){
        echo $this->nombre;
        echo " ";
    }
}

$hector = new Persona("Héctor");
$hector->imprimir();
```

Constructores en PHP 8

Una de las grandes novedades que ofrece PHP 8 es la **simplificación de los constructores** con parámetros, lo que se conoce como promoción de las propiedades del constructor.

Para ello, en vez de tener que declarar las propiedades como privadas o protegidas, y luego dentro del constructor tener que asignar los parámetros a estas propiedades, el propio constructor promociona las propiedades.

Veámoslo mejor con un ejemplo. Imaginemos una clase Punto donde queremos almacenar sus coordenadas:

En PHP 8, quedaría del siguiente modo (mucho más corto, lo que facilita su legibilidad):

ej6:

```
class Punto {  
  
    protected float $x;  
    protected float $y;  
    protected float $z;  
  
    public function __construct(  
        float $x = 0.0,  
        float $y = 0.0,  
        float $z = 0.0  
    ) {  
        $this->x = $x;  
        $this->y = $y;  
        $this->z = $z;  
    }  
}
```

ej6:

```
class Punto2{  
    public function __construct(  
        protected float $x,  
        protected float $y,  
        protected float $z)  
    {}  
}
```

El orden importa

A la hora de codificar el orden de los elementos debe ser:

```
<?php
declare(strict_types=1);

class NombreClase {
    // propiedades

    // constructor

    // getters - setters

    // resto de métodos
}
?>
```

strict_types=1: desde PHP 7 lo usamos para controlar que la inicialización de las variables coincida siempre con el tipo de dato que le insertamos, si no fuese así obtendríamos un **TypeError**.

Clases estáticas

Son aquellas que tienen propiedades y/o métodos estáticos (también se conocen como de clase, porque su valor se comparte entre todas las instancias de la misma clase).

- Se declaran con `static` y se referencian con `::`.
- Si queremos acceder a un método estático, se antepone el nombre de la clase: `Producto::nuevoProducto()`.
- Si desde un método queremos acceder a una propiedad estática de la misma clase, se utiliza la referencia `self`: `self::$numProductos`

Se le llama **Paamayim Nekudotayim**, operador de resolución de ámbito o doble dos puntos "`::`" (**double colon**) al operador que permite acceder a **constantes** y a **métodos estáticos**, además de poder **sobrescribir propiedades o métodos de una clase**.

ej7:

```
<?php
class Producto {
    const IVA = 0.23;
    private static $numProductos = 0;

    public static function nuevoProducto() {
        self::$numProductos++;
    }
}
Producto::nuevoProducto();
$impuesto = Producto::IVA;
```

Clases estáticas

También podemos tener clases normales que tengan alguna propiedad estática:

ej7: Producto2.php

```
<?php
class Producto2{
    const IVA = 0.21;
    private static $numProductos =0;
    private $codigo;
    public function __construct(String $cod){

        self::$numProductos++;
        $this->codigo=$cod;
    }
    public function mostrarResumen() : String {
        return "El producto ". $this->codigo." es el número " . self::$numProductos;
    }
} // End class

$prod1 = new Producto2("PlayStation 5");
$prod2 = new Producto2("Nintengo Switch");
$prod3 = new Producto2("XB0X Series X");
$prod4 = new Producto2("Steam Deck");
echo $prod2->mostrarResumen();
```

Introspección

Al trabajar con clases y objetos, existen un conjunto de funciones ya definidas por el lenguaje que permiten obtener información sobre los objetos:

- **instanceof**: permite comprobar si un objeto es de una determinada clase
- **get_class**: devuelve el nombre de la clase
- **get_declared_class**: devuelve un array con los nombres de las clases definidas
- **class_alias**: crea un alias
- **class_exists** / **method_exists** / **property_exists**: true si la clase / método / propiedad está definida
- **get_class_methods** / **get_class_vars** / **get_object_vars**: Devuelve un array con los nombres de los métodos / propiedades de una clase / propiedades de un objeto que son accesibles desde dónde se hace la llamada. (Ver Ejemplo 8)

Clonado

Al asignar dos objetos no se copian, se crea una nueva referencia. Si queremos una copia, hay que clonarlo mediante el método `clone(object) : object`. Si queremos modificar el clonado por defecto, hay que definir el método mágico `__clone()` que se llamará después de copiar todas las propiedades. Más información en <https://www.php.net/manual/es/language.oop5.cloning.php>

Herencia

- PHP soporta herencia simple, de manera que una clase solo puede heredar de otra, no de dos clases a la vez. Para ello se utiliza la palabra clave `extends`. Si queremos que la clase A hereda de la clase B haremos: `class A extends B`
- El hijo hereda los atributos y métodos públicos y protegidos.

Cada clase en un archivo

Como ya hemos comentado, deberíamos colocar cada clase en un archivo diferente para posteriormente utilizarlo mediante `include`. En los siguientes ejemplos los hemos colocado juntos para facilitar su legibilidad.

- Por ejemplo, tenemos una clase `Item` y una `Tv` que hereda de `Item` (ver ejemplo 9)

ej9: herencia.php

```
<?php
class Item {
    public $codigo;
    public $nombre;
    public $nombreCorto;
    public $PVP;

    public function mostrarResumen() {
        echo "<p>Prod:". $this->codigo. "</p>";
    }
}

class Tv extends Item {
    public $pulgadas;
    public $tecnologia;
}
```


Herencia

Podemos utilizar las siguientes funciones para averiguar si hay relación entre dos clases:

- `get_parent_class(object)`: string
- `is_subclass_of(object, string)`: bool

ej9: herencia.php

```
$t = new Tv();
$t->codigo = 33;
if ($t instanceof Item) {
    echo $t->mostrarResumen();
}

$padre = get_parent_class($t);
echo "<br>La clase padre es: " . $padre;
$objetoPadre = new $padre;
echo $objetoPadre->mostrarResumen();

if (is_subclass_of($t, 'Item')) {
    echo "<br>Soy un hijo de Item";
}
```

Herencia

SOBRESCRIBIR MÉTODOS

- Podemos crear métodos en los hijos con el mismo nombre que el padre, cambiando su comportamiento. Para invocar a los métodos del padre -> `parent::nombreMetodo()`

ej9: herencia.php

```
class Tv extends Item {  
    public $pulgadas;  
    public $tecnologia;  
  
    /* SOBRESCRIBIR UN MÉTODO DEL PADRE:*/  
  
    public function mostrarResumen() {  
        parent::mostrarResumen();  
        echo "<p>TV ".$this->tecnologia." de ".$this->pulgadas."</p>";  
    }  
}
```

Herencia

CONSTRUCTOR EN HIJOS

- En los hijos no se crea ningún constructor de manera automática. Por lo que si no lo hay, se invoca automáticamente al del padre. En cambio, si lo definimos en el hijo, hemos de invocar al del padre de manera explícita.

```
antiguo.php

<?php
class Producto {
    public string $codigo;

    public function __construct(string $codigo) {
        $this->codigo = $codigo;
    }

    public function mostrarResumen() {
        echo "<p>Prod:". $this->codigo."</p>";
    }
}

class Tv extends Producto {
    public $pulgadas;
    public $tecnologia;

    public function __construct(string $codigo, int $pulgadas, string $tecnologia) {
        parent::__construct($codigo);
        $this->pulgadas = $pulgadas;
        $this->tecnologia = $tecnologia;
    }

    public function mostrarResumen() {
        parent::mostrarResumen();
        echo "<p>TV ". $this->tecnologia." de ". $this->pulgadas."</p>";
    }
}
```

```
ej10: nuevo.php

<?php
class Prod{
    public function __construct(private string $codigo) { }
    public function mostrarResumen() {
        echo "<p>Prod:". $this->codigo."</p>";
    }
}

class Tele extends Prod{
    public function __construct(
        String $codigo,
        private int $pulgadas,
        private String $tecnologia){
        parent::__construct($codigo);
    }

    public function mostrarResumen() {
        parent::mostrarResumen();
        echo "<p>TV ". $this->tecnologia." de ". $this->pulgadas."</p>";
    }
}
```

Clases abstractas

- Las clases abstractas obligan a heredar de una clase, ya que no se permite su instanciación. Se define mediante **abstract** class NombreClase {.
- Una clase abstracta puede contener propiedades y métodos no-abstractos, y/o métodos abstractos.
- Cuando una clase hereda de una clase abstracta, obligatoriamente debe implementar los métodos que tiene el padre marcados como abstractos.

ej11: CocheAbstract.php

```
<?php
abstract class CocheAbstract{
    public function getRuedas(){
        return 4;
    }
    abstract public function setPotencia($potencia);
    abstract public function getPotencia();
}
```

ej11: Audi.php

```
class Audi extends CocheAbstract{
    public $brand = 'Audi';
    protected $potencia;
    public function setPotencia($potencia){
        $this->potencia = $potencia;
    }
    public function getPotencia(){
        return $this->potencia;
    }
}
```

Clases finales

- Son clases opuestas a abstractas, ya que evitan que se pueda heredar una clase o método para sobrescribirlo.

ej12: Electrodomestico.php

```
<?php
class Electrodomestico{
    private $codigo;
    public function getCodigo(){
        return $this->codigo;
    }
    final public function mostrarResumen(): String {
        return "Producto ".$this->codigo;
    }
}
```

ej12: Microondas.php

```
<?php
include_once("Electrodomestico.php");
final class Microondas extends Electrodomestico{
    private $potencia;
    public function getPotencia() : int {
        return $this->potencia;
    }
    // No podemos implementar mostrarResumen()
}
// No podremos heredar de Microondas
```

Interfaces

- Permite definir un contrato con las firmas de los métodos a cumplir. Así pues, sólo contiene declaraciones de funciones y todas deben ser públicas.
- Se declaran con la palabra clave **interface** y luego las clases que cumplan el contrato lo realizan mediante la palabra clave **implements**.
- Se permite la herencia de interfaces. Además, una clase puede implementar varias interfaces (en este caso, sí soporta la herencia múltiple, pero sólo de interfaces).

ej13: interfaces.php

```
<?php
interface Mostrable{
    public function mostrarResumen(): string;
}
interface MostrableTodo extends Mostrable{
    public function mostrarTodo(): string;
}
interface Facturable{
    public function generarFactura(): string;
}

class Producto3 implements MostrableTodo, Facturable{
    // Implementaciones de los métodos

    // Obligatoriamente deberá implementar public function mostrarResumen, mostrarTodo y generarFactura

    public function mostrarResumen(): string{
        return "resumen";
    }
    public function mostrarTodo(): string{
        return "mostrar";
    }
    public function generarFactura(): string{
        return "factura";
    }
}
```

Métodos encadenados

- Sigue el planteamiento de la programación funcional, y también se conoce como *method chaining*.
- Plantea que sobre un objeto se realizan varias llamadas.
- Para facilitarlo, vamos a modificar todos sus métodos mutadores (que modifican datos, setters, ...) para que devuelvan una referencia a \$this:

ej14: chaining.php

```
<?php
include_once("Libro.php");
$p1 = new Libro();
$p1->setNombre("Harry Potter");
$p1->setAutor("JK Rowling");
echo $p1;
echo "<br>";
// Method chaining
$p2 = new Libro();
$p2->setNombre("La Bestia")->setAutor("Carmen Mola");
echo $p2;
```

ej14: Libro.php

```
<?php
class Libro {
    private $nombre;
    private $autor;
    public function getNombre() : string{
        return $this->nombre;
    }
    public function setNombre($nombre){
        $this->nombre = $nombre;
        return $this;
    }
    public function getAutor() : string{
        return $this->autor;
    }
    public function setAutor($autor){
        $this->autor = $autor;
        return $this;
    }
    public function __toString(): string{
        return $this->nombre." de ".$this->autor;
    }
}
```

Métodos mágicos

Todas las clases PHP ofrecen un conjunto de métodos, también conocidos como *magic methods* que se pueden sobrescribir para sustituir su comportamiento. Algunos de ellos ya los hemos utilizado. Ante cualquier duda, es conveniente consultar la documentación oficial.

Los más destacables son:

- `__construct()` → constructor
- `__destruct()` → se invoca al perder la referencia. Se utiliza para cerrar una conexión a la BD, cerrar un fichero, ...
- `__toString()` → representación del objeto como cadena. Es decir, cuando hacemos `echo $objeto` se ejecuta automáticamente este método.
- `__get(propiedad)`, `__set(propiedad, valor)` → Permitiría acceder a las propiedad privadas, aunque siempre es más legible/mantenible codificar los getter/setter.
- `__isset(propiedad)`, `__unset(propiedad)` → Permite averiguar o quitar el valor a una propiedad.
- `__sleep()`, `__wakeup()` → Se ejecutan al recuperar (`unserialize()`) o almacenar un objeto que se serializa (`serialize()`), y se utilizan para permite definir qué propiedades se serializan.
- `__call()`, `__callStatic()` → Se ejecutan al llamar a un método que no es público. Permiten sobrecargar métodos.

Espacios de nombres

Desde PHP 5.3 y también conocidos como Namespaces, permiten organizar las clases/interfaces, funciones y/o constantes de forma similar a los paquetes en Java.

- Se declaran en la primera línea mediante la palabra clave namespace seguida del nombre del espacio de nombres asignado (cada subnivel se separa con la barra invertida \):
- Por ejemplo, para colocar la clase Producto dentro del namespace Dwes\Ejemplos lo haríamos así:

```
<?php
namespace Dwes\Ejemplos;

const IVA = 0.21;

class Producto {
    public $nombre;

    public function muestra() : void {
        echo"<p>Prod:" . $this->nombre . "</p>";
    }
}
```

Recomendación:

Un sólo namespace por archivo y crear una estructura de carpetas respetando los niveles/subniveles (igual que se hace en Java)

Espacios de nombres

Acceso:

- Para referenciar a un recurso que contiene un namespace, primero hemos de tenerlo disponible haciendo uso de include o require. Si el recurso está en el mismo namespace, se realiza un acceso directo (se conoce como acceso sin cualificar).

Realmente hay tres tipos de acceso:

- sin cualificar: recurso
- cualificado: rutaRelativa\recurso → no hace falta poner el namespace completo.
- totalmente cualificado: \rutaAbsoluta\recurso

```
<?php
namespace Dwes\Ejemplos;

include_once("Producto.php");

echo IVA; // sin cualificar
echo Utilidades\IVA;
// acceso cualificado. Daría error, no existe \Dwes\Ejemplos\Utilidades\IVA
echo \Dwes\Ejemplos\IVA; // totalmente cualificado

$p1 = new Producto();
// lo busca en el mismo namespace y encuentra \Dwes\Ejemplos\Producto
$p2 = new Model\Producto();
// daría error, no existe el namespace Model. Está buscando \Dwes\Ejemplos\Model\Producto

$p3 = new \Dwes\Ejemplos\Producto(); // \Dwes\Ejemplos\Producto
```

Espacios de nombres

Para evitar la referencia cualificada podemos declarar el uso mediante use (similar a hacer import en Java). Se hace en la cabecera, tras el namespace:

Los tipos posibles son:

- use const nombreCualificadoConstante
- use function nombreCualificadoFuncion
- use nombreCualificadoClase
- use nombreCualificadoClase as NuevoNombre // para renombrar elementos

Por ejemplo, si queremos utilizar la clase `\Dwes\Ejemplos\Producto` desde un recurso que se encuentra en la raíz, haríamos:

```
<?php
include_once("Dwes\Ejemplo\Producto.php");

use const Dwes\Ejemplos\IVA;
use \Dwes\Ejemplos\Producto;

echo IVA;
$p1 = new Producto();
```

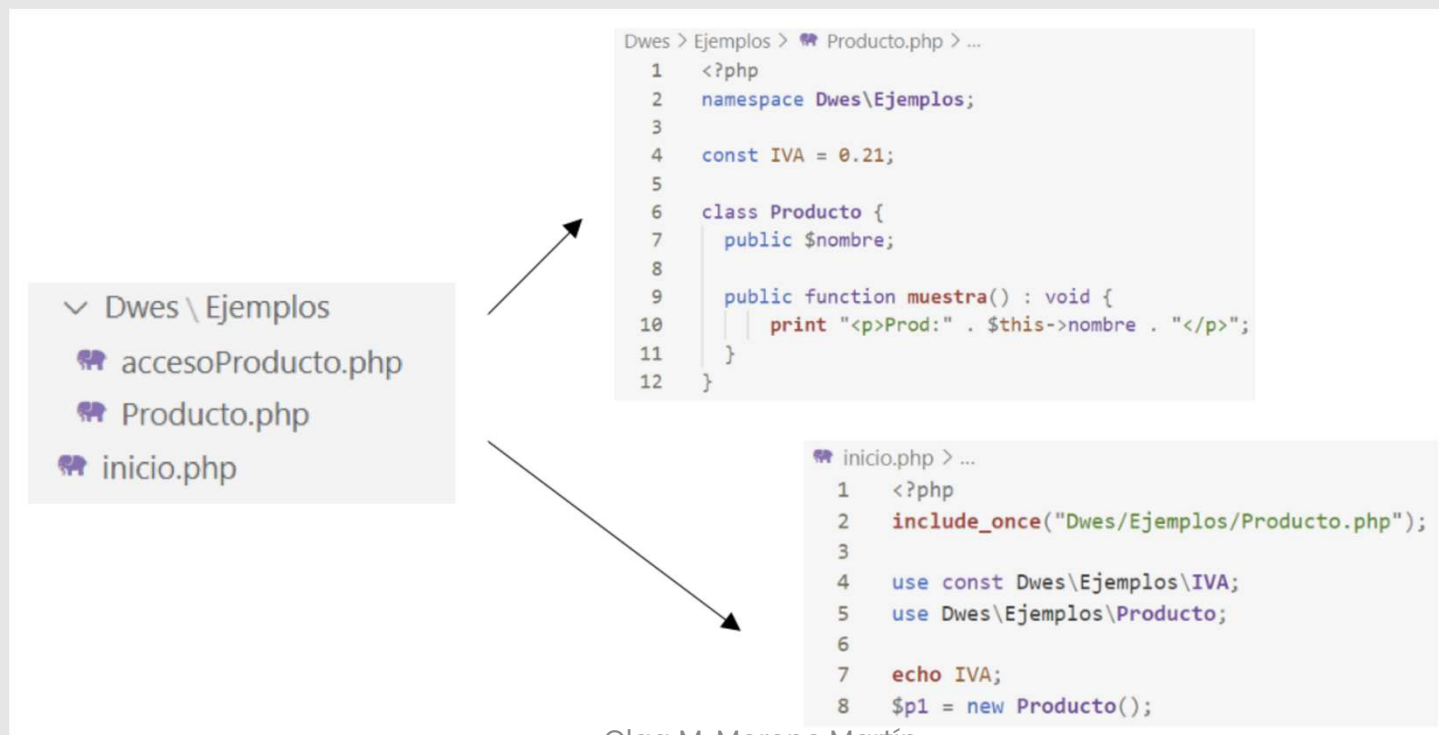
To use or not to use

En resumen, use permite acceder sin cualificar a recursos que están en otro *namespace*. Si estamos en el mismo espacio de nombre, no necesitamos use.

Espacios de nombres

Organización:

- Todo proyecto, conforme crece, necesita organizar su código fuente. Se plantea una organización en la que los archivos que interactúan con el navegador se colocan en el raíz, y las clases que definamos van dentro de un namespace (y dentro de su propia carpeta src o app).



Espacios de nombres

Organización, includes y usos:

- Colocaremos cada recurso en un fichero aparte.
- En la primera línea indicaremos su namespace (si no está en el raíz).
- Si utilizamos otros recursos, haremos un `include_once` de esos recursos (clases, interfaces, etc...).
- Cada recurso debe incluir todos los otros recursos que reference: la clase de la que hereda, interfaces que implementa, clases utilizadas/recibidas como parámetros, etc...
- Si los recursos están en un espacio de nombres diferente al que estamos, emplearemos `use` con la ruta completa para luego utilizar referencias sin cualificar.

Autoload

- ¿No es tedioso tener que hacer el include de las clases? El autoload viene al rescate.
- Así pues, permite cargar las clases (no las constantes ni las funciones) que se van a utilizar y evitar tener que hacer el include_once de cada una de ellas. Para ello, se utiliza la función `spl_autoload_register`

```
<?php
spl_autoload_register( function( $nombreClase ) {
    include_once $nombreClase.'.php';
} );
?>
```

Por qué se llaman *autoload*?

Porque antes se realizaba mediante el método mágico `__autoload()`, el cual está *deprecated* desde PHP 7.2

Autoload

Y ¿cómo organizamos ahora nuestro código aprovechando el autoload?

- Para facilitar la búsqueda de los recursos a incluir, es recomendable colocar todas las clases dentro de una misma carpeta. Nosotros la vamos a colocar dentro de `app` (más adelante, cuando estudiemos Laravel veremos el motivo de esta decisión). Otras carpetas que podemos crear son `test` para colocar las pruebas PHPUnit que luego realizaremos, o la carpeta `vendor` donde se almacenarán las librerías del proyecto (esta carpeta es un estándar dentro de PHP, ya que Composer la crea automáticamente, ya lo veremos más adelante).
- Como hemos colocado todos nuestros recursos dentro de `app`, ahora nuestro `autoload.php` (el cual colocamos en la carpeta raíz) sólo va a buscar dentro de esa carpeta:

```
<?php
spl_autoload_register( function( $nombreClase ) {
    include_once "app/".$nombreClase.'.php';
} );
?>
```

Autoload y rutas erróneas

En *Ubuntu* al hacer el *include* de la clase que recibe como parámetro, las barras de los namespace (`\`) son diferentes a las de las rutas (`/`). Por ello, es mejor que utilicemos el fichero `autoload`:

```
<?php
spl_autoload_register( function( $nombreClase ) {
    $ruta = "app\\".$nombreClase.'.php';
    $ruta = str_replace("\\", "/", $ruta); // Sustituimos las barras
    include_once $ruta;
} );
?>
```

Gestión de errores

- PHP clasifica los errores que ocurren en diferentes niveles. Cada nivel se identifica con una constante. Por ejemplo:
- **E_ERROR**: errores fatales, no recuperables. Se interrumpe el script.
- **E_WARNING**: advertencias en tiempo de ejecución. El script no se interrumpe.
- **E_NOTICE**: avisos en tiempo de ejecución.
- Podéis comprobar el listado completo de constantes de <https://www.php.net/manual/es/errorfunc.constants.php>

Gestión de errores

Para la configuración de los errores podemos hacerlo de dos formas:

A nivel de php.ini:

- **error_reporting**: indica los niveles de errores a notificar
- **error_reporting** = E_ALL & ~E_NOTICE -> Todos los errores menos los avisos en tiempo de ejecución.
- **display_errors**: indica si mostrar o no los errores por pantalla. En entornos de producción es común ponerlo a off

Mediante código con las siguientes funciones:

- **error_reporting(codigo)** -> Controla qué errores notificar
- **set_error_handler(nombreManejador)** -> Indica que función se invocará cada vez que se encuentre un error. El manejador recibe como parámetros el nivel del error y el mensaje
- A continuación tenemos un ejemplo mediante código:

ej16: errores.php

```
<?php
error_reporting(E_ALL & ~E_NOTICE & ~E_WARNING);
$resultado = $dividendo / $divisor;

error_reporting(E_ALL & ~E_NOTICE);
set_error_handler("miManejadorErrores");
$resultado = $dividendo / $divisor;
restore_error_handler(); // vuelve al anterior

function miManejadorErrores($nivel, $mensaje) {
    switch($nivel) {
        case E_WARNING:
            echo "<strong>Warning</strong>: $mensaje.<br/>";
            break;
        default:
            echo "Error de tipo no especificado: $mensaje.<br/>";
    }
}
```

Excepciones

- La gestión de excepciones forma parte desde PHP 5. Su funcionamiento es similar a *Java*, haciendo uso de un **bloque try / catch / finally**. Si detectamos una situación anómala y queremos lanzar una excepción, deberemos realizar `throw new Exception` (adjuntando el mensaje que lo ha provocado).

ej17: excepciones.php

```
<?php
try {
    if ($divisor == 0) {
        throw new Exception("División por cero.");
    }
    $resultado = $dividendo / $divisor;
} catch (Exception $e) {
    echo "Se ha producido el siguiente error: ".$e->getMessage();
}
```

- La clase `Exception` es la clase padre de todas las excepciones. Su constructor recibe `mensaje[,codigoError][,excepcionPrevia]`.
- A partir de un objeto `Exception`, podemos acceder a los métodos `getMessage()` y `getCode()` para obtener el mensaje y el código de error de la excepción capturada.
- El propio lenguaje ofrece un conjunto de excepciones ya definidas, las cuales podemos capturar (y lanzar desde PHP 7). Se recomienda su consulta en la documentación oficial.

Creando excepciones

- Para crear una excepción, la forma más corta es crear una clase que únicamente herede de Exception.
- Si queremos, y es recomendable dependiendo de los requisitos, podemos sobrecargar los métodos mágicos, por ejemplo, sobrecargando el constructor y llamando al constructor del padre, o rescribir el método `__toString` para cambiar su mensaje:

ej17: MiExcepcion.php

```
<?php
class MiExcepcion extends Exception {
    public function __construct($msj, $codigo = 0, Exception $previa = null
    ) {
        // código propio
        parent::__construct($msj, $codigo, $previa);
    }
    public function __toString() {
        return __CLASS__ . ": [{$this->code}]: {$this->message}\n";
    }
    public function miFuncion() {
        echo "Una función personalizada para este tipo de excepción\n";
    }
}
```

Excepciones

- Si definimos una excepción de aplicación dentro de un namespace, cuando referenciamos a Exception, deberemos referenciarla mediante su nombre totalmente cualificado (\Exception), o utilizando use:

ej17: referenciando.php

```
<?php
namespace \Dwes\Ejemplos;
class AppExcepcion extends \Exception {}
?>
```

ej17: referenciando.php

```
<?php
namespace \Dwes\Ejemplos;

use Exception;

class AppExcepcion extends Exception {}

?>
```

Excepciones múltiples

- Se pueden usar excepciones múltiples para comprobar diferentes condiciones. A la hora de capturarlas, se hace de más específica a más general.
- Si en el mismo catch queremos capturar varias excepciones, hemos de utilizar el operador |:

ej17: excepcionesMultiples.php

```
<?php
class MainException extends Exception {}
class SubException extends MainException {}

try {
    throw new SubException("Lanzada SubException");
} catch (MainException | SubException $e) {
    echo "Capturada Exception " . $e->getMessage();
}
```

ej17: excepcionesMultiples.php

```
<?php
$email = "ejemplo@ejemplo.com";
try {
    // Comprueba si el email es válido
    if(filter_var($email, FILTER_VALIDATE_EMAIL) === FALSE) {
        throw new MiExcepcion($email);
    }
    // Comprueba la palabra ejemplo en la dirección email
    if(strpos($email, "ejemplo") !== FALSE) {
        throw new Exception("$email es un email de ejemplo no válido");
    }
} catch (MiExcepcion $e) {
    echo $e->miFuncion();
} catch(Exception $e) {
    echo $e->getMessage();
}
```

Excepciones

- Desde PHP 7, existe el tipo Throwable, el cual es un interfaz que implementan tanto los errores como las excepciones, y nos permite capturar los dos tipos a la vez:

```
ej17: throwable.php

<?php
try {
    // tu codigo
} catch (Throwable $e) {
    echo 'Forma de capturar errores y excepciones a la vez';
}
```

- Si sólo queremos capturar los errores fatales, podemos hacer uso de la clase Error:

```
ej17: throwable.php

<?php
try {
    // Genera una notificación que no se captura
    echo $variableNoAsignada;
    // Error fatal que se captura
    funcionQueNoExiste();
} catch (Error $e) {
    echo "Error capturado: " . $e->getMessage();
}
```

Relanzar excepciones

- En las aplicaciones reales, es muy común capturar una excepción de sistema y lanzar una de aplicación que hemos definido nosotros. También podemos lanzar las excepciones sin necesidad de estar dentro de un try/catch.

ej17: relanzar.php

```
<?php
class AppException extends Exception {}

try {
    // Código de negocio que falla
} catch (Exception $e) {
    throw new AppException("AppException: ".$e->getMessage(), $e->getCode(), $e);
}
```

SPL

Standard PHP Library es el conjunto de funciones y utilidades que ofrece PHP, como:

- Estructuras de datos
- Pila, cola, cola de prioridad, lista doblemente enlazada, etc...
- Conjunto de iteradores diseñados para recorrer estructuras agregadas, arrays, resultados de bases de datos, árboles XML, listados de directorios, etc.
- Podéis consultar la documentación en <https://www.php.net/manual/es/book.spl.php> o ver algunos ejemplos en <https://diego.com.es/tutorial-de-la-libreria-spl-de-php>
- También podéis consultar la documentación de estas excepciones en <https://www.php.net/manual/es/spl.exceptions.php>
- También define un conjunto de excepciones que podemos utilizar para que las lancen nuestras aplicaciones:
 - LogicException (extends Exception)
 - BadFunctionCallException
 - BadMethodCallException
 - DomainException
 - InvalidArgumentException
 - LengthException
 - OutOfRangeException
 - RuntimeException (extends Exception)
 - OutOfBoundsException
 - OverflowException
 - RangeException
 - UnderflowException
 - UnexpectedValueException

¿PREGUNTAS?

Bibliografía

- Páginas web que han ayudado en la construcción de estas diapositivas:
- <https://github.com/olga3emes/PHP-Web/wiki>
- <http://phpdevenezuela.github.io/php-the-right-way/>
- <https://www.mclibre.org/consultar/php/>
- <https://www.php.net/manual/es/index.php>
- Manual de OO en PHP – www.desarrolloweb.com
- Ediciones ENI: PHP (Acceso restringido)

Sesiones, Cookies y Autenticación

Índice

1. Variables de servidor
2. Formularios
3. Validación
4. Parámetro multivalor
5. Subiendo archivos
6. Cabeceras de respuesta
7. Gestión del estado
8. Autenticación de usuarios

Variables de servidor

PHP almacena la información del servidor y de las peticiones HTTP en seis arrays globales:

- `$_ENV`: información sobre las variables de entorno
- `$_GET`: parámetros enviados en la petición GET
- `$_POST`: parámetros enviados en el envío POST
- `$_COOKIE`: contiene las cookies de la petición, las claves del array son los nombres de las cookies
- `$_SERVER`: información sobre el servidor
- `$_FILES`: información sobre los ficheros cargados vía upload

Si nos centramos en el array `$_SERVER` podemos consultar las siguientes propiedades:

- `PHP_SELF`: nombre del script ejecutado, relativo al document root (p.ej: tienda/carrito.php)
- `SERVER_SOFTWARE`: (p.ej: Apache)
- `SERVER_NAME`: dominio, alias DNS (p.ej: www.elche.es)
- `REQUEST_METHOD`: GET
- `REQUEST_URI`: URI, sin el dominio
- `QUERY_STRING`: todo lo que va después de ? en la URL (p.ej: `heroe=Batman&nombre=Bruce`)

Más información en <https://www.php.net/manual/es/reserved.variables.server.php>

Variables de servidor

Otras propiedades relacionadas:

PATH_INFO: ruta extra tras la petición. Si la URL es <http://www.php.com/php/pathInfo.php/algo/cosa?foo=bar>, entonces `$_SERVER['PATH_INFO']` será `/algo/cosa`.

REMOTE_HOST: hostname que hizo la petición

REMOTE_ADDR: IP del cliente

AUTH_TYPE: tipo de autenticación (p.ej: Basic)

REMOTE_USER: nombre del usuario autenticado

Apache crea una clave para cada cabecera HTTP, en mayúsculas y sustituyendo los guiones por subrayados:

HTTP_USER_AGENT: agente (navegador)

HTTP_REFERER: página desde la que se hizo la petición

ej1.php

```
<?php
echo $_SERVER["PHP_SELF"]."<br>";
echo $_SERVER["SERVER_SOFTWARE"]."<br>";
echo $_SERVER["SERVER_NAME"]."<br>";
echo $_SERVER["REQUEST_METHOD"]."<br>";
echo $_SERVER["REQUEST_URI"]."<br>";
echo $_SERVER["QUERY_STRING"]."<br>";
?>
```

ej1.php

```
echo $_SERVER["HTTP_USER_AGENT"]."<br>";
```

Formularios

A la hora de enviar un formulario, debemos tener claro cuando usar GET o POST

GET: los parámetros se pasan en la URL

- <2048 caracteres, sólo ASCII
- Permite almacenar la dirección completa (marcador / historial)
- Idempotente: dos llamadas con los mismos datos siempre debe dar el mismo resultado
- El navegador puede cachear las llamadas

POST: parámetros ocultos (no encriptados)

- Sin límite de datos, permite datos binarios.
- No se pueden cachear
- No idempotente → actualizar la BBDD

Así pues, para recoger los datos accederemos al array dependiendo del método del formulario que nos ha invocado.

ej2.php

```
<?php
$par = $_GET["parametro"];
$par = $_POST["parametro"];
?>
```

Formularios

- Para los siguientes apartados nos vamos a basar en el siguiente ejemplo:

ej3.php

```
<form action="formulario.php" method="GET">
  <p><label for="nombre">Nombre del alumno:</label>
    <input type="text" name="nombre" id="nombre" value="" />
  </p>
  <p><input type="checkbox" name="modulos[]" id="modulosDWES" value="DWES" />
    <label for="modulosDWES">Desarrollo web en entorno servidor</label>
  </p>
  <p><input type="checkbox" name="modulos[]" id="modulosDWEC" value="DWEC" />
    <label for="modulosDWEC">Desarrollo web en entorno cliente</label>
  </p>
  <input type="submit" value="Enviar" name="enviar" />
</form>
```


Validación

Respecto a la validación, es conveniente siempre hacer validación doble:

- En el cliente mediante JS
- En servidor, antes de llamar a negocio, es conveniente volver a validar los datos.

ej3.php

```
<?php
if (isset($_GET["parametro"])) {
    $par = $_GET["parametro"];
    // comprobar si $par tiene el formato adecuado, su valor, etc...
}
```

Librerías de validación

Existen diversas librerías que facilitan la validación de los formularios, como son respect/validation o particle/validator.

Cuando estudiemos Laravel profundizaremos en la validación de forma declarativa.

Parámetro multivalor

Existen elementos HTML que envían varios valores:

- select múltiple
- checkbox

Para recoger los datos, el nombre del elemento debe ser un array.

De manera que luego al recoger los datos:

ej4.php

```
<p><select name="lenguajes[]" multiple="true">
  <option value="c">C</option>
  <option value="java">Java</option>
  <option value="php">PHP</option>
  <option value="python">Python</option>
</select></p>

<p><input type="checkbox" name="lenguajes[]" value="c" /> C<br /></p>
<p><input type="checkbox" name="lenguajes[]" value="java" /> Java<br /></p>
<p><input type="checkbox" name="lenguajes[]" value="php" /> Php<br /></p>
<p><input type="checkbox" name="lenguajes[]" value="python" /> Python<br /></p>
```

formulario.php

```
<?php
$lenguajes = $_GET["lenguajes"];
foreach ($lenguajes as $lenguaje) {
  echo "$lenguaje <br />";
}
```

Volviendo a rellenar un formulario

- Un sticky form es un formulario que recuerda sus valores. Para ello, hemos de rellenar los atributos value de los elementos HTML con la información que contenían.
- **ES IMPORTANTE, a los usuarios no les gusta tener que volver a rellenar un montón de campos si algo falla. Recordad que si no hay fallos, no es necesario y debe aparecer el formulario vacío.**

Ejemplos - PHP - ej5.php

```
<?php
if (!empty($_POST['modulos']) && !empty($_POST['nombre'])) {
    // Aquí se incluye el código a ejecutar cuando los datos son correctos
    echo "Todo correcto";
} else {
    // Generamos el formulario
    $nombre = $_POST['nombre'] ?? "";
    $modulos = $_POST['modulos'] ?? [];
?>
<form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="POST">
    <p><label for="nombre">Nombre del alumno:</label>
        <input type="text" name="nombre" id="nombre" value="<?= $nombre ?>" ></p>

    <p><input type="checkbox" name="modulos[]" id="modulosDWES" value="DWES"
    <?php if (in_array("DWES", $modulos)) echo 'checked="checked"'; ?> />
        <label for="modulosDWES">Desarrollo web en entorno servidor</label>
    </p>

    <p><input type="checkbox" name="modulos[]" id="modulosDWE" value="DWE"
    <?php if (in_array("DWE", $modulos)) echo 'checked="checked"'; ?> />
        <label for="modulosDWE">Desarrollo web en entorno cliente</label>
    </p>
    <input type="submit" value="Enviar" name="enviar" />
</form>
<?php } ?>
```

Subiendo archivos

- Se almacenan en el servidor en el array `$_FILES` con el nombre del campo del tipo file del formulario.

ej6

```
<form enctype="multipart/form-data" action="<?php echo $_SERVER['PHP_SELF']; ?>" method="POST">
  Archivo: <input name="archivoEnviado" type="file" />
  <br />
  <input type="submit" name="btnSubir" value="Subir" />
</form>
```

Configuración en php.ini:

- `file_uploads: on / off`
- `upload_max_filesize: 2M`
- `upload_tmp_dir`: directorio temporal. No es necesario configurarlo, cogerá el predeterminado del sistema
- `post_max_size`: tamaño máximo de los datos POST. Debe ser mayor a `upload_max_filesize`.
- `max_file_uploads`: número máximo de archivos que se pueden cargar a la vez.
- `max_input_time`: tiempo máximo empleado en la carga (GET/POST y upload → normalmente se configura en 60)
- `memory_limit: 128M`
- `max_execution_time`: tiempo de ejecución de un script (no tiene en cuenta el upload)

Subiendo archivos

- Para cargar los archivos, accedemos al array `$_FILES`:

Cada archivo cargado en `$_FILES` tiene:

- name: nombre
- tmp_name: ruta temporal
- size: tamaño en bytes
- type: tipo MIME
- error: si hay error, contiene un mensaje. Si ok → 0.

```
ej6

<?php
    if(isset($_FILES['image'])){
        $errors= array();
        $file_name = $_FILES['image']['name'];
        $file_size = $_FILES['image']['size'];
        $file_tmp = $_FILES['image']['tmp_name'];
        $file_type = $_FILES['image']['type'];
        $file_ext=strtolower(pathinfo($file_name, PATHINFO_EXTENSION));
        $extensions= array("jpeg","jpg","png");
        if(in_array($file_ext,$extensions)=== false){
            $errors[]="Extensión no permitida. Prueba con imágenes jpeg,jpng o png.";
        }
        if($file_size > 2097152) {
            $errors[]="El archivo no puede pesar más de 2MB";
        }
        if(empty($errors)==true) {
            move_uploaded_file($file_tmp,"../images/".$file_name);
            echo "La imagen se ha guardado correctamente";
        }else{
            print_r($errors);
        }
    }
}

?>
<html>
    <body>
        <form action = "" method = "POST" enctype = "multipart/form-data">
            <input type = "file" name = "image" />
            <input type = "submit"/>
        </form>

    </body>
</html>
```

Cabeceras de respuesta

- Debe ser lo primero a devolver. Se devuelven mediante la función `header(cadena)`. Mediante las cabeceras podemos configurar el tipo de contenido, tiempo de expiración, redireccionar el navegador, especificar errores HTTP, etc.
- Se puede comprobar en las herramientas del desarrollador de los navegadores web mediante Developer Tools → Network → Headers.
- Es muy común configurar las cabeceras para evitar consultas a la caché o provocar su renovación:

ej7

```
<?php header("Content-Type: text/plain"); ?>
<?php header("Location: http://www.ejemplo.com/inicio.html");
exit();
?>
<?php
//TODO: gmdate sustituye en uso a gmstrftime deprecated v8.1.
// tres horas
$now = time();
echo date("d m Y H:i:s", $now) . "\n";
$horas3 = date("d m Y H:i:s", $now + 60 * 60 * 3);
echo $horas3 . "\n";
header("Expires: {$horas3}");
// un año
$now = time();
$anyo1 = date("d m Y H:i:s", $now + 365 * 86440);
echo $anyo1 . "\n";
header("Expires: {$anyo1}");
// se marca como expirado (fecha en el pasado)
$pasado = date("d m Y H:i:s");
echo $pasado;
header("Expires: {$pasado}");
// evitamos cache de navegador y/o proxy
header("Expires: Mon, 26 Jul 1997 05:00:00 GMT");
header("Last-Modified: " . gmdate("D, d M Y H:i:s") . " GMT");
header("Cache-Control: no-store, no-cache, must-revalidate");
header("Cache-Control: post-check=0, pre-check=0", false);
header("Pragma: no-cache");
```

Gestión del estado

- HTTP es un protocolo stateless, sin estado propio, pero sí tenemos estado de las peticiones (no confundir una cosa con la otra). Por ello, se simula el estado mediante el uso de cookies, tokens o la sesión. El estado es necesario para procesos tales como el carrito de la compra, operaciones asociadas a un usuario, etc... El mecanismo de PHP para gestionar la sesión emplea cookies de forma interna. Las cookies se almacenan en el navegador, y la sesión en el servidor web.

Cookies

- Las cookies se almacenan en el array global `$_COOKIE`. Lo que coloquemos dentro del array, se guardará en el cliente. Hay que tener presente que el cliente puede no querer almacenarlas.
- Existe una limitación de 20 cookies por dominio y 300 en total en el navegador.
- En PHP, para crear una cookie se utiliza la función `setcookie`:

```
<?php
setcookie(nombre [, valor [, expira [, ruta [, dominio [, seguro [, httponly ]]]]]]);
setcookie(nombre [, valor = "" [, opciones = [] ]] )
?>
```


Gestión del estado

Cookies

- Destacar que el nombre no puede contener espacios ni el carácter;. Respecto al contenido de la cookie, no puede superar los 4 KB.
- Por ejemplo, mediante cookies podemos comprobar la cantidad de visitas diferentes que realiza un usuario:

ej8

```
<?php
$accesosPagina = 0;
$accesosPagina = $_COOKIE['accesos']; // recuperamos una cooki
if (isset($_COOKIE['accesos'])) {      e
    setcookie('accesos', ++$accesosPagina); // le asignamos un valo
}                                       r
?>
```

Inspeccionando las cookies
Si queremos ver que contienen las cookies que tenemos almacenadas en el navegador, se puede comprobar su valor en *Dev Tools* → *Application* → *Storage*

Gestión del estado

Cookies

- El tiempo de vida de las cookies puede ser tan largo como el sitio web en el que residen. Ellas seguirán ahí, incluso si el navegador está cerrado o abierto.
- Para borrar una cookie se puede poner que expiren en el pasado:

```
<?php
setcookie(nombre, "", 1) // pasado
?>
```

- O que caduquen dentro de un periodo de tiempo determinado:

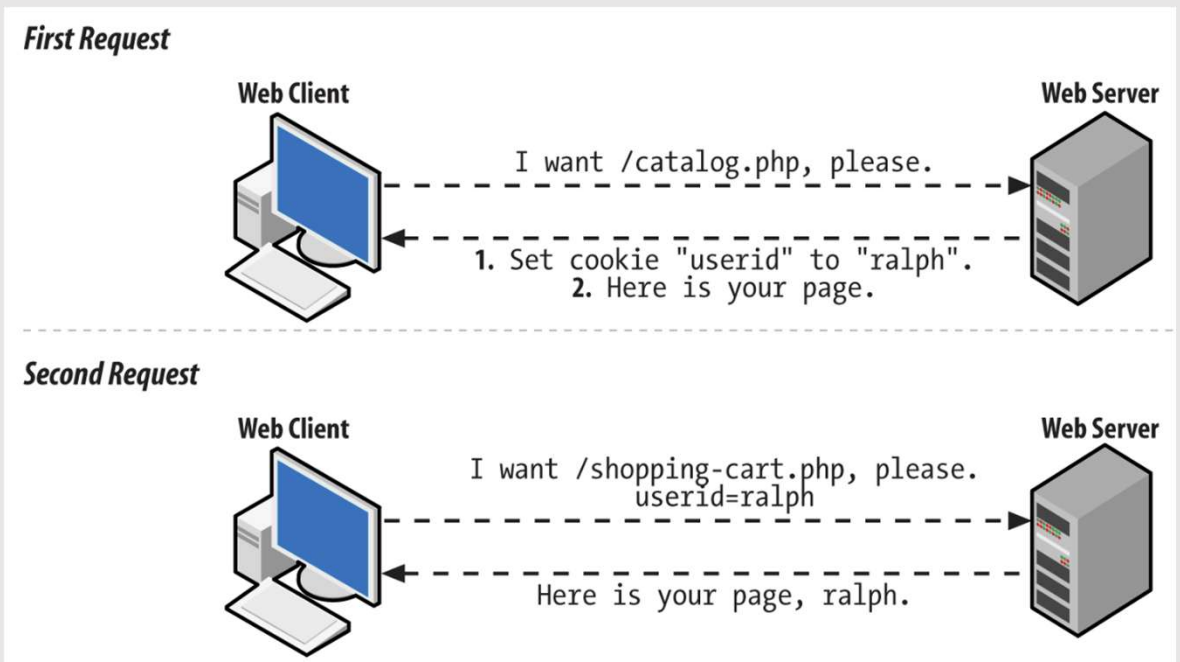
```
<?php
setcookie(nombre, valor, time() + 3600) // Caducan dentro de una hora
?>
```

Gestión del estado

Cookies

Se utilizan para:

- Recordar los inicios de sesión
- Almacenar valores temporales de usuario.
- Si un usuario está navegando por una lista paginada de artículos, ordenados de cierta manera, podemos almacenar el ajuste de la clasificación.
- La alternativa en el cliente para almacenar información en el navegador es el objeto **LocalStorage**.



Gestión del estado

Sesión

- La sesión añade la gestión del estado a HTTP, almacenando en este caso la información en el servidor. Cada visitante tiene un ID de sesión único, el cual por defecto se almacena en una cookie denominada **PHPSESSID**. Si el cliente no tiene las cookies activas, el ID se propaga en cada URL dentro del mismo dominio. Cada sesión tiene asociado un almacén de datos mediante el array global **\$_SESSION**, en el cual podemos almacenar y recuperar información.
- La sesión comienza al ejecutar un script PHP. Se genera un nuevo ID y se cargan los datos del almacén:



Gestión del estado

Sesión

- Las operaciones que podemos realizar con la sesión son:

```
<?php
session_start(); // carga la sesión
session_id(); // devuelve el id
$_SESSION[clave] = valor; // inserción
session_destroy(); // destruye la sesión
unset($_SESSION[clave]); // borrado
```

- Vamos a ver mediante un ejemplo como podemos insertar en una página datos en la sesión para posteriormente en otra página acceder a esos datos. Por ejemplo, en ej9.php y session.php.

ej9

```
<?php
session_start(); // inicializamos
$_SESSION["daw"] = "Ilerna"; // asignación
$daw = $_SESSION["daw"]; // recuperación
echo "Estamos en $daw ";
?>
<br />
<a href="session.php">Y luego</a>
```

session.php

```
<?php
session_start();
$daw = $_SESSION["daw"]; // recuperación
echo "Otra vez, en $daw ";
?>
```

Gestión del estado

Configurando la sesión en php.ini:

Las siguiente propiedades de php.ini permiten configurar algunos aspectos de la sesión:

- `session.save_handler`: controlador que gestiona cómo se almacena (files)
- `session.save_path`: ruta donde se almacenan los archivos con los datos (si tenemos un clúster, podríamos usar `/mnt/sessions` en todos los servidor de manera que apuntan a una carpeta compartida)
- `session.name`: nombre de la sesión (PHSESSID)
- `session.auto_start`: Se puede hacer que se autocargue con cada script. Por defecto está deshabilitado
- `session.cookie_lifetime`: tiempo de vida por defecto

Más información en la documentación oficial.

Autenticación de usuarios

Una sesión establece una relación anónima con un usuario particular, de manera que podemos saber si es el mismo usuario entre dos peticiones distintas. Si preparamos un sistema de login, podremos saber quién utiliza nuestra aplicación.

Para ello, preparemos un sencillo sistema de autenticación:

1. Mostrar el formulario login/password
2. Comprobar los datos enviados
3. Añadir el login a la sesión
4. Comprobar el login en la sesión para realizar tareas específicas del usuario
5. Eliminar el login de la sesión cuando el usuario la cierra.

Vamos a ver en código cada paso del proceso. (ej10)

Autenticación en producción

En la actualidad la autenticación de usuario no se realiza gestionando la sesión directamente, sino que se realiza mediante algún framework que abstraer todo el proceso o la integración de mecanismos de autenticación tipo *OAuth*, como estudiaremos en la última unidad mediante *Laravel*.

¿PREGUNTAS?



PROMETEO

PROMETEO

Unidad 6



Licencia



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

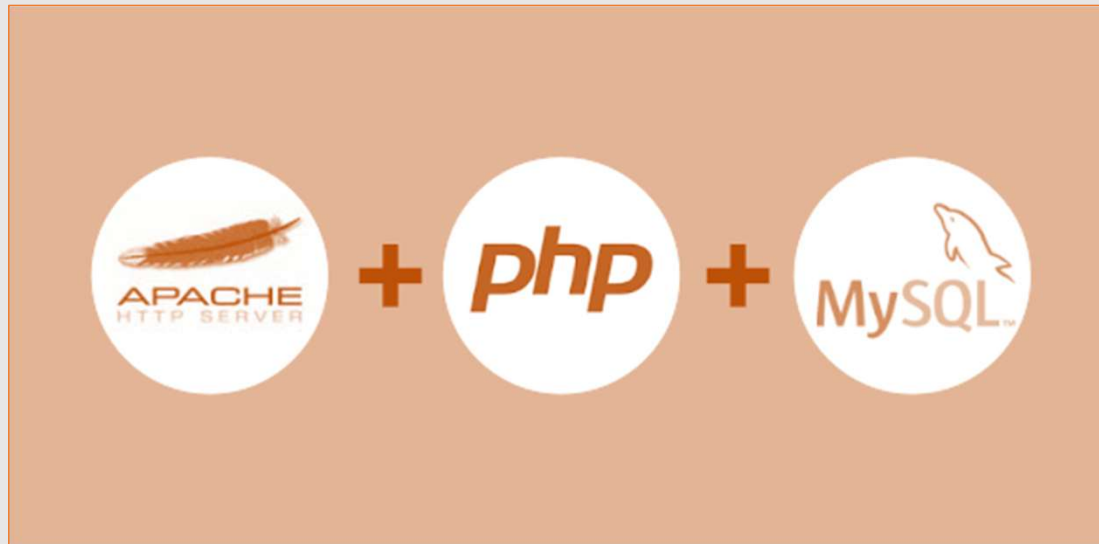
En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

ÍNDICE

1. Introducción
2. MySQLi
3. Ejercicios
4. PHP Data Objects :: PDO
5. Ejercicios

Introducción

- En esta unidad vamos a aprender a acceder a datos que se encuentran en un servidor; recuperando, editando y creando dichos datos a través de una base de datos.
- A través de las distintas capas o niveles, de las cuales 2 de ellas ya conocemos (Apache, PHP) y MySQL la que vamos a estudiar en este tema y deberíamos estar familiarizados con el lenguaje SQL (Módulo Bases de Datos).



MySQLi

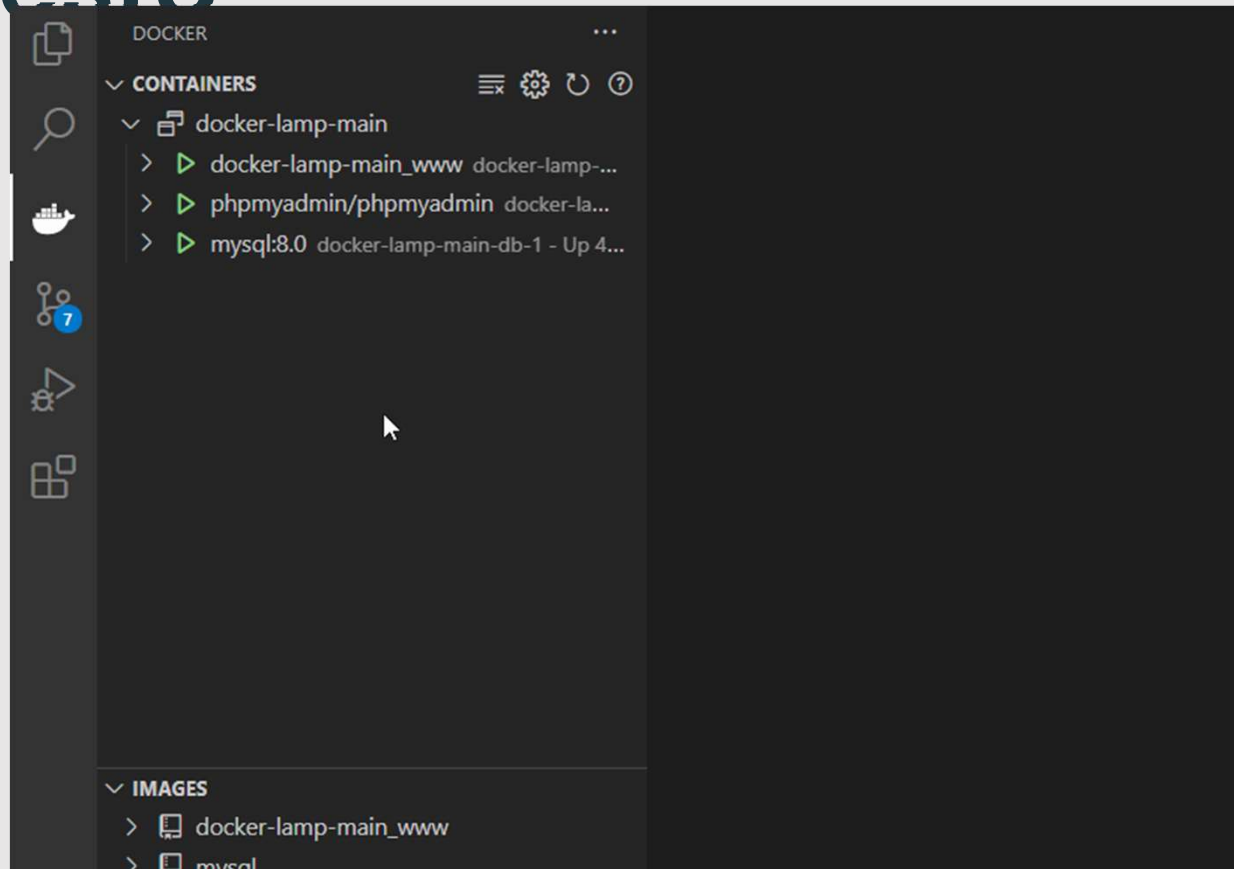
- PHP hace uso de esta extensión mejorada para poder comunicarse con las bases de datos, ya sean MySQL (4.1 o superior) o MariaDB.
- Cualquier consulta que queramos hacer a una base de datos necesitaremos hacer uso de la extensión mysqli()
- Veamos como conectarnos con una base de datos a través del código PHP:

```
<?php
// "SERVIDOR", "USUARIO", "CONTRASEÑA", "BASE DE DATOS"
$conexion = mysqli_connect("nombreServidor","tuUsuario","laPassword","NombreDeLaBBDD");

// COMPROBAMOS LA CONEXIÓN
if(mysqli_connect_errno()) {
    echo "Failed to connect to MySQL: " . mysqli_connect_error();
    exit();
}

echo "<h1>Bienvenid@ a MySQL !!</h1>";
?>
```

si queremos usar Docker podemos hacer esto



mysqli – hacer consultas

```
<?php

$conexion = mysqli_connect("d939ebf6a741", "root", "1234", "pruebas");

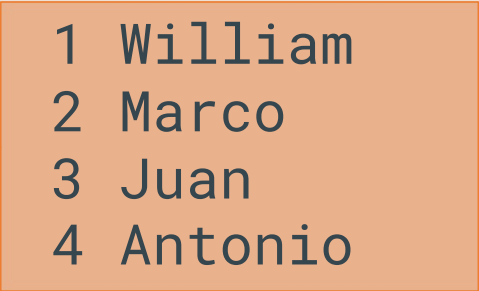
// COMPROBAMOS LA CONEXIÓN
if (mysqli_connect_errno()) {
    echo "Failed to connect to MySQL: " . mysqli_connect_error();
    exit();
}

// CONSULTA A LA BASE DE DATOS
$consulta = "SELECT * FROM `Personas`";
$listaUsuarios = mysqli_query($conexion, $consulta);

// COMPROBAMOS SI EL SERVIDOR NOS HA DEVUELTO RESULTADOS
if($listaUsuarios) {

    // RECORREMOS CADA RESULTADO QUE NOS DEVUELVE EL SERVIDOR
    foreach ($listaUsuarios as $usuario) {
        echo "$usuario[id] $usuario[name] <br>";
    }
}

?>
```



1 William
2 Marco
3 Juan
4 Antonio

ejercicios – MYSQLi

Crea una nueva base de datos con el nombre lol.

En nuestra base de datos lol que acabamos de crear, vamos a crear la tabla champ con los siguientes campos:

- id [*]
 - name
 - rol
 - difficulty
 - description
-
- Acuérdate que [*] significa que es clave primaria y no olvides poner el tipo de datos de cada uno de los campos.
 - Rellena la tabla champ con, al menos 5 registros, con los datos que tú quieras o si lo prefieres, puedes basarte en la página oficial del juego.

ejercicios – MySQLi

- Crea el archivo 001campeones.php donde listes todos los campeones del LOL que has metido en tu base de datos. Acuérdate que para ello deberás hacer una conexión con la base de datos y un foreach para cada campeón que tengas albergado en la tabla champ.
- Modifica el archivo 001campeones.php y guárdalo como 002campeones.php pero pon al lado de cada uno de los campeones listados un botón para editar y otro para borrar. Cada uno de esos botones hará la correspondiente función dependiendo del id del campeón seleccionado.
- Al pinchar en editar, el usuario será redirigido al archivo 003editando.php donde mostrará un formulario con los campos rellenos por los datos del campeón seleccionado. Al darle al botón de guardar los datos se guardarán en la base de datos y el usuario será redirigido a la lista de champs para poder ver los cambios.
- Al pinchar en borrar, el usuario será preguntado a través de un mensaje de JavaScript si está seguro de que quiere borrar al campeón seleccionado. En el mensaje de confirmación debe aparecer el nombre del campeón seleccionado. Si el usuario pincha en Aceptar el campeón será eliminado de la base de datos y el usuario será redirigido nuevamente al listado de campeones para comprobar que, efectivamente dicho campeón se ha eliminado de la lista.

ejercicios – MySQLi

- Modifica el archivo 001campeones.php y guárdalo como 003campeones.php para que se muestre como una tabla con las columnas de la propia tabla de la base de datos, es decir; id, nombre, rol, dificultad, descripción.
- Al lado de cada nombre de cada columna, pon 2 iconos que sean $\wedge$ $\vee$ y que cada uno de ellos ordene el listado en función de cuál se haya pinchado.
- Si se ha pulsado en Nombre el icono de $\wedge$, el listado debe aparecer ordenado por nombre ascendente. Si por el contrario se ha pulsado $\vee$ tendrá que ordenarse por nombre descendente.
- Ten en cuenta que cada icono debe llevar consigo un enlace al listado que contenga parámetros en la URL que satisfagan las opciones seleccionadas así que haced uso de `$_GET` para poder capturarlos y escribid las consultas SQL que sean necesarias para hacer cada uno de los filtros.

PHP Data Objects :: PDO

- De la misma manera que hemos visto con mysqli, PHP Data Objects (o PDO) es un driver de PHP que se utiliza para trabajar bajo una interfaz de objetos con la base de datos. A día de hoy es lo que más se utiliza para manejar información desde una base de datos, ya sea relacional o no relacional.
- De igual manera que pasaba con los objetos en PHP nativos, en la interfaz de MySQL la cosa cambia la hora de conectarnos con una base de datos.

```
<?php
    $conexion = new PDO('mysql:host=localhost; dbname=dwes', 'root', 'password');
```

PHP Data Objects :: PDO

- Además, con PDO podemos usar las excepciones con try catch para gestionar los errores que se produzcan en nuestra aplicación, para ello, como hacíamos antes, debemos encapsular el código entre bloques try / catch.

En primer lugar, creamos la conexión con la base de datos a través del constructor PDO pasándole la información de la base de datos. En segundo lugar, establecemos los parámetros para manejar las excepciones, en este caso hemos utilizado:

- `PDO::ATTR_ERRMODE` indicándole a PHP que queremos un reporte de errores.
- `PDO::ERRMODE_EXCEPTION` con este atributo obligamos a que lance excepciones, además de ser la opción más humana y legible que hay a la hora de controlar errores.

Cualquier error que se lance a través de PDO, el sistema lanzará una `PDOException`.

```
<?php
$dsn = 'mysql:dbname=prueba;host=127.0.0.1';
$usuario = 'usuario';
$contraseña = 'contraseña';
try {
    $mbd = new PDO($dsn, $usuario, $contraseña);
    $mbd->setAttribute(PDO::ATTR_ERRMODE, PDO::
ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    echo 'Falló la conexión: ' . $e->getMessage();
}
?>
```

PHP Data Objects :: PDO

Fichero de configuración de la BD

- De la misma manera que creamos un archivo de funciones y albergamos todas las funciones que se usan de manera global en la aplicación (idea muy útil), podemos establecer un archivo de constantes donde definamos los parámetros de conexión con la base de datos, así lo hacen los frameworks y lo veremos con Laravel, más adelante.

```
<?php

const DSN = "mysql:host=localhost;dbname=dwes";
const USUARIO = "root";
const PASSWORD = "abc123";

/*
 *
 * NO SUBAS ESTE ARCHIVO A git
 *
 */
```

PHP Data Objects :: PDO

Sentencias preparadas

- Se trata de sentencias que se establecen como si fueran **plantillas** de la consulta SQL que vamos a lanzar, aceptando parámetros que son establecidos a posteriori de la declaración de la sentencia preparada.
- Las sentencias preparadas evitan la inyección de SQL (SQL Injection) y mejoran el rendimiento de nuestras aplicaciones o páginas web.

```
<?php
$sql = "INSERT INTO Clientes VALUES (?, ?, ?, ?)";
```

- Cada interrogante de la consulta debe ser reemplazado por un parámetro, como se muestra más abajo.
- Una vez tenemos la plantilla de nuestra consulta, debemos seguir con la preparación, junto con 3 métodos más de PHP para su completa ejecución:
 - **prepare**: prepara la sentencia antes de ser ejecutada
 - **bind**: el tipo de unión (bind) de dato que puede ser mediante '?' o ':parámetro'
 - **execute**: se ejecuta la consulta uniendo la plantilla con las variables o parámetros que hemos establecido.

PHP Data Objects :: PDO

```
<?php
//Borrando con parámetros
include "config/conexion.php";

$conexion = null;

try {
    $cantidad = $_GET["cantidad"];

    $conexion = new PDO(DSN, USUARIO, PASSWORD);
    $conexion -> setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "DELETE FROM stock WHERE unidades = ?";
    $sentencia = $conexion -> prepare($sql);

    $isOk = $sentencia -> execute([$cantidad]);
    $cantidadAfectada = $sentencia -> rowCount();

    echo $cantidadAfectada;
} catch (PDOException $e) {
    echo $e -> getMessage();
}

$conexion = null;
```

PHP Data Objects :: PDO

Ejemplo bindParam

```
<?php
include "config/conexion.php";

$conexion=null;

try {
    $cantidad = $_GET["cantidad"] ?? 0;

    $conexion = new PDO(DSN, USUARIO, PASSWORD);
    $conexion -> setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "DELETE FROM stock WHERE unidades = :cant";

    $sentencia = $conexion -> prepare($sql);
    $sentencia -> bindParam(":cant", $cantidad);

    $isOk = $sentencia -> execute();
    $cantidadAfectada = $sentencia -> rowCount();

    echo $cantidadAfectada;
} catch (PDOException $e) {
    echo $e -> getMessage();
}

$conexion = null;
```


PHP Data Objects :: PDO

bindParam VS bindValue

- Utilizaremos bindValue() cuando tengamos que i deberemos usar bindParam() cuando tengamos un array.
- Para más información y uso de las variables PDO

```
<?php
// se asignan nombre a los parámetros
$sql = "DELETE FROM stock WHERE unidades = :cant";
$sentencia = $conexion -> prepare($sql);

// bindParam enlaza por referencia
$cantidad = 0;

$sentencia -> bindParam(":cant", $cantidad);
$cantidad = 1;

// se eliminan con cant = 1
$isOk = $sentencia -> execute();

// bindValue enlaza por valor
$cantidad = 0;

$sentencia -> bindValue(":cant", $cantidad);
$cantidad = 1;

// se eliminan con cant = 0
$isOk = $sentencia->execute();
```

PHP Data Objects :: PDO

Insertando registros

- A la hora de insertar registros en una base de datos, debemos tener en cuenta que en la tabla puede haber valores autoincrementados. Para salvaguardar esto, lo que debemos hacer es dejar ese campo autoincrementado vacío, pero a la hora de hacer la conexión, debemos recuperarlo con el método `lastInsertId()`.

```
<?php
$nombre = $_GET["nombre"] ?? "Ilerna Shop";
$telefono = $_GET["telefono"] ?? "666123456";

$sql="INSERT INTO tienda(nombre, tlf) VALUES (:nombre, :telefono)";

$sentencia = $conexion -> prepare($sql);
$sentencia -> bindParam(":nombre", $nombre);
$sentencia -> bindParam(":telefono", $telefono);

$isOk = $sentencia -> execute();
$idGenerado = $conexion -> lastInsertId();

echo $idGenerado;
```

PHP Data Objects :: PDO

Consultando registros

A la hora de recuperar los resultados de una consulta, bastará con invocar al método `PDOStatement::fetch` para listar las filas generadas por la consulta.

Pero debemos elegir el tipo de dato que queremos recibir entre los 3 que hay disponibles:

- `PDO::FETCH_ASSOC`: array indexado cuyos keys son el nombre de las columnas.
- `PDO::FETCH_NUM`: array indexado cuyos keys son números.
- `PDO::FETCH_BOTH`: valor por defecto. Devuelve un array indexado cuyos keys son tanto el nombre de las columnas como los números.

Consulta	SELECT * FROM artículos		
Columnas	id	articulo	precio
1a línea del resultado	1	Albaricoques	35.5

Resultado de un fetch (lectura con diferentes funciones)

PDO::FETCH_NUM		PDO::FETCH_ASSOC:		PDO::FETCH_BOTH	
Clave	Valor	Clave	Valor	Clave	Valor
0	1	id	1	id	1
1	Albaricoques	articulo	Albaricoques	0	1
2	35.5	precio	35.5	texto	Albaricoques
				1	Albaricoques
				precio	35.5
				2	35.5

PHP Data Objects :: PDO

```
<?php
// consulta con array asociativo
include "config/conexion.php";
$conexion = null;

try{
    $conexion = new PDO(DSN, USUARIO, PASSWORD);
    $conexion -> setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "select * from tienda";

    $sentencia = $conexion -> prepare($sql);
    $sentencia -> setFetchMode(PDO::FETCH_ASSOC);
    $sentencia -> execute();

    while($fila = $sentencia -> fetch()){
        echo "Codigo:" . $fila["cod"] . "<br />";
        echo "Nombre:" . $fila["nombre"] . "<br />";
        echo "Teléfono:" . $fila["tlf"] . "<br />";
    }

} catch(PDOException $e) {
    echo $e -> getMessage();
}

$conexion = null;
```

PHP Data Objects :: PDO

- Recuperando datos con una matriz como resultado de nuestra consulta:

```
$sentencia = $conexion->prepare($sql);  
$sentencia->setFetchMode(PDO::FETCH_ASSOC);  
$sentencia->execute();  
  
$tiendas = $sentencia->fetchAll();  
  
foreach ($tiendas as $tienda) {  
    echo "Codigo:" . $tienda["cod"] . "<br />";  
    echo "Nombre:" . $tienda["nombre"] . "<br />";  
}
```

PHP Data Objects :: PDO

- Pero si lo que queremos es leer datos con forma de objeto utilizando `PDO::FETCH_OBJ`, debemos crear un objeto con propiedades públicas con el mismo nombre que las columnas de la tabla que vayamos a consultar.

```
$sentencia = $conexion->prepare($sql);
$sentencia->setFetchMode(PDO::FETCH_OBJ);
$sentencia->execute();

while ($t = $sentencia->fetch()) {
    echo "Codigo:" . $t->cod . "<br />";
    echo "Nombre:" . $t->nombre . "<br />";
    echo "Teléfono:" . $t->tlf . "<br />";
}
```

PHP Data Objects :: PDO

Consultas con modelos

- Llevamos tiempo creando clases en PHP y las consultas también admiten este tipo de datos mediante el uso de `PDO::FETCH_CLASS`
- Si usamos este método, debemos tener en cuenta que los nombres de los atributos privados deben coincidir con los nombres de las columnas de la tabla que vayamos a manejar.
- Así pues, si por lo que sea cambiamos la estructura de la tabla DEBEMOS CAMBIAR nuestra clase para que todo siga funcionando.
- Pero **¿qué pasa si nuestras clases tienen constructor?** pues que debemos indicarle, al método `FETCH`, que rellene las propiedades después de llamar al constructor y para ello hacemos uso de `PDO::FETCH_PROPS_LATE`.

PHP Data Objects :: PDO

Consultas con modelos:

```
<?php

class Persona
{
    private int $cod;
    private string $nombre;
    private string $tlf;

    public function getCodigo(): int
    {
        return $this->cod;
    }

    public function getNombre(): string
    {
        return $this->nombre;
    }

    public function getTelefono(): string
    {
        return $this->tlf;
    }
}
```

```
$sql = "SELECT * FROM persona";
$sentencia = $conexion->prepare($sql);

// Aquí 'Persona' es el nombre de nuestra clase
$sentencia->setFetchMode(PDO::FETCH_CLASS, "Persona");
$sentencia->execute();

while ($t = $sentencia->fetch()) {
    echo "Codigo: " . $t->getCodigo() . "<br />";
    echo "Nombre: " . $t->getNombre() . "<br />";
    echo "Teléfono: " . $t->getTelefono() . "<br />";

    var_dump($t);
}
```

```
//Con constructor: PDO::FETCH_PROPS_LATE -----

$sql = "SELECT * FROM persona";

$sentencia = $conexion -> prepare($sql);
$sentencia -> setFetchMode(PDO::FETCH_CLASS |
PDO::FETCH_PROPS_LATE, Persona::class);
$sentencia -> execute();

$personas = $sentencia -> fetchAll();
```


PHP Data Objects :: PDO

Consultas con LIKE

- Para utilizar el comodín LIKE u otros comodines, debemos asociarlo al dato y NUNCA en la propia consulta.

```
$sql = "SELECT * FROM tienda where nombre like :nombre or tlf like :tlf";

$sentencia = $conexion -> prepare($sql);
$sentencia -> setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE, Persona::class);

$cadBuscar = "%" . $busqueda . "%";

$sentencia -> execute(["nombre" => $cadBuscar, "tlf" => $cadBuscar]);

$result = $sentencia -> fetchAll();
```

PHP Data Objects :: PDO

Login & Password

Para gestionar un sistema completo de login y password con contraseñas cifradas, necesitamos un método que cifre esos strings que el usuario introduce como contraseña; tanto en el formulario de registro como en el del login, ya que al codificar una contraseña, después tenemos que decodificarla para comprobar que ambas contraseñas (la que introduce el usuario en el login y la que tenemos en la base de datos) coincidan.

Necesitamos pues:

- `password_hash()`: para almacenar la contraseña en la base de datos a la hora de hacer el INSERT
- `PASSWORD_DEFAULT` almacenamos la contraseña usando el método de cifrado bcrypt
- `PASSWORD_BCRYPT` almacenamos la contraseña usando el algoritmo BLOWFISH compatible con `crypt()`
- `password_verify()` para verificar el usuario y la contraseña

PHP Data Objects :: PDO

```
//Sign in
$usu = $_POST["usuario"];
$pas = $_POST["password"];
$sql = "INSERT INTO usuarios(usuario, password) VALUES (:usuario, :password)";

$sentencia = $conexion -> prepare($sql);
$isOk = $sentencia -> execute([
    "usuario" => $usu,
    "password" => password_hash($pas,PASSWORD_DEFAULT)
]);
```

```
//Log in -----
$usu = $_POST["login"] ?? "";

$sql = "select * from usuarios where usuario = ?";

$sentencia = $conexion -> prepare($sql);
$sentencia -> execute([$usu]);

$usuario = $sentencia -> fetch();

if($usuario && password_verify($_POST['pass'], $usuario['password'])) {
    echo"OK!";
} else {
    echo"K0";
}
```

Ejercicios

Aprovecha lo que hiciste de los ejercicios con MySQLi, pero utilizando PDO.

Crea una tabla nueva dentro de la base de datos lol que ya tienes y crea un sistema de login con usuarios. Introduce en la base de datos al menos 3 usuarios diferentes con sus contraseñas distintas.

Recuerda que:

La tabla nueva ha de llamarse user. Los campos a crear en la nueva tabla deben ser:

- id
- name
- username
- password
- email

Las contraseñas deben ser cifradas antes de guardar los datos en la base de datos.

Crea el formulario 005registro.php donde el usuario introduzca los datos de registro y vincúlalo con 006nuevoUsuario.php para que recoja los datos mediante POST y los inserte en la base de datos si todo ha ido bien.

Queda PROHIBIDÍSIMO acceder a 007nuevoUsuario.php sin el formulario rellenado.

La sentencia de INSERT debe estar controlada para que no pueda introducirse ningún dato en blanco.

Ten en cuenta que estás modificando la base de datos y no queremos campos mal rellenos. Si todo ha ido bien, muestra un mensaje por pantalla diciendo El usuario XXX ha sido introducido en el sistema con la contraseña YYY.

PROMETEO

Unidad 7



Licencia



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatarario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

Índice

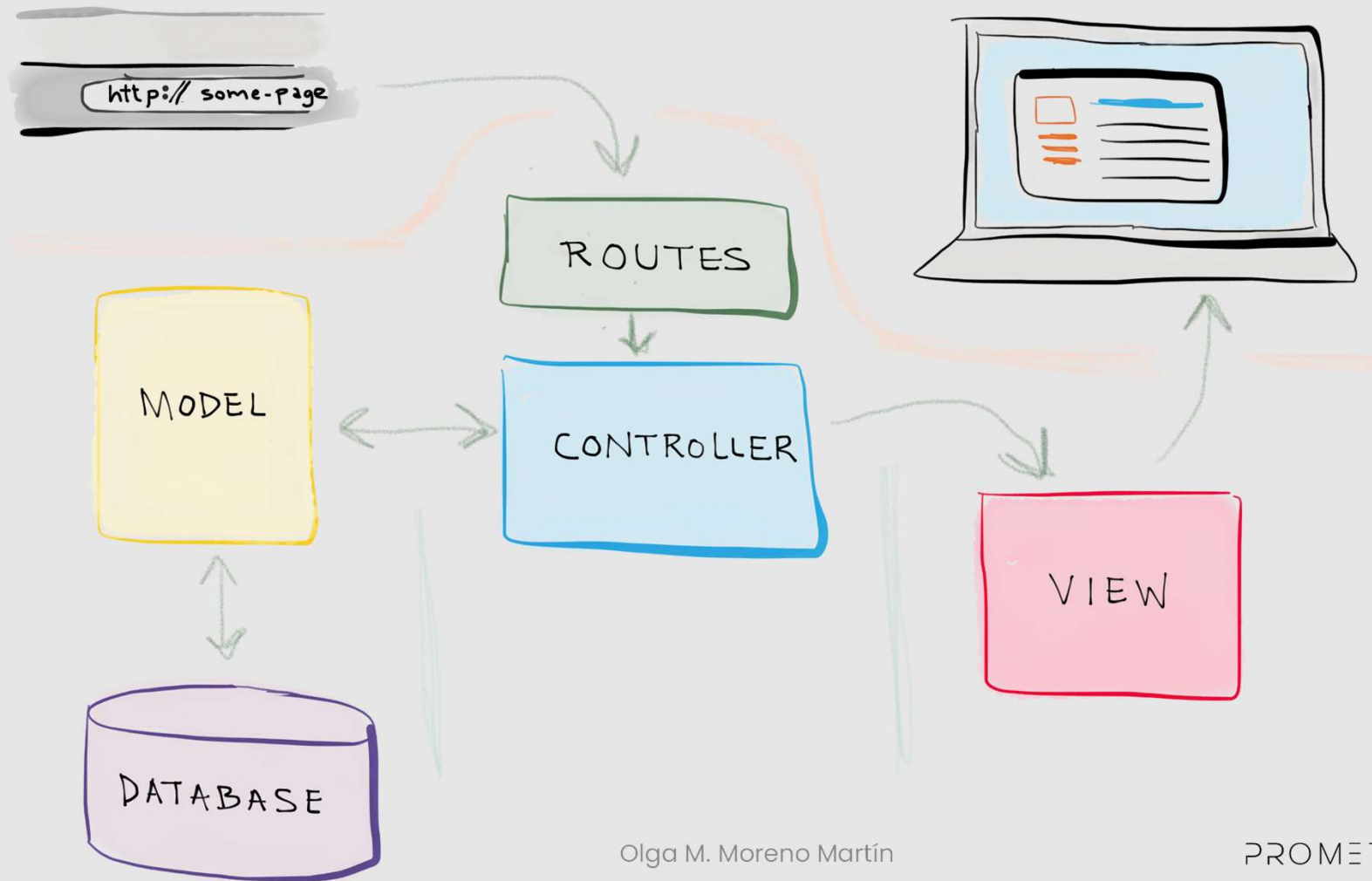
Laravel

1. Puesta en marcha
2. Instalación en entorno local
3. Migraciones
4. Carpetas principales
5. Rutas
6. Estructuras de control y plantillas
7. Modelos y relaciones con Eloquent

Laravel

- Laravel es un framework de PHP para ayudarnos en un tipo de desarrollo sobre aplicaciones escritas en este lenguaje de programación. Este framework o más bien podríamos llamarlo compañero de ahora en adelante, nos ayuda en muchas cosas al desarrollar una aplicación, por medio de sus sistema de paquetes y de ser un framework del tipo MVC (Modelo-Vista-Controlador) da como resultado que podamos “despreocuparnos” en ciertos aspecto del desarrollo, cómo instanciar clases y métodos para usarlos en muchas partes de nuestra aplicación sin la necesidad de escribirlo y repetirlos muchas veces con lo que eso conlleva a la hora de modificar algo en el código.
- Funciona como muchos otros, pero desde la línea de comandos contamos con el famoso Artisan que es el nombre que le dan a esta interfaz por comandos para ejecutar muchas funcionalidades, como ver todas las rutas de la aplicación disponible, o poner a correr la aplicación o pararla. Es tan potente y sencillo de usar que una vez que lo has probado en alguna aplicación te puedes “malacostumbrar” y echarlo de menos cuando haces otra aplicación con otro framework que no cuenta con él.
- Documentación Oficial MUY BUENA: <https://laravel.com/docs/12.x/>

Laravel

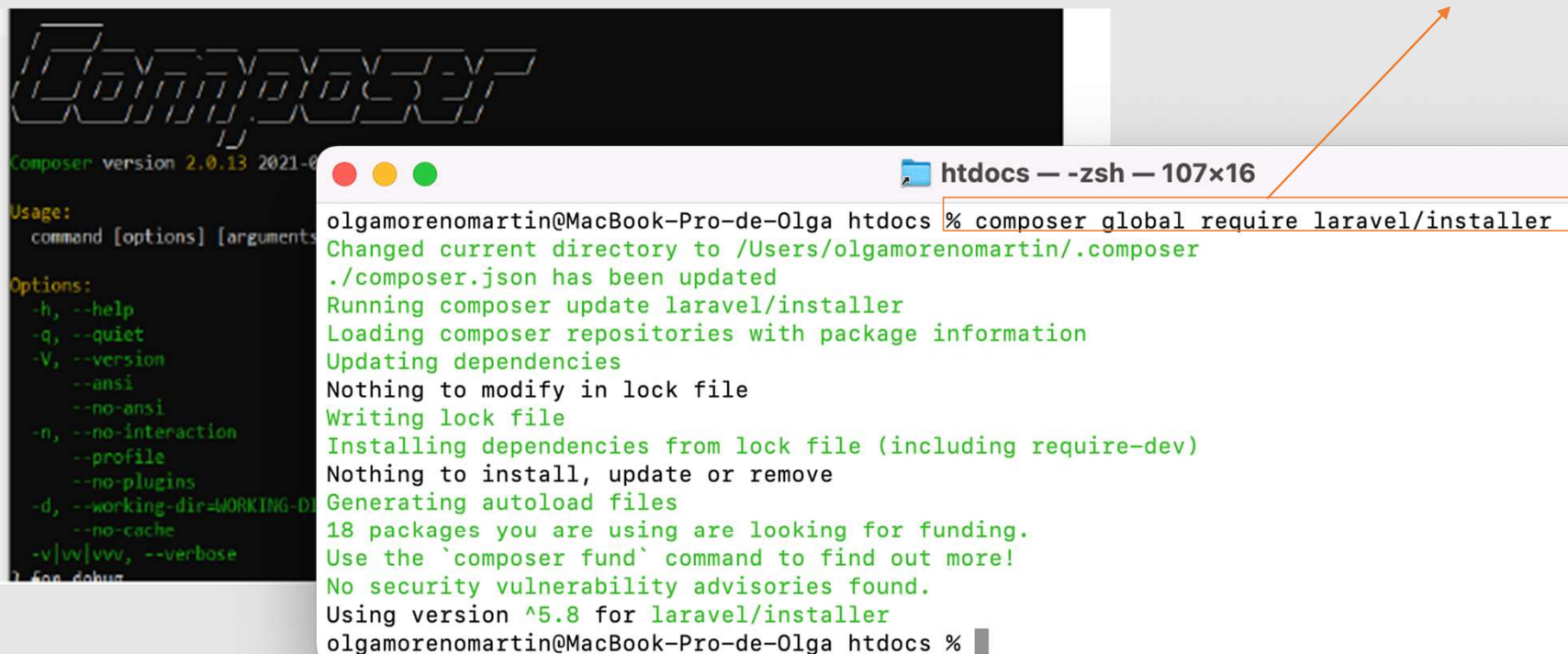


Laravel – características

- Su **motor de plantilla**, llamado **Blade**, da numerosas posibilidades para hacer unas páginas visualmente muy potentes y eficaces, capaz de utilizar sus propias variables y reutilizarlas.
- Su arquitectura es conocida como **MVC (Modelo-Vista-Controlador)** que da muchas facilidades para relacionar de manera clara y sencilla todas las partes de una aplicación. Esta arquitectura es muy usada en el mundo del software, otros framework pueden distintos de Laravel pueden resultar muy similares gracias a compartir la misma arquitectura MVC.
- **Eloquent ORM**, es muy intuitivo para **escribir consultas en PHP sobre objetos**. Otros framework cuenta con Doctrine por ejemplo, otro tipo de ORM que quizás te podría sonar más que el que usa Laravel.
- En **seguridad**, ofrece un nivel bastante fuerte con mecanismos de hash y salt para encriptar por medio de librerías como BCrypt, que también lo usa por ejemplo Zend Framework.
- **Artisan**, su **sistema de comandos** otorga al framework gran poder y a los programadores grandes facilidades y posibilidades, para crear controladores, entidades o actualizar la base de datos por ejemplo entre muchísimas cosas.
- **Librerías y modularidad**. Laravel aparte de sus propias librerías cuenta con ayuda de Symfony en otras muchas, otro MVC de los más usados en los últimos tiempos y con una gran comunidad detrás que hace que su avance y evolución sea muy significativo. También condiciona que la evolución de Laravel en parte dependa de Symfony en estos aspectos.
- **Base de datos (migraciones y seeders)**. Permite actualizar y migrar la base de datos una vez que el desarrollo ya está comenzamos y hay cambios en el código conforme se requiera sin necesidad de borrarla y volverla a crear, gracias a esto el riesgo de perder datos sean del valor que sean es mínimo. Además, gracias a su Schema Builder hace que no requiera usar el SQL, cuenta con un sistema intuitivo en PHP para hacerlo más fácil.

Instalación en local

- Una vez instalado composer y en la carpeta que queramos (aunque htdocs ya la tenemos) vamos a instalar Laravel en global



The image shows a terminal window titled "htdocs — -zsh — 107x16". The terminal output is as follows:

```
Composer version 2.0.13 2021-06-25 14:00:00

Usage:
  command [options] [arguments]

Options:
  -h, --help
  -q, --quiet
  -V, --version
  --ansi
  --no-ansi
  -n, --no-interaction
  --profile
  --no-plugins
  -d, --working-dir=WORKING-DIR
  --no-cache
  -v|vv|vvv, --verbose
  -f, --for-debug

olgamorenomartin@MacBook-Pro-de-Olga htdocs % composer global require laravel/installer
Changed current directory to /Users/olgamorenomartin/.composer
./composer.json has been updated
Running composer update laravel/installer
Loading composer repositories with package information
Updating dependencies
Nothing to modify in lock file
Writing lock file
Installing dependencies from lock file (including require-dev)
Nothing to install, update or remove
Generating autoload files
18 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
Using version ^5.8 for laravel/installer
olgamorenomartin@MacBook-Pro-de-Olga htdocs %
```

An orange arrow points from the terminal window to the text "instalado composer" in the list above.

instalación en local

composer create-project
laravel/laravel **example**

Nombre de
nuestro proyecto



```
htdocs — zsh — 107x34
Using version ^5.8 for laravel/installer
olgamorenomartin@MacBook-Pro-de-Olga htdocs % composer create-project laravel/laravel example
Creating a "laravel/laravel" project at "./example"
Installing laravel/laravel (v11.1.4)
- Downloading laravel/laravel (v11.1.4)
- Installing laravel/laravel (v11.1.4): Extracting archive
Created project in /Applications/XAMPP/xamppfiles/htdocs/example
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 107 installs, 0 updates, 0 removals
- Locking brick/math (0.12.1)
- Locking carbonphp/carbon-doctrine-types (3.2.0)
- Locking dflydev/dot-access-data (v3.0.3)
- Locking doctrine/inflector (2.0.10)
- Locking doctrine/lexer (3.0.1)
- Locking dragonmantank/cron-expression (v3.3.3)
- Locking egulias/email-validator (4.0.2)
- Locking fakerphp/faker (v1.23.1)
- Locking filp/whoops (2.15.4)
- Locking fruitcake/php-cors (v1.3.0)
- Locking graham-campbell/result-type (v1.1.3)
- Locking guzzlehttp/guzzle (7.9.2)
- Locking guzzlehttp/promises (2.0.3)
- Locking guzzlehttp/psr7 (2.7.0)
- Locking guzzlehttp/uri-template (v1.0.3)
- Locking hamcrest/hamcrest-php (v2.0.1)
- Locking laravel/framework (v11.21.0)
- Locking laravel/pint (v1.17.2)
- Locking laravel/prompts (v0.1.25)
- Locking laravel/sail (v1.31.1)
- Locking laravel/serializable-closure (v1.3.4)
- Locking laravel/tinker (v2.9.0)
- Locking league/commonmark (2.5.3)
```

Instalación en local

- Una vez dentro del proyecto nos encontraremos con esta estructura:

```
example — -zsh — 79x27
[olgamorenomartin@MacBook-Pro-de-Olga example % ls -la]
total 664
drwxr-xr-x  24 olgamorenomartin  admin    768  2 sep 18:57 .
drwxrwxr-x@ 11 root             admin    352  2 sep 18:57 ..
-rw-r--r--   1 olgamorenomartin  admin    258 16 jul 16:39 .editorconfig
-rw-r--r--   1 olgamorenomartin  admin   1128  2 sep 18:57 .env
-rw-r--r--   1 olgamorenomartin  admin   1077 16 jul 16:39 .env.example
-rw-r--r--   1 olgamorenomartin  admin    186 16 jul 16:39 .gitattributes
-rw-r--r--   1 olgamorenomartin  admin    258 16 jul 16:39 .gitignore
-rw-r--r--   1 olgamorenomartin  admin   4109 16 jul 16:39 README.md
drwxr-xr-x   5 olgamorenomartin  admin    160 16 jul 16:39 app
-rwxr-xr-x   1 olgamorenomartin  admin    350 16 jul 16:39 artisan
drwxr-xr-x   5 olgamorenomartin  admin    160 16 jul 16:39 bootstrap
-rw-r--r--   1 olgamorenomartin  admin   1925 16 jul 16:39 composer.json
-rw-r--r--   1 olgamorenomartin  admin  287528  2 sep 18:57 composer.lock
drwxr-xr-x  12 olgamorenomartin  admin    384 16 jul 16:39 config
drwxr-xr-x   7 olgamorenomartin  admin    224  2 sep 18:57 database
-rw-r--r--   1 olgamorenomartin  admin    244 16 jul 16:39 package.json
-rw-r--r--   1 olgamorenomartin  admin   1191 16 jul 16:39 phpunit.xml
drwxr-xr-x   6 olgamorenomartin  admin    192 16 jul 16:39 public
drwxr-xr-x   5 olgamorenomartin  admin    160 16 jul 16:39 resources
drwxr-xr-x   4 olgamorenomartin  admin    128 16 jul 16:39 routes
drwxr-xr-x   5 olgamorenomartin  admin    160 16 jul 16:39 storage
drwxr-xr-x   5 olgamorenomartin  admin    160 16 jul 16:39 tests
drwxr-xr-x  40 olgamorenomartin  admin   1280  2 sep 18:57 vendor
-rw-r--r--   1 olgamorenomartin  admin    263 16 jul 16:39 vite.config.js
olgamorenomartin@MacBook-Pro-de-Olga example %
```

instalación en local

- Para configurar nuestra app en el entorno local tenemos que editar .env

```
inicial — nano .env — 102x28
UW PICO 5.09 File: .env

APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:qhkgi79Azu+0YA0UC19jcma1S0DWwjJdMYgVWVbD0Cc=
APP_DEBUG=true
APP_URL=http://localhost

LOG_CHANNEL=stack
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

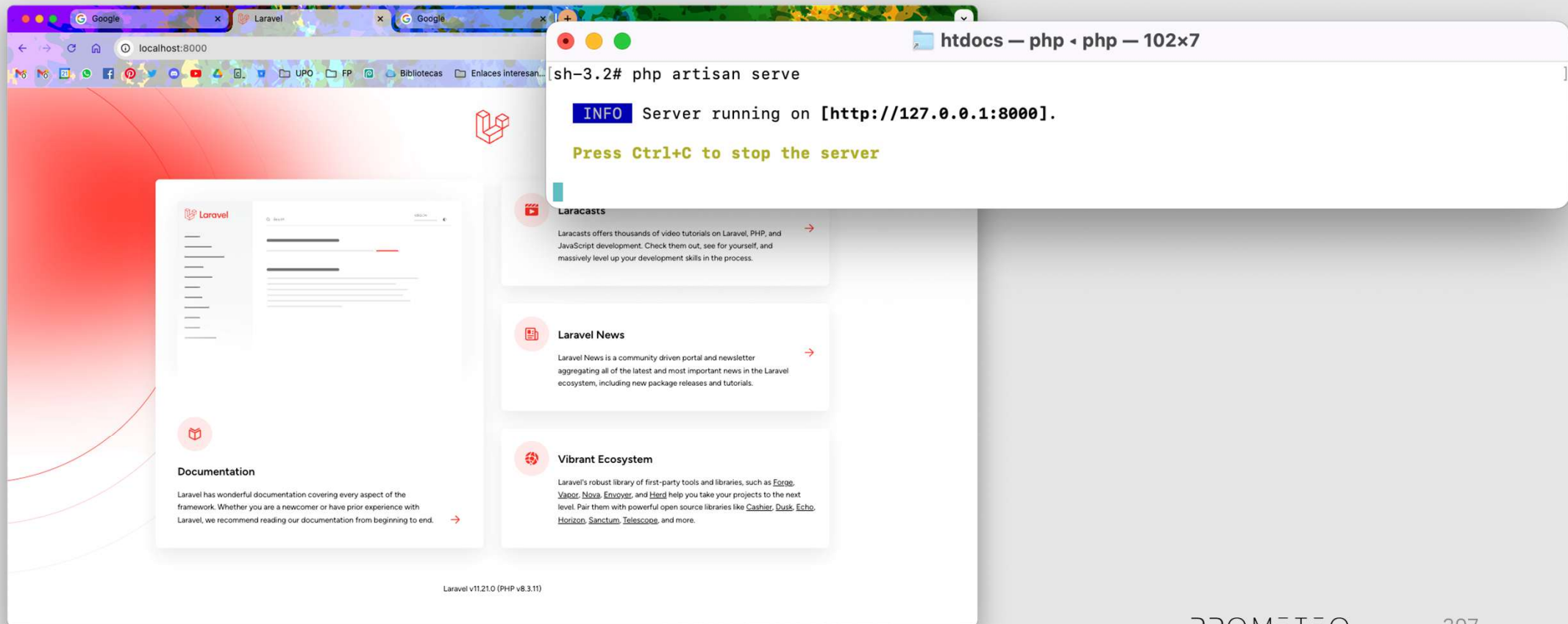
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=

BROADCAST_DRIVER=log
CACHE_DRIVER=file
FILESYSTEM_DISK=local
QUEUE_CONNECTION=sync
SESSION_DRIVER=file
SESSION_LIFETIME=120

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg  ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is   ^V Next Pg  ^U UnCut Text ^T To Spell
```


instalación en local

- Para lanzar el servidor podemos usar:



Instalación en local

- Ahora entramos en la carpeta database/migrations, vemos que se han creado unos archivos.

```
migrations — -zsh — 102x9
/Applications/XAMPP/htdocs — php ◀ php
...tdocs/example-app/database/migrations —
Last login: Tue Jan 24 14:49:42 on ttys000
olga@MacBook-Pro-de-olga htdocs % cd example-app
olga@MacBook-Pro-de-olga example-app % cd database/migrations
olga@MacBook-Pro-de-olga migrations % ls
2014_10_12_000000_create_users_table.php
2014_10_12_100000_create_password_resets_table.php
2019_08_19_000000_create_failed_jobs_table.php
2019_12_14_000001_create_personal_access_tokens_table.php
olga@MacBook-Pro-de-olga migrations %
```

El *unique* impide que se le añadan emails iguales a dos usuarios diferentes. Esto incluye que no se le puede dar un dato vacío a ambos.

rememberToken es un código modificado al entrar y salir, para evitar accesos indeseados.

Timestamps permite a Laravel identificar el orden de las migraciones.

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });
    }
}
```


Instalación en local

```
example-app — -zsh — 67x31
...ns/XAMPP/htdocs — php • php
...docs/example-app — -zsh +

[olga@MacBook-Pro-de-olga migrations % php artisan migrate
Could not open input file: artisan
[olga@MacBook-Pro-de-olga migrations % cd ..
[olga@MacBook-Pro-de-olga database % cd ..
[olga@MacBook-Pro-de-olga example-app % php artisan migrate

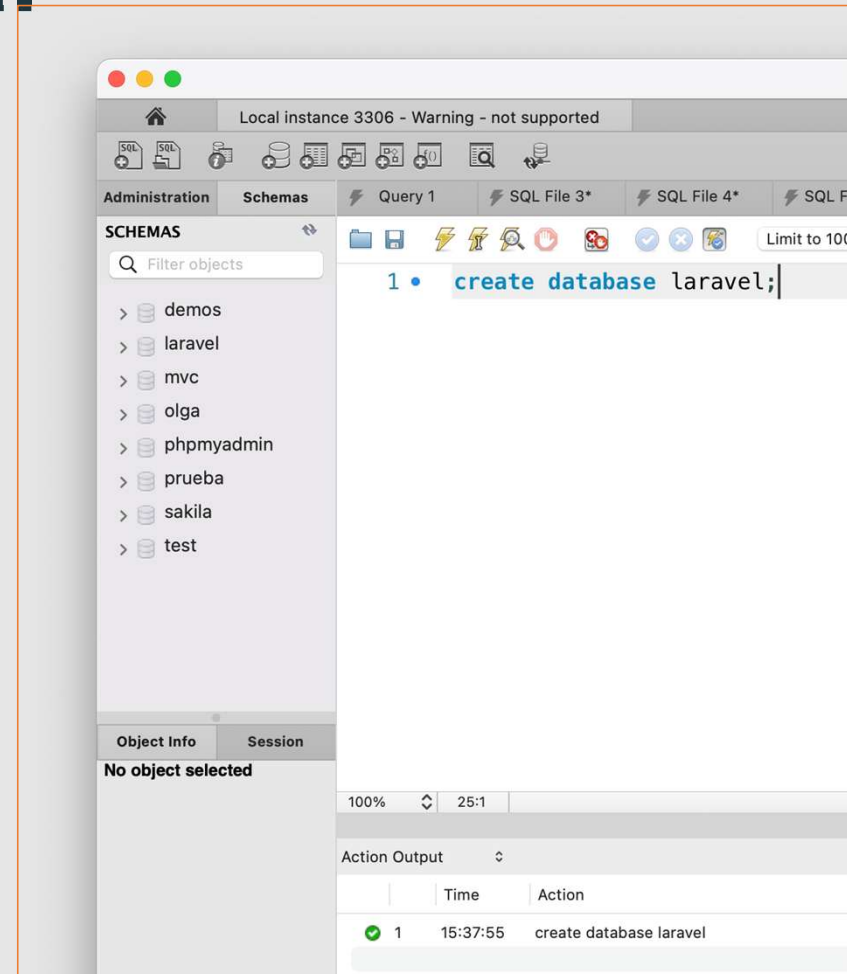
INFO Preparing database.

Creating migration table ..... 39ms DONE

INFO Running migrations.

2014_10_12_000000_create_users_table ..... 41ms DONE
2014_10_12_100000_create_password_resets_table ..... 42ms DONE
2019_08_19_000000_create_failed_jobs_table ..... 30ms DONE
2019_12_14_000001_create_personal_access_tokens_table 35ms DONE

olga@MacBook-Pro-de-olga example-app %
```



Migraciones

Con las migraciones vamos a gestionar la base de datos de nuestro sitio web; tanto crear nuevas BBDD como editarlas desde Laravel.

Las migraciones de un sitio hecho con Laravel se alojan en la ruta `database/migrations` y tienen extensión `.php`.

Si te fijas, nada más instalar Laravel, se han creado varios archivos de migraciones que podemos usar, pero de momento los dejamos ahí y NO LOS BORRAMOS porque son muy útiles.

Archivos `.env`

Es de uso común trabajar con archivos de entorno llamados también archivos `.env`. Normalmente, en un proyecto real puedes encontrarte con varios archivos de este tipo en función del despliegue que se quiera hacer; como por ejemplo:

- `test.env` config. para desplegar en entorno seguro de pruebas
- `release.env` config. para desplegar cambios de versión que se envía a los beta testers
- `production.env` config. para desplegar el código ya probado al resto del mundo.

En nuestro caso, como no vamos a desplegar nada, sólo vamos a usar un único archivo `.env` que vimos como utilizar en la instalación. En este archivo debemos configurar los datos de nuestro servidor MySQL y rellenarlo con la información correspondiente a nuestra base de datos **ya creada**.

Migraciones

- Una vez tengamos esto, lo que nos queda es ejecutar el comando de las migraciones a través del CLI `artisan`
- Si todo ha salido bien obtendremos el siguiente resultado donde podremos observar que todas las migraciones se han insertado correctamente en la base de datos.

```
olga@MacBook-Pro-de-olga example-app % php artisan migrate

INFO: Preparing database.

Creating migration table ..... 39ms DONE

INFO: Running migrations.

2014_10_12_000000_create_users_table ..... 41ms DONE
2014_10_12_100000_create_password_resets_table ..... 42ms DONE
2019_08_19_000000_create_failed_jobs_table ..... 30ms DONE
2019_12_14_000001_create_personal_access_tokens_table 35ms DONE
```

- Si nos vamos al cliente que utilizemos para manejar la base de datos (Workbench, por ejemplo) veremos que en nuestra base de datos se han creado todas las tablas de la migración que hemos ejecutado y además una tabla que se llama migrations.
- La tabla migrations es simplemente un registro de todas las migraciones llevadas a cabo. Además, podemos hacer un rollback en caso de que queramos reestablecer nuestra base de datos.
- O si preferimos hacer un reset para dejarla limpia:

`php artisan migrate:rollback`

`php artisan migrate:reset`

Carpetas en Laravel

Al crear un nuevo proyecto con este framework, Laravel crea una serie de carpetas por defecto. Esta estructura de carpetas es la recomendada para utilizar Laravel.

Public

- Esta es la carpeta más importante ya que es donde se ponen todos los archivos que el cliente va a mostrar al usuario cuando introduzcamos la URL de nuestro sitio web. Normalmente se carga el archivo `index.php` por defecto.

Routes

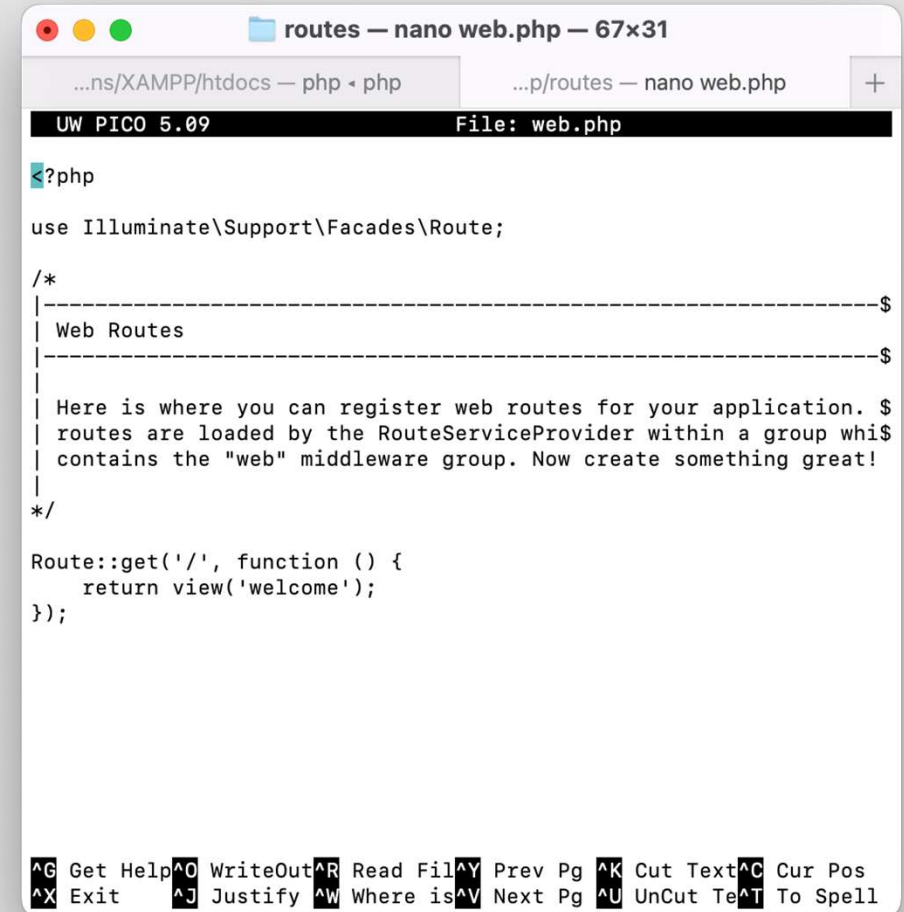
- Otra de las carpetas que más vamos a usar a lo largo de este curso de Laravel. En ella se albergan todas las rutas (redirecciones web) de nuestro proyecto, pero más concretamente en el archivo `web.php`
- Dada una ruta → se cargará una vista

Resources

- Esta es nuestra carpeta de recursos donde guardaremos los siguientes archivos, que también, están separados por sus carpetas... como cada nombre indica:
- `css` Archivos CSS
- `js` Archivos JS (JavaScript)
- `lang` Archivos relacionados con el idioma del sitio (variables & strings)
- `views` Archivos de nuestras vistas, lo que las rutas cargan

Rutas

- Las rutas en Laravel (y en casi cualquier Framework) sirven para redireccionar al cliente (o navegador) a las vistas que nosotros queramos.
- Estas rutas se configuran en el archivo `public/routes/web.php` donde se define la ruta que el usuario pone en la URL después del dominio y se retorna la vista que se quiere cargar al introducir dicha dirección en el navegador.
- En el ejemplo de arriba vamos a cargar la vista llamada `welcome` que hace referencia a la vista `resources/views/welcome.blade.php`



```
...ns/XAMPP/htdocs — php < php    ...p/routes — nano web.php +
File: web.php
?php
use Illuminate\Support\Facades\Route;

/*
|-----$
| Web Routes
|-----$
| Here is where you can register web routes for your application. $
| routes are loaded by the RouteServiceProvider within a group whi$
| contains the "web" middleware group. Now create something great!
|
*/

Route::get('/', function () {
    return view('welcome');
});

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Te ^T To Spell
```

rutas – alias

- Es interesante darle un alias o un nombre a nuestras rutas para poder utilizar dichos alias en nuestras plantillas de Laravel que veremos más adelante.
- Para ello, basta con utilizar la palabra name al final de la estructura de la ruta y darle un nombre que queramos; normalmente descriptivo y asociado a la vista que tiene que cargar el enrutador de Laravel.

```
<?php
```

```
Route::get('/users', function () {  
    return view('users');  
}) -> name('usuarios');
```

```
<?php
```

```
/* http://localhost/datos/ */  
Route::view('datos', 'usuarios');
```

- Después veremos que es muy útil ya que a la hora de refactorizar o hacer un cambio, si tenemos enlaces o menús de navegación que apuntan a esta ruta, sólo tendríamos que cambiar el parámetro dentro del get() y no tener que ir archivo por archivo.
- Laravel nos proporciona una manera más cómoda a la hora de cargar una vista si no queremos parámetros ni condiciones. Tan sólo definiremos la siguiente línea que hace referencia la ruta datos en la URL y va a cargar el archivo usuarios.php de nuestra carpeta views como le hemos indicado en el segundo parámetro.

rutas – pArámetros

- Ya hemos visto que con PHP podemos pasar parámetros a través de la URL, como si fueran variables, que las recuperábamos a través del método GET o POST.
- Con Laravel también podemos introducir parámetros pero de una forma más vistosa y ordenada, de tal manera que sea visualmente más cómodo de recordar y de indexar por los motores de búsqueda como Google.
- <http://localhost/cliente/324>
- Para configurar este tipo de rutas en nuestro archivo de rutas public/routes/web.php, tenemos que considerar algunas cosas: ¿Qué pasa si no introducimos un id y sólo navegamos hasta cliente/ ? ... Nos va a devolver un 404.
- Para resolver esto, podemos definir una ruta por defecto en caso de que el id (o parámetro) no sea pasado. Para ello usaremos el símbolo ? en nuestro nombre de ruta e inicializaremos la variable con el valor que queramos. También podremos agregar una expresión regular para obligar al parámetro a cumplir con ser un número.

```
<?php
```

```
Route::get('cliente/{id?}', function($id = 1) {  
    return ('Cliente con el id: ' . $id);  
}) -> where('id', '[0-9]+');
```

Estructuras de control

- Como en todo buen lenguaje de programación, en Laravel también tenemos estructuras de control.
- En Blade (plantillas de Laravel) siempre que iniciemos un bloque de estructura de control DEBEMOS cerrarla
- @foreach ~ @endforeach lo usamos para recorrer arrays
- @if ~ @endif para comprobar condiciones lógicas
- @switch ~ @endswitch en función del valor de una variable ejecutar un código
- @case define la casuística del switch
- @break rompe la ejecución del código en curso
- @default si ninguna casuística se cumple
- Acordaros que podemos pasar variables a través de las rutas como si fueran parámetros. Pero en este caso, vamos a ver otra directiva más; el uso de @compact.

```
<?php

$equipo = ['María', 'Alfredo', 'William', 'Verónica'];

@foreach ($equipo as $nombre)
    <p> {{ $nombre }} </p>
@endforeach
```

```
<?php

// Uso de @compact
$equipo = ['María', 'Alfredo', 'William', 'Verónica'];

// Route::view('nosotros', ['equipo' => 'equipo']);
Route::view('nosotros', @compact('equipo'));
```


plantillas o templates

A través de las plantillas de Laravel vamos a escribir menos código PHP y vamos a tener nuestros archivos mejor organizados.

Blade es el sistema de plantillas que trae Laravel, por eso los archivos de plantillas que guardamos en el directorio de views llevan la extensión **blade.php**.

De esta manera sabemos inmediatamente que se trata de una plantilla de Laravel y que forma parte de una vista que se mostrará en el navegador.

Directivas

Laravel tiene un gran número de directivas que podemos utilizar para ahorrarnos mucho código repetitivo entre otras funciones.

Digamos que las directivas son pequeñas funciones ya escritas que aceptan parámetros y que cada una de ellas hace una función diferente dentro de Laravel.

- **@yield** Define el contenido dinámico que se va a cargar. Se usa conjuntamente con **@section**. Es decir, que lo que declaramos con **@yield** luego lo “rellenamos” con **@section**.
- **@section** y **@endsection** bloque de código dinámico
- **@extends** importa el contenido de una plantilla ya creada para poder utilizarla

Ejercicio 1

1. Crea un sitio web con Laravel que contenga el título "Bienvenidos a Laravel, DAW", un texto de bienvenida (puede ser un poco de Lorem Ipsum) y a continuación un menú de navegación con sus correspondientes alias y los siguientes enlaces:

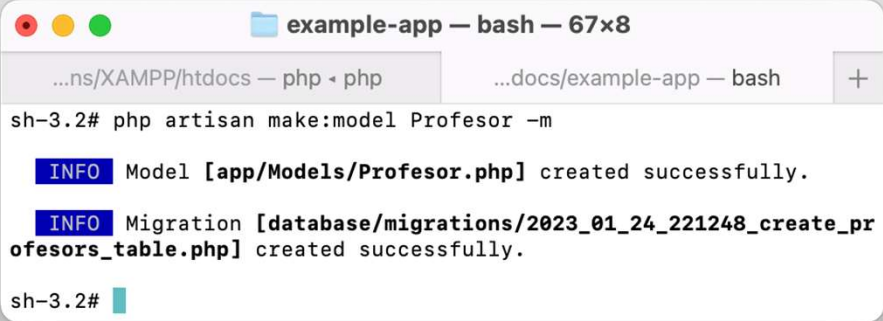
- a) Inicio enlace a la página principal donde se visualizará el texto de Lorem Ipsum además de los elementos estáticos (Título y menú de navegación).
- b) ¿Quiénes somos? enlace que vaya a la página "nosotros" y muestre, además de los elementos estáticos de todo el sitio, un h2 que diga "Estás en la sección sobre nosotros".
- c) Proyecto enlace que cargue una vista con el siguiente texto "Estás en el proyecto número: X" donde X es un número entero que podemos introducirlo en la propia ruta. Si no se mete ningún número en la ruta, por defecto tiene que ser 1; por ejemplo

- <http://localhost:8000/proyecto/210937>

Recuerda que el título y el menú de navegación han de aparecer en todas las vistas que cargues. (Usa las plantillas)

modelos

- Gracias a Eloquent y su integración con Laravel, podremos crear modelos de datos de una manera automatizada a través de artisan
- Ahora que ya sabemos manejar las migraciones es hora de crear nuestras propias migraciones pero a través de Eloquent.
- A través de la instrucción `make:model` creamos un nuevo modelo de datos, a continuación ponemos el nombre **siempre empezando en Mayúscula y en SINGULAR** y pasamos el parámetro relacionado con las migraciones `-m`.
`php artisan make:model Profesor -m`
- Si todo ha salido bien, veremos en nuestro directorio de migraciones `database/migrations` un nuevo archivo que se llama `2022_01_07_81237_create_profesor_table.php`.



```
example-app — bash — 67x8
...ns/XAMPP/htdocs — php < php
...docs/example-app — bash +
sh-3.2# php artisan make:model Profesor -m

[INFO] Model [app/Models/Profesor.php] created successfully.
[INFO] Migration [database/migrations/2023_01_24_221248_create_profesors_table.php] created successfully.
sh-3.2#
```

- El siguiente paso es **ver nuestro archivo de migraciones y editarlo** para que contenga las tablas que nosotros queramos. Si lo visualizamos tan sólo tendrá la estructura básica con un par de tablas. Ejemplo: notas

modelos

- En esta Schema podemos poner todas las tablas que nosotros queramos y establecer el tipo de dato para cada una de ellas, así como el nombre.
- Una vez que estamos satisfechos con nuestro esquema debemos volver a ejecutar el código de migración a través de artisan para que se introduzca esta nueva información en la base de datos.
- Para ver todos los tipos de datos que maneja Eloquent puedes visitar la documentación oficial.

<https://laravel.com/docs/11.x/migrations#available-column-types>

modelos – relaciones con eloquent

A través de Eloquent vamos a poder gestionar las relaciones entre nuestras tablas de la base de datos de una manera muy sencilla y sin sentencias SQL.

Uno a uno (1 a 1)

- Para crear este tipo de relaciones en Eloquent y Laravel, debemos tener creadas las tablas que vayamos a relacionar y establecer la relación entre ellas a través del método `hasOne`.
- Supongamos que tenemos una tabla usuario que está relacionada con la tabla teléfono.
- En este caso, Eloquent asume que en Usuario existe la FK `user_id` pero ¿qué pasa si tenemos otro nombre? Pues se lo pasamos como parámetro.
- Si la PK no se llama id, también debemos pasarla por parámetro.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * Get the phone associated with the user.
     */
    public function phone()
    {
        return $this->hasOne(Phone::class);
    }
}

return $this->hasOne(Phone::class, 'foreign_key', 'local_key');
```

modelos – relaciones con eloquent

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Phone extends Model
{
    /**
     * Get the user that owns the phone.
     */
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

```
return $this->belongsTo(User::class, 'foreign_key', 'owner_key');
```

Uno a Uno INVERSA

Ahora que podemos acceder al modelo teléfono desde el modelo usuario, vamos a ver cómo hacerlo de manera inversa, es decir, cómo acceder desde el modelo usuario desde el modelo teléfono gracias al método `belongsTo()`.

modelos – relaciones con eloquent

Uno a Muchos (1 a M)

- En este caso, las relaciones de 1 a muchos podemos decir que en una entrada de un blog, o en un post de Facebook, hay muchos comentarios relacionados a esa misma publicación.
- Para empezar, ya sabemos que debemos crear el modelo y en este caso usaremos el método `hasMany()` para obtener los datos relacionados con ese post o entrada en el blog.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    /**
     * Get the comments for the blog post.
     */
    public function comments()
    {
        return $this->hasMany(Comment::class);
    }
}
```

modelos – relaciones con eloquent

Uno a Muchos (1 a M)

- Cuidado con las claves ajenas, que aquí pasa lo mismo...
- Eloquent establece por defecto el sufijo `_id` por lo tanto, en este ejemplo buscaría por `post_id`. Si no queremos eso o nuestra clave ajena tiene otro nombre, se lo pasamos por parámetro en el método `hasMany` como hacíamos más arriba.
- Ahora, al haber más de un dato, necesitamos iterar, por tanto debemos crear un bucle para poder sacar cada dato.
- Además, como todas las relaciones son sentencias SQL, podemos anidar varios filtros en función de lo que queramos sacar.

```
return $this->hasMany(Comment::class, 'foreign_key');  
  
return $this->hasMany(Comment::class, 'foreign_key', 'local_key');
```

```
use App\Models\Post;  
  
$comments = Post::find(1)->comments;  
  
foreach ($comments as $comment) {  
    //  
}
```

```
$comment = Post::find(1)->comments()  
    ->where('title', 'foo')  
    ->first();
```


modelos – relaciones con eloquent

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    /**
     * Get the post that owns the comment.
     */
    public function post()
    {
        return $this->belongsTo(Post::class);
    }
}
```

Uno a Muchos INVERSA

- Ahora que podemos acceder a todos los comentarios de una publicación, definamos una relación para permitir que un comentario acceda a su publicación principal.
- Y ahora, a través de la propiedad de relación dinámica...
- Pasaría lo mismo con el nombre de la clave ajena, si no se llama de la misma manera que Eloquent establece con el sufijo _id podemos pasarle como parámetro el nombre de la clave donde debe buscar.

```
use App\Models\Comment;

$comment = Comment::find(1);

return $comment->post->title;
```

modelos – relaciones con eloquent

Muchos a Muchos (N a M)

- Este tipo de relaciones son las más complicadas ya que, en un Blog del estilo Wordpress por ejemplo, un usuario puede tener muchos roles (lector, autor, administrador) pero un rol pueden tenerlo varios usuarios, es decir, puede haber muchos usuarios administradores, otros lectores y demás.
- Para realizar este tipo de relaciones necesitaríamos 3 tablas diferentes.
- Lo primero de todo, vamos a crear las tablas con sus modelos a excepción de la tabla pivote `role_user` que sólo crearemos la tabla, sin su modelo.

php artisan make:migration
create_role_user_table --create=role_user

```
users
  id - integer
  name - string

roles
  id - integer
  name - string

role_user
  user_id - integer
  role_id - integer
```

```
$table->id();
$table->unsignedInteger('user_id');
$table->unsignedInteger('role_id');
$table->timestamps();
```

modelos – relaciones con eloquent

Muchos a Muchos (N a M)

- Recordad User.php ya existe.
- Y ahora, a través de la propiedad de relación dinámica...

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    /**
     * The roles that belong to the user.
     */
    public function roles()
    {
        return $this->belongsToMany(Role::class);
    }
}
```

```
return $this->belongsToMany(Role::class)->withTimestamps();
```

```
return $this->belongsToMany(Role::class, 'role_user', 'user_id', 'role_id');
```

```
use App\Models\User;

$user = User::find(1);

foreach ($user->roles as $role) {
    //
}
```

```
$roles = User::find(1)->roles()->orderBy('name')->get();
```

(ACONSEJABLE)

modelos – relaciones con eloquent

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Role extends Model
{
    /**
     * The users that belong to the role.
     */
    public function users()
    {
        return $this->belongsToMany(User::class);
    }
}
```

Muchos a Muchos INVERSA

- Para definir el "inverso" de una relación de muchos a muchos, debemos establecer un método en el modelo relacionado que también devuelva el resultado del método belongsToMany. Según el ejemplo que estamos siguiendo...
- Si usamos la declaración (ACONSEJABLE) tendremos el atributo **pivot** disponible con los timestamps, recordad, hay veces que queremos guardar más atributos en la tabla pivote.

```
use App\Models\User;

$user = User::find(1);

foreach ($user->roles as $role) {
    echo $role->pivot->created_at;
}
```

Ejercicio 2

2. Vamos a hacer un ejemplo con una APP que gestione alumnos y asignaturas, de tal manera que MUCHOS ALUMNOS pueden cursar MUCHAS ASIGNATURAS

¿Qué necesitamos para este ejemplo?

- 3 migraciones para crear las tablas
- Alumnos /// Materias /// AlumnoMateria
- Modificar los archivos de las migraciones `create_alumnos_table` y `create_materias_table`.
- Ejecutar las Migraciones
- 2 modelos para Alumnos y Materias
- Método dentro de Alumno para crear la relación Alumno -> Materia y en materia para saber que alumnos las tienen (la inversa).
- Rellenar la base de datos

Controladores

- Los controladores son el lugar perfecto para definir la lógica de negocio de nuestra aplicación o sitio web.
- Hace de intermediario entre la vista (lo que vemos con nuestro navegador o cliente) y el servidor donde la app está alojada.
- Por defecto, los controladores se guardan en una carpeta específica situada en `app/Http/Controllers` y tienen extensión `.php`. Se llaman igual que el modelo al que referencian pero con **nombre en PLURAL**, con la **primera en MAYÚSCULAS**.
- Para crear un controlador nuevo debemos hacer uso de nuestro querido artisan donde le diremos que cree un controlador con el nombre que nosotros queramos.
- Abrimos la consola y nos situamos en la raíz de nuestro proyecto, como siempre:
`php artisan make:controller PagesController`
- Si todo ha salido bien, recibiremos un mensaje por consola con que todo ha ido bien y podremos comprobar que, efectivamente se ha creado el archivo `PagesController.php` con una estructura básica de controlador, dentro de la carpeta `Controllers` que hemos descrito anteriormente.
- Ahora podemos modificar nuestro archivo de rutas `web.php` para dejarlo limpio de lógica y trasladar ésta a nuestro nuevo controlador.
- La idea de esto es dejar el archivo `web.php` tan limpio como podamos para que, de un vistazo, se entienda todo perfectamente.
- RECUERDA que sólo movemos la lógica, mientras que las cláusulas como `where` y `name` las seguimos dejando en el archivo de rutas `web.php`

miniejemplo:

```
<body>
  <!-- CONTENIDO ESTÁTICO PARA TODAS LAS SECCIONES -->
  <h1>Bienvenid@s a Laravel</h1>
  <hr>

  <!-- MENÚ -->
  <nav>
    <a href={{ route('noticias') }}>Blog</a> |
    <a href={{ route('galeria') }}>Fotos</a>
  </nav>

  <!-- CONTENIDO DINÁMICO EN FUNCIÓN DE LA SECCIÓN QUE SE VISITA -->
  <header>
    @yield('apartado')
  </header>
</body>
```

```
<?php
// blog.blade.php

@extends('plantilla')

@section('apartado')
  <h2>Estás en BLOG</h2>
@endsection
```

```
<?php
// fotos.blade.html

@extends('plantilla')

@section('apartado')
  <h2>Estás en FOTOS</h2>
@endsection
```

```
<?php

// web.php

Route::view('blog', 'blog') -> name('noticias');
Route::view('fotos', 'fotos') -> name('galeria');
```

```
// Uso de @compact
$equipo = ['María', 'Alfredo', 'William', 'Verónica'];

// Route::view('nosotros', ['equipo' => 'equipo']);
Route::view('nosotros', @compact('equipo'));
```

Falta el
nosotros.blade.php

```
$equipo = ['María', 'Alfredo', 'William', 'Verónica'];

@foreach ($equipo as $nombre)
  <p> {{ $nombre }} </p>
@endforeach
```

miniejemplo:

```
<?php

// PagesController.php

namespace App\Http\Controllers;

class PagesController extends Controller
{
    public function inicio() { return view('welcome'); }

    public function datos() {
        return view('usuarios', ['id' => 56]);
    }

    public function cliente($id = 1) {
        return ('Cliente con el id: ' . $id);
    }

    public function nosotros($nombre = null) {
        $equipo = [
            'Paco',
            'Enrique',
            'Maria',
            'Veronica'
        ];

        return view('nosotros', @compact('equipo', 'nombre'));
    }
}
```

```
<?php

// web.php (v2.0) Refactorizado

use App\Http\Controllers\PagesController;
use Illuminate\Support\Facades\Route;

Route::get('/', [ PagesController::class, 'inicio' ]);
Route::get('datos', [ PagesController::class, 'datos' ]);
Route::get('cliente/{id?}', [ PagesController::class, 'cliente' ] -> where('id', '[0-9]+'));
Route::get('nosotros/{nosotros?}', [ PagesController::class, 'nosotros' ] -> name('nosotros'));
```


Ejercicio 3

Partiendo del ejercicio 2:

- a) Crear el controlador para la vista
- b) Crear la ruta de nuestra vista
- c) Crear la vista con los datos

Formularios – previa

Para trabajar con formularios, vamos a usar un modelo muy sencillo.

- Le vamos a llamar Nota, y va a tener los atributos nombre y descripción.

```
php artisan make:model Nota -m  
php artisan make:controller NotasController
```

- En la migration:

```
<?php
```

```
Schema::create('notas', function (Blueprint $table) {  
    $table->id();  
    $table->timestamps();  
  
    $table->string('nombre');  
    $table->text('descripcion');  
});
```

Formularios – previa

- Una vez creado el controlador vamos a routes/web.php:

```
Route::get('notas', [ NotasController::class, 'notas' ] );
```

- Luego editamos el controlador:

```
public function notas() {  
    $notas = Nota::all(); // Nos saca todas las notas de la BBDD  
    return view('notas', @compact('notas'));  
}
```

Ahora, creamos la vista con la tabla que pinte los datos de la variable 'notas' de la ruta.

Formularios – previa

```
// notas.blade.php
<h1>Notas desde base de datos</h1>
<table border="1">
  <thead>
    <tr>
      <th>Nombre</th>
      <th>Descripción</th>
    </tr>
  </thead>
  @foreach ($notas as $nota)
    <tr>
      <td>{{$nota -> nombre}}</td>
      <td>{{$nota -> descripcion}}</td>
    </tr>
  @endforeach
</table>
```

Formularios – previa

- Hay que fijarse bien en los nombres de las columnas que tienen nuestras bases de datos, es justo lo que va después de -> y siempre rodeado por los símbolos {{ }} ya que estamos en un archivo de plantilla.
- ¿Qué pasaría si sólo queremos acceder a un único elemento? como si hiciésemos un `SELECT * from usuarios where id = 1`
- Para eso, tenemos una instrucción específica en Eloquent que nos soluciona el problema. En este caso usaremos la instrucción `findOrFail` y como buenos usuarios de Laravel, lo utilizaremos dentro del controlador `NotasController.php`.

```
public function detalle($id) {  
    $nota = Nota::findOrFail($id);  
    return view('notas.detalle', @compact('nota'));  
}
```

- Hay que acordarse que debemos configurar la ruta en nuestro archivo de rutas.
`Route::get('notas/{id?}', [ NotasController::class, 'detalle' ]) -> name('notas.detalle');`

Y por último, debemos crear la plantilla, pero como es un archivo de detalle o que está relacionado con otra plantilla ya creada, podemos crear una carpeta con el nombre de la plantilla y dentro, el archivo de plantilla en cuestión.

Formularios – previa

```
//detalle.blade.php
```

```
<h1>Detalle de la nota</h1>
```

```
<h3>ID: {{ $nota -> id }}</h3>
```

```
<h3>Nombre: {{ $nota -> nombre }}</h3>
```

```
<h3>Descripción: {{ $nota -> descripcion }}</h3>
```

Formularios

Ahora que ya sabemos cómo cargar de una base de datos, vamos a ver cómo insertarlos con Laravel y sin escribir ni una sola línea de SQL. Gracias al método `save()` de Laravel podremos guardar datos que provengan de un formulario desde nuestras plantillas. Para ello, lo primero que necesitamos lo siguiente:

- formulario HTML que recoja los datos que el usuario introduce
- Una ruta que sea la encargada de recibir los datos del formulario
- Método POST para enviar los datos al servidor
- Un método en nuestro controlador que procese los datos y los guarde a través de `save()`
- La cláusula de seguridad `@csrf` para evitar ataques desde otros sitios

Así pues, empecemos por el formulario:

```
<form action="{{ route('notas.crear') }}" method="POST">
  @csrf {{-- Cláusula para obtener un token de formulario al enviarlo --}}

  <input type="text" name="nombre" placeholder="Nombre de la nota" class="form-control mb-2" autofocus>
  <input type="text" name="descripcion" placeholder="Descripción de la nota" class="form-control mb-2">
  <button class="btn btn-primary btn-block" type="submit">
    Crear nueva nota
  </button>
</form>
```

Formularios

- Como vemos, creamos 2 inputs relacionados con nuestras columnas dentro de la tabla, en este caso nombre y descripción
- El action del formulario debe apuntar a una nueva ruta que vayamos a crear y donde enviemos los datos mediante POST.
- Ahora crearemos la ruta en nuestro archivo de rutas web.php

```
Route::post('notas', [ NotasController::class, 'crear' ]) -> name('notas.crear');
```

- Si nos fijamos, ya no estamos haciendo uso del get sino del método post y como son métodos diferentes, podemos nombrar la ruta de la misma manera que en get ya que no habrá conflicto.
- Por otro lado, necesitamos invocar nuestro NotasController y decirle que vamos a utilizar el método crear que bueno, todavía no existe pero que vamos a crear a continuación. No olvidemos crear un alias para poder vincularlo al action del formulario SUPER IMPORTANTE.
- Para terminar, editaremos nuestro archivo NotasController.php para que el controlador que estamos usando tenga el método que hemos nombrado previamente.
-

Formularios

```
// estamos en NotasController.php
public function crear(Request $request) {
    $notaNueva = new Nota;
    $notaNueva -> nombre = $request -> nombre;
    $notaNueva -> descripcion = $request -> descripcion;
    $notaNueva -> save();

    return back() -> with('mensaje', 'Nota agregada exitosamente');
}
```

Nuestro método crear va a recibir un objeto llamado `$request` (el cual podemos cambiarle el nombre perfectamente) de tipo Request por lo que **DEBEMOS IMPORTAR Request** para poder utilizar dicha clase de Laravel.

```
use Illuminate\Http\Request;
```

Formularios

- Dentro del método crear que acabamos de escribir, debemos hacer uso del modelo Nota que ya creamos en otros ejemplos e instanciarla a través de una variable, en nuestro caso \$notaNueva.
- De esta manera, podremos acceder a los métodos y propiedades de la misma, por eso podemos utilizar nombre y descripción.
- Así mismo, la variable \$request está formada por los mismos nombres de input que hemos puesto en el formulario de la plantilla, si os acordáis eran name="nombre" y name="descripcion".
- Finalmente usamos el método save() para guardar estos nuevos datos en la base de datos y retornamos a la página del formulario con el método back() añadiendo un mensaje con with().
- Pero ¿dónde va a salir este mensaje? -- lo tenemos que declarar en nuestra plantilla

```
@if (session('mensaje'))  
    <div class="mensaje-nota-creada">  
        {{ session('mensaje') }}  
    </div>  
@endif
```

Validaciones

- Laravel nos proporciona herramientas para poder validar los datos que el usuario introduce en los campos del formulario.
- Además de poder hacerlo con la etiqueta required de HTML5, debemos validar los datos a través del Framework.
- Para ello, necesitamos modificar varios elementos:
- En primer lugar, nuestro archivo controller
- En segundo lugar, nuestra plantilla que carga el formulario
- Empecemos con el controlador. A través del método validate() le decimos a Eloquent qué campos son requeridos para poder enviar el formulario. Utilizaremos para ello un array asociativo con el nombre del input y la palabra reservada required

```
$request -> validate([ 'nombre' => 'required', 'descripcion' => 'required' ]);
```

Seguidamente nos moveremos a la vista donde esté el formulario y a través de la directiva @error crearemos un bloque html con nuestro mensaje de error por cada uno de los inputs requeridos.

```
@error('nombre') <div class="alert alert-danger"> No olvides rellenar el nombre </div> @enderror
```

Validaciones

- Pero ¿qué pasa cuando ha habido un error y nos muestra el mensaje que hemos escrito? Si te fijas, los campos que habías rellenado perderán la información, pero con Laravel podemos persistirlos sin hacer que el usuario vuelva a introducirlos.
- Para poder persistir los datos una vez enviados pero con algún error de campo requerido, utilizaremos la directiva `old()` como value del input dentro de nuestro formulario y le pasaremos el nombre del input declarado en la etiqueta name.

```
// estamos en notas.blade.php
```

```
<input  
  type="text"  
  name="nombre"  
  value="{{ old('nombre') }}"  
  class="form-control mb-2"  
  placeholder="Nombre de la nota"  
  autofocus >
```

Editando registros

- Después de tener campos en la base de datos, lo interesante sería poder editarlos a través de un formulario.
- Laravel nos proporciona las herramientas necesarias para ello; veamos pues lo que necesitamos para poder realizar el cambio a través de la directiva put().
- Para poder hacer el cambio de registros necesitamos lo siguiente:
- Un enlace para redirigir a la página de editar, pasando el id del elemento en cuestión
- Una nueva ruta que apunte a nuestra plantilla de editar
- Una plantilla para poder editar con un formulario que reciba los datos a editar
- Una nueva función dentro de nuestro controlador para poder manejar los datos ya introducidos
- Una nueva ruta que utilice el método put() para poder actualizar los datos
- Un nuevo método dentro de nuestro controlador para actualizar los datos nuevos introducidos

```
<a href="{{ route('notas.editar', $nota) }}" class="btn btn-warning btn-sm"> Editar </a>
```

Editando registros

- Ahora creamos la ruta:

```
Route::get('editar/{id}', [ NotasController::class, 'editar' ]) -> name('notas.editar');  
Route::put('editar/{id}', [ NotasController::class, 'actualizar' ]) -> name('notas.actualizar');
```

Necesitamos una nueva vista:

- resources/views/notas/editar.blade.php

```
<h2>Editando la nota {{ $nota -> id }}</h2> @if (session('mensaje')) <div class="alert alert-  
success">{{ session('mensaje')}}</div> @endif <form action="{{ route('notas.actualizar', $nota ->  
id) }}" method="POST"> @method('PUT') {{-- Necesitamos cambiar al método PUT para editar --}}  
@csrf {{-- Cláusula para obtener un token de formulario al enviarlo --}} @error('nombre') <div  
class="alert alert-danger"> El nombre es obligatorio </div> @enderror @error('descripcion') <div  
class="alert alert-danger"> La descripción es obligatoria </div> @enderror <input type="text"  
name="nombre" class="form-control mb-2" value="{{ $nota -> nombre }}" placeholder="Nombre  
de la nota" autofocus > <input type="text" name="descripcion" placeholder="Descripción de la  
nota" class="form-control mb-2" value="{{ $nota -> descripcion }}" > <button class="btn btn-  
primary btn-block" type="submit">Guardar cambios</button> </form>
```

Editando registros

- Y por último, modificamos nuestro Controlador:

```
public function editar($id) {  
    $nota = Nota::findOrFail($id);  
    return view('notas.editar', compact('nota'));  
}  
  
public function actualizar(Request $request, $id) {  
    $request -> validate([  
        'nombre' => 'required',  
        'descripcion' => 'required'  
    ]);  
  
    $notaActualizar = Nota::findOrFail($id);  
    $notaActualizar -> nombre = $request -> nombre;  
    $notaActualizar -> descripcion = $request -> descripcion;  
    $notaActualizar -> save();  
    return back() -> with('mensaje', 'Nota actualizada');  
}
```

Eliminando Registros

- A la hora de eliminar un registro nuevo, no necesitamos crear una plantilla nueva ya que podemos mandar la instrucción directamente a través de otro formulario.
- Por lo tanto, para eliminar un registro de la base de datos utilizaremos lo siguiente.
- Un formulario básico con un único botón de eliminar
- Usaremos el método DELETE para sobrescribir el método del formulario HTML
- Una ruta nueva para controlar el action de este nuevo formulario
- Un nuevo método dentro de nuestro Controlador que lleve la lógica para borrar el registro
- Vamos a ver cómo meter ese formulario dentro de nuestro listado de notas

```
// estamos en notas.blade.php
```

```
<form action="{{ route('notas.eliminar', $nota) }}" method="POST" class="d-inline">  
  @method('DELETE')  
  @csrf  
  <button class="btn btn-danger btn-sm" type="submit">Eliminar</button>  
</form>
```


Eliminando Registros

- Ahora que ya tenemos montado el formulario en nuestra plantilla, pasemos a crear la ruta que hemos colocado en el action del formulario para borrar elementos.

```
Route::delete('eliminar/{id}', [ NotasController::class, 'eliminar' ]) -> name('notas.eliminar');
```

- El último paso que nos queda es modificar el Controller:

```
public function eliminar($id) {  
    $notaEliminar = Nota::findOrFail($id);  
    $notaEliminar -> delete();  
  
    return back() -> with('mensaje', 'Nota Eliminada');  
}
```

Si todo ha salido bien, habremos creado un sitio en Laravel y Eloquent que es capaz de hacer un CRUD validando campos en formularios e insertando datos reales en una base de datos.

Paginación

- Para añadir paginación a nuestros resultados, Eloquent tiene un método que se llama `paginate()` donde le pasamos un número entero como parámetro para indicarle el número de resultados que queremos por página.

```
public function notas() {  
    // $notas = Nota::all();  
    $notas = Nota::paginate(5);  
  
    return view('notas', compact('notas'));  
}
```

En la vista usamos: `{{ $notas->links() }}`

- Ahora veremos ciertos elementos HTML que se han generado en nuestra vista, esto es porque Laravel hace uso de una librería de paginación situada en la carpeta `vendor/laravel/framework/src/illuminate/Pagination`.
- Para usar Bootstrap para paginar debemos ir a `App\Providers` y a `ServiceProvider` y añadir en el método `boot()` `Paginator::useBootstrap()`; e importar `Paginator`.

Bootstrap + vite



Para trabajar con la parte visual de Laravel podemos interactuar con Bootstrap, pero necesitamos gestionar la dependencia con Vite.

Vite (palabra en francés para "rápido", pronunciado como /vit/, como "veet") es una herramienta de compilación que tiene como objetivo proporcionar una experiencia de desarrollo más rápida y ágil para proyectos web modernos.

`composer require laravel/ui -dev`

`php artisan ui bootstrap --auth`

`npm install bootstrap-icons --save-dev`

Abrir el archivo `resources\sass\app.scss` y añadir:

`@import 'bootstrap-icons/font/bootstrap-icons.css';`

`npm install & npm run dev & npm run build`

Comprueba:

- Abre el archivo `vite.config.js` que se encuentra en la raíz del proyecto y cambia la referencia `resources/css/app.css` por `resources/scss/app.scss`

Abre el archivo `resources/js/app.js` y agrega el siguiente código: `import './bootstrap'`

Para poderlo usar con libertad en nuestras vistas (`blade.php`) `@vite(['resources/js/app.js', 'resources/scss/app.scss'])` en el `<head>`

```
// Fonts
@import url('https://fonts.bunny.net/css?family=Nunito');

// Variables
@import 'variables';

// Bootstrap
@import 'bootstrap/scss/bootstrap';
@import 'bootstrap-icons/font/bootstrap-icons.css';
```